

Milan Hladík

Celočíselné Programování

text k přednášce

6. března 2026



Tento text zatím slouží jako doprovodný text přednášky Celočíslné programování na studiu informatiky MFF UK v Praze.

Autor čerpal především z knih Nemhauser and Wolsey [1999]; Schrijver [1998]; Wolsey [1998]. Další klasické knihy jsou např. Korbut and Finkelstein [1971]; Taha [1975]. O historii pěkně pojednává soubor esejů Grötschel [2012].

Děkuji mnoha svým studentům za cenný feedback, včetně upozornění na drobné chyby či nejasné formulace.

Případné připomínky a chyby zasílejte prosím na adresu hladik@kam.mff.cuni.cz.

Obsah

Obsah	5
I Teorie a metody	7
1 Teorie	8
1.1 Úvod	8
1.2 Celočíselný polyedr	10
1.3 Unimodularita – kdy složité se stává jednodušším	13
1.4 Složitost podrobně	17
1.5 Umění formulace celočíselných programů	21
2 Technika reformulace a linearizace (RLT)	27
3 Metody sečných nadrovin aneb řeznictví pana Gomoryho	35
3.1 Duální simplexová metoda	35
3.2 Lexikografie	36
3.3 ℓ -metoda	37
3.4 První Gomoryho algoritmus	40
3.5 Druhý Gomoryho algoritmus	46
3.6 Další řezy	49
3.6.1 Integer rounding a Chvátalovy–Gomoryho řezy	49
3.6.2 Generování logických nerovností	50
3.6.3 Perfektní párování a květinové nerovnosti	51
4 Metody branch & bound	53
4.1 Procházení výpočetního stromu	57
4.2 Volba proměnné pro větvení	57
4.3 Další varianty	58
4.4 Implementační záležitosti	59
4.5 Aktuální trendy	60
4.6 Software	60
5 Heuristiky	63
5.1 Heuristiky pro nalezení přípustného řešení	63
5.2 Heuristiky pro nalezení dobrého řešení	64
6 Dekompozice	65
6.1 Bendersova dekompozice	65
6.2 Lagrangeova relaxace a dekompozice	68
7 Column generation	73

II	Speciální případy	75
8	Problém batohu	76
8.1	Složitost	77
8.2	Relaxace	79
8.3	Sečné nadroviny	80
8.4	Branch & cut	81
8.5	Preprocessing II.	82
8.6	Aplikace: sečné nadroviny pro 0-1 programování	82
9	Set covering	85
9.1	Branch & bound	86
9.2	Preprocessing	86
9.3	Hladový algoritmus	86
10	Problém obchodního cestujícího	89
10.1	Formulace	91
10.2	Sečné nadroviny	92
10.3	Relaxace a dolní odhady na optimální hodnotu	93
10.4	Heuristiky	95
10.5	Historie a výpočty	98
11	Facility location problem	101
11.1	Relaxace	102
11.2	Branch & bound	103
11.3	Heuristiky pro UFLP	103
11.4	Bendersova dekompozice pro UFLP	104
11.5	Lagrangeova relaxace pro UFLP	105
12	Softwarové úlohy	107
	Značení	109
	Literatura	111

Část I

Teorie a metody

Kapitola 1

Teorie

1.1 Úvod

V celočíselném programování se střetávají dva odlišné světy: spojitá a kombinatorická optimalizace. Střetávají se dvojitým způsobem. Jednak se formulaci problému, kde se vyskytují jak spojité proměnné a funkce, a jednak v metodách na řešení, kdy moderní algoritmy kombinují techniky z obou disciplín.

Celočíselné programování zažilo v minulých dekáдах intenzivní rozvoj a je stále oblastí s intenzivním výzkumem. Je to způsobeno tím, že obrovská řada praktických problémů se dá zformulovat jako celočíselný program. Přitom je celočíselné programování těžké na řešení, takže představuje výzvu pro další výzkum a vývoj a má potenciál na další posun.

Využití celočíselného programování je skutečně nepřeberné. Z praktických úloh to jsou především problémy operačního výzkumu a příbuzných oblastí. Tyto úlohy zahrnují problémy z rozvrhování, plánování, různé typy dopravních úloh, přiřazovací úlohy, hledání Nashova ekvilibria v teorii her, ekonomické úlohy (řízení zásob, model výroby, vytváření portfolia) atp.

Na celočíselné programování narazíme kdykoli pracujeme s veličinami a proměnnými, které nabývají pouze diskrétních hodnot. Představit si můžeme počet osob, které vykonají nějakou práci, počet automobilů, které chceme vyrobit, nebo počet telekomunikačních věží, které postavíme. Speciálním typem jsou binární proměnné, které udávají, jestli nějaká situace nastane nebo nikoli. Pomocí celočíselného lineárního programování můžeme modelovat nelineární nehladné funkce (jako je minimum, maximum, komplementarita, absolutní hodnota nebo obecněji po částech lineární funkce) či dokonce nespojité funkce (např. skokové ceny), nebo logické operace (disjunkce, implikace aj.).

Motivační příklady

Sílu celočíselného programování a co všechno umíme modelovat pomocí celočíselných podmínek nahlédneme v sekci 1.5. Stěžejní praktické modely celočíselného programování pak probíráme podrobně v kapitolách 8 (Problém batohu), 10 (Problém obchodního cestujícího) a 11 (Facility location problem). Než se k nim ale dostaneme, uvádíme několik ilustrativních příkladů.

Příklad 1.1 (MAX-CUT problem). MAX-CUT je úloha nalezení maximálního řezu v neorientovaném grafu $G = (V, E)$. Jinými slovy, chceme rozdělit vrcholy V na dvě netriviální části V_1, V_2 tak, aby počet hran mezi oběma partitami byl co největší.

Úlohu můžeme formulovat jako následující celočíselný lineární program:

$$\max \frac{1}{4} \sum_{\{i,j\} \in E} (1 - x_i x_j) \quad \text{za podm. } x_i \in \{\pm 1\}, \forall i \in V.$$

Je-li x^* optimální řešení, tak proměnné $x_i^* = 1$ odpovídají první partitě V_1 a proměnné $x_i^* = -1$ odpovídají druhé partitě V_2 . Koeficient $\frac{1}{4}$ vychází jednak z toho, že graf je neorientovaný, a tudíž každou hranu započítáme dvakrát, a jednak z toho, že výraz $(1 - x_i x_j)$ má hodnotu buď 0 nebo 2.

Tato formulace má výhodu zejména, pokud problém MAX-CUT aproximujeme pomocí semidefinitního programování. Existují ale i jiné formulace tohoto problému, jako například ta následující:

$$\begin{aligned} \max \quad & \sum_{\{i,j\} \in E} y_{ij} \quad \text{za podm.} \quad y_{ij} \leq x_i + x_j, \quad \forall \{i,j\} \in E, \\ & y_{ij} \leq 2 - x_i - x_j, \quad \forall \{i,j\} \in E, \\ & y_{ij} \in [0, 1], \quad \forall \{i,j\} \in E, \\ & x_i \in \{0, 1\}, \quad \forall i \in V. \end{aligned}$$

Binární proměnná x pak definuje hledané dvě partity.

Různé formulace mohou být výhodné pro různé účely, proto je výběr nejlepší formulace poměrně složitý problém.

Příklad 1.2 (Celočíselnost proměnných vs. bilineární podmínka). Celočíselnost proměnných jde přeformulovat jako bilineární podmínka a naopak. Například binární proměnnou x přeformulujeme takto:

$$x \in \{0, 1\} \iff x \in [0, 1] \wedge x(1 - x) = 0.$$

Naopak, bilineární rovnici $xy = 0$ se spojitými reálnými proměnnými $x, y \geq 0$ můžeme ekvivalentně přepsat jako

$$x \leq ab, \quad y \leq a(1 - b), \quad b \in \{0, 1\},$$

kde $a > 0$ je dost velká konstanta (zde předpokládáme, že máme omezený přípustný region pro proměnné, ale i tomu se dá vyhnout).

Z praktického hlediska je ale obvykle lepší se takovýmto transformacím vyhnout a využít původní formulaci, poněvadž transformace často vedou k degeneracím a slabým odhadům.

Klasifikace

V následujících odstavcích představíme základní typy úloh celočíselného programování.

Celočíselné programování. Úlohou celočíselného programování se rozumí úloha

$$\begin{aligned} \max \quad & f(x) \quad \text{za podm.} \quad g_j(x) \leq 0 \quad \forall j = 1, \dots, m, \\ & x_i \in \mathbb{Z} \quad \forall i \in C, \\ & x_i \in \mathbb{R} \quad \forall i \in \{1, \dots, n\} \setminus C, \end{aligned}$$

kde $C \subseteq \{1, \dots, n\}$. Jedná se tedy o úlohu matematického programování s podmínkami na celočíselnost některých proměnných. Pokud $C = \{1, \dots, n\}$, jde o čistou úlohu celočíselného programování (všechny proměnné celočíselné), jinak ji nazýváme smíšenou.

Lineární celočíselné programování. Jsou-li funkce $f(x), g_j(x)$ lineární, pak jde o lineární celočíselné programování

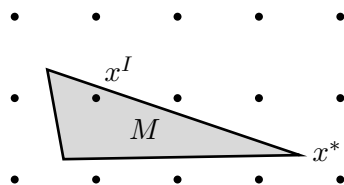
$$\begin{aligned} \max \quad & c^T x \quad \text{za podm.} \quad Ax \leq b \\ & x_i \in \mathbb{Z} \quad \forall i \in C. \end{aligned}$$

V tomto textu se budeme věnovat výhradně tomuto lineárnímu případu. Uvidíme, že i tato formulace je dost obecná a pokrývá širokou třídu úloh. Navíc techniky, které se naučíme pro lineární úlohy, se používají i pro nelineární úlohy.

0-1 programování. Speciálním typem lineárního celočíselného programu je 0-1 program s binárními proměnnými

$$\max \quad c^T x \quad \text{za podm.} \quad Ax \leq b, \quad x \in \{0, 1\}^n.$$

Na druhou stranu, zas tak speciálním typem není. Předpokládejme omezenost množiny přípustných řešení (srov. poznámka 1.26), tedy existuje $u > 0$ tak, že každé přípustné řešení x splňuje $|x_i| \leq u, \forall i$. Pak každou proměnnou x_i nahradíme binárním rozvojem $x_i = \sum_{k=0}^{\lfloor \log(u) \rfloor} 2^k y_{ki}$, kde $y_{ki} \in \{0, 1\}$ jsou binární proměnné. Celkový počet proměnných vzrostl $\lfloor \log(u) \rfloor$ -krát.



Obrázek 1.1: Ilustrace případu, kdy optimum x^* lineární relaxace (1.2) dá bod, který je daleko od skutečného celočíselného optima x^I . Navíc žádné rozumné zaokrouhlení bodu x^* nedá přípustné řešení.

Značení

Pro reálné číslo r používáme $\lfloor r \rfloor = \max\{z \in \mathbb{Z}; z \leq r\}$ pro jeho (dolní) celou část, $\lceil r \rceil = \min\{z \in \mathbb{Z}; z \geq r\}$ pro horní celou část a $\{r\} = r - \lfloor r \rfloor$ pro zlomkovou část.

1.2 Celočíselný polyedr

Mějme $A \in \mathbb{Q}^{m \times n}$, $b \in \mathbb{Q}^m$. Tento předpoklad sice je na újmu obecnosti (srov. poznámka 1.4), ale z praktického hlediska se vesměs setkáváme jen s racionálními daty, a reprezentovat na počítači můžeme také jen (některá) racionální čísla.

V této kapitole se nadále budeme zabývat čistou úlohou lineárního celočíselného programování

$$\max c^T x \text{ za podm. } x \in M \cap \mathbb{Z}^n, \quad (1.1)$$

kde

$$M = \{x \in \mathbb{R}^n; Ax \leq b, x \geq 0\}.$$

Bez podmínek celočíselnosti dostaneme lineární program

$$\max c^T x \text{ za podm. } x \in M. \quad (1.2)$$

Úloha (1.2) se nazývá *lineární*, nebo též *celočíselná relaxace* (celočíselná relaxace protože došlo k „uvolnění“ celočíselných podmínek a lineární relaxace, protože jsme dostali lineární podmínky). Jeho optimální hodnota dává horní odhad na optimální hodnotu (1.1), což je základem pro mnoho metod na řešení celočíselných programů. Nicméně, z optimálního řešení úlohy (1.2) určit optimum původní úlohy není jednoduché. Prosté zaokrouhlování zde nefunguje: jednak zaokrouhlením nemusíme dostat přípustné řešení, a jednak zaokrouhlené řešení může být stále dost daleko (libovolně daleko) od optima; viz obrázek 1.1.

Označme a pojmenujme jako *celočíselný polyedr*

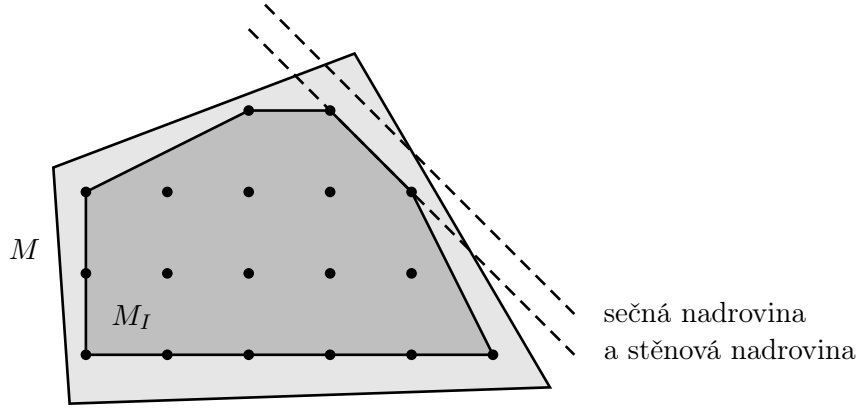
$$M_I := \text{conv}\{x \in \mathbb{Z}^n; Ax \leq b, x \geq 0\},$$

konvexní obal množiny přípustných řešení. Zřejmě platí $(M \cap \mathbb{Z}^n) \subseteq M_I \subseteq M$. Úloha (1.1) se pak redukuje na lineární program

$$\max c^T x \text{ za podm. } x \in M_I.$$

Potíž je, že sestavit množinu M_I je velmi těžké, až na speciální případy probírané v sekci 1.3. Můžeme se každopádně aspoň k M_I přiblížit tím, že množinu M budeme ořezávat přidáváním lineárních podmínek, které odstraní kus M , ale neodříznou nic z M_I . Takovéto podmínky se nazývají *řezy* nebo také *sečné nadroviny* v lineárním případě a tvoří základ metod probíraných v kapitole 3. Sečná nadrovina, která určuje facetu (stěnu dimenze $n - 1$) polyedru M_I se nazývá *stěnová* (ang. *facet-defining*), viz obrázek 1.2.

Následující věta potvrzuje, že M_I je konvexní polyedr, bez ohledu na to, je-li v omezení rovnice nebo nerovnice. Pro M omezené je to zřejmé z duálně nerovnicovo-vrcholového popisu polyedru, ale pro neomezený případ to zřejmé na první pohled není. Množina M_I je zjevně konvexní, ale případná potíž by mohla být s polyedralitou, protože podle definice je M_I konvexním obalem (potenciálně) nekonečně mnoha bodů.



Obrázek 1.2: Konvexní polyedr M , celočíselné body v něm a jejich konvexní obal M_I . Sečná a stěnová sečná nadrovina.

Věta 1.3 (Meyer, 1974). M_I je konvexní polyedr.

Důkaz. To, že M_I je konvexní, plyne z jeho definice. Zbývá tedy dokázat, že je to polyedr. Buďte v_i , $i \in I$, vrcholy a h_j , $j \in J$, směry neomezených hran M . Protože v_i a h_j mají racionální složky, můžeme bez újmy na obecnost předpokládat, že $h_j \in \mathbb{Z}^n$ (čehož docílíme přenásobením nejmenším společným násobkem jmenovatelů). Každý bod $x \in M$ lze psát ve tvaru

$$x = \sum_{i \in I} \alpha_i v_i + \sum_{j \in J} \beta_j h_j = \sum_{i \in I} \alpha_i v_i + \sum_{j \in J} \lfloor \beta_j \rfloor h_j + \sum_{j \in J} \{\beta_j\} h_j \quad (1.3)$$

pro určité $\alpha_i \geq 0$, $\sum_{i \in I} \alpha_i = 1$, $\beta_j \geq 0$. Označme

$$Q := \left\{ \sum_{i \in I} \alpha_i v_i + \sum_{j \in J} \beta_j h_j; \alpha_i \geq 0, \sum_{i \in I} \alpha_i = 1, 0 \leq \beta_j \leq 1 \right\},$$

$$C := \left\{ \sum_{j \in J} \beta_j h_j; \beta_j \geq 0 \right\}.$$

Zde Q je omezený konvexní polyedr a C konvexní polyedrický kužel. Polyedr Q reprezentuje omezenou část M a C jeho neomezené směry. Pak můžeme psát

$$M = Q + C := \{x + y; x \in Q, y \in C\}.$$

Vlastnost kužele C je ta, že $M + C = M$.

Naším cílem bude ukázat, že $M_I = Q_I + C$, viz obrázek 1.3.

„ $\subseteq$ “ Buď $x \in M \cap \mathbb{Z}^n$. Podle (1.3) je

$$x - \sum_{j \in J} \lfloor \beta_j \rfloor h_j = \sum_{i \in I} \alpha_i v_i + \sum_{j \in J} \{\beta_j\} h_j \in Q$$

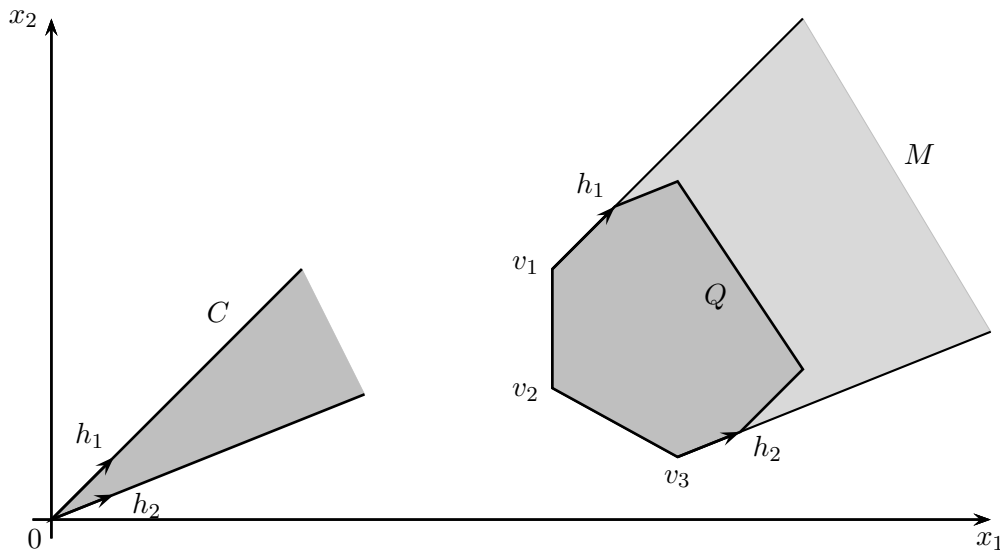
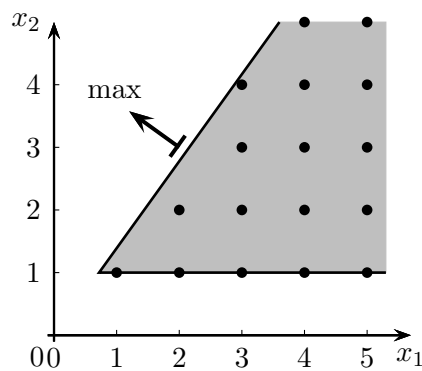
celočíslný vektor, a tudíž $x \in (Q \cap \mathbb{Z}^n + C) \subseteq (Q_I + C)$. Tím pádem z konvexity $Q_I + C$ dostáváme $M_I = \text{conv}(M \cap \mathbb{Z}^n) \subseteq (Q_I + C)$.

„ $\supseteq$ “ Platí $Q_I + C \subseteq M_I + C = M_I + C_I \subseteq (M + C)_I = M_I$, srov. cvičení 1.2. $\square$

Poznámka 1.4. Bez podmínek racionality A , b věta neplatí, např. pro

$$\max -\alpha x_1 + x_2 \text{ za podm. } -\alpha x_1 + x_2 \leq 0, x_2 \geq 1, x \in \mathbb{Z}^2,$$

kde $\alpha > 0$ je iracionální, viz obrázek 1.4. V tomto případě M_I není konvexní polyedr, protože má nekonečně mnoho hran. Tato úloha navíc nemá optimální řešení, přestože je přípustná a omezená (tj., účelová funkce je omezená). Důvod je ten, že na stěně $\alpha x_1 - x_2 = 0$ se nenachází žádný celočíselný bod, ale celočíselnými body se ke stěně můžeme přiblížit libovolně přesně. Supremum účelové funkce je tudíž 0, ale hodnota se nenabyde. Takováto situace pro racionální data nastat nemůže.

Obrázek 1.3: Rozdělení $M = C + Q$, důkaz věty 1.3.

Obrázek 1.4: Nekonečný polyedr, poznámka 1.4.

Poznámka 1.5. Důsledek z důkazu věty: Je-li $M_I \neq \emptyset$, pak M a M_I mají stejný kužel neomezených směrů, a tudíž úloha (1.1) je neomezená právě tehdy, když je neomezená (1.2).

Na druhou stranu, může nastat i patologický případ, kdy $M \neq \emptyset$ je neomezená, ale $M_I = \emptyset$. Pak stále platí $M_I = Q_I + C$, jenomže $Q_I = \emptyset$ a díky tomu i $M_I = \emptyset$. Příkladem může být množina

$$M = \{x \in \mathbb{R}^2; x_1 \geq 1, 0.2 \leq x_2 \leq 0.8\}.$$

Cvičení

- 1.1. Zobecněte větu 1.3 na smíšenou úlohu, obsahující celočíselné i spojité proměnné. To znamená, dokažte, že konvexní obal množiny $\{x \in \mathbb{R}^m, y \in \mathbb{Z}^n; Ax + By \leq c\}$ je konvexním polyedrem.
- 1.2. Rozhodněte zda pro konvexní polyedry P, Q platí $(P+Q)_I = P_I + Q_I$, popř. aspoň nějaká inkluze.
- 1.3. Najděte minimální nerovnicový popis celočíselného polyedru P_I pro P popsany nerovnostmi

$$\begin{pmatrix} -2 & -1 \\ 1 & -2 \\ -2 & 1 \\ 1 & 2 \end{pmatrix} x \leq \begin{pmatrix} -1 \\ 1 \\ 1 \\ 6 \end{pmatrix}.$$

- 1.4. Navrhněte nejlepší formulaci pro popis množiny $P_I = \text{conv}(\mathbb{Z}^2 \cap P)$, kde polyedr P tvoří množina řešení soustavy

$$\begin{pmatrix} -2 & -1 \\ 1 & -2 \\ -2 & 1 \\ 1 & 2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \leq \begin{pmatrix} -1 \\ 1 \\ 1 \\ 6 \end{pmatrix}.$$

- 1.5. Ukažte, že následující formulace celočíselných podmínek jsou ekvivalentní (popisují stejnou přípustnou množinu):

- $P_1 = \{x \in \{0, 1\}^4 : 12x_1 + 9x_2 + 6x_3 + 3x_4 \leq 14\}$,
- $P_2 = \{x \in \{0, 1\}^4 : 4x_1 + 3x_2 + 2x_3 + x_4 \leq 4\}$,
- $P_3 = \{x \in \{0, 1\}^4 : 4x_1 + 3x_2 + 2x_3 + x_4 \leq 4, x_1 + x_2 + x_3 \leq 1, x_1 + x_4 \leq 1\}$.

Která formulace je podle vás nejvhodnější pro modelování a řešení úlohy celočíselného lineárního programování? Proč?

1.3 Unimodularita – kdy složité se stává jednodušším

Nejprve připomeňme, že subdeterminant matice $A \in \mathbb{Z}^{m \times n}$ je determinant její podmatice, to jest takové matice, která vznikne z A odstraněním některých řádků a sloupců. Pokud matice A má hodnotu m , pak její sloupcová báze je regulární podmatice řádu m .

Definice 1.6 (Unimodularita a totální unimodularita).

- Matice $A \in \mathbb{Z}^{m \times n}$ je *totálně unimodulární*, pokud každý její subdeterminant je -1 , 0 či 1 .
- Matice $A \in \mathbb{Z}^{m \times n}$ hodnoty m je *unimodulární*, pokud každá její sloupcová báze má determinant -1 či 1 .

Sloupcovou bázi matice A budeme značit A_B , kde B je množina indexů dané báze. Toto značení používáme obecněji, tedy pro libovolnou indexovou množinu $J = \{j_1, \dots, j_k\}$ je $A_J = (A_{*j_1} \mid \dots \mid A_{*j_k})$; zde závisí na pořadí prvků v J .

Poznámka 1.7. Připomeňme některé věty pro práci s determinanty:

- *Laplaceův rozvoj podle k -tého sloupce:* $\det(A) = \sum_i (-1)^{i+k} a_{ik} \det(A^{ik})$, kde A^{ik} je matice A bez řádku i a sloupce k .
- *Cramerovo pravidlo:* Mějme soustavu rovnic $Ax = b$, kde A je regulární. Pak i -tá složka řešení je $x_i = \frac{\det(A_i^b)}{\det(A)}$, kde A_i^b je matice, která vznikne z A nahrazením i -tého sloupce za vektor b .

Vztah mezi unimodularitou a totální unimodularitou vystihuje následující tvrzení.

Věta 1.8. *Bud' $A \in \mathbb{Z}^{m \times n}$. pak A je totálně unimodulární právě tehdy, když $(A \mid I_m)$ je unimodulární.*

Důkaz. „ $\Rightarrow$ “ Bud' B sloupcová báze $(A \mid I)$, kterážto matice je zřejmě celočíselná. Pak z Laplaceova rozvoje podle sloupců I_B dostaneme $\det(A_B \mid I_B) = \pm \det(A'_B) = \pm 1$, kde A'_B je matice vzniklá z A_B odstraněním řádků, kde I_B má jedničky.

„ $\Leftarrow$ “ Bud' A' čtvercová podmatice A . Uvažme bázi B takovou, že A' je částí A_B a v řádcích, kde se nachází, I_B neobsahuje jedničky. Pak $\det(A') = \pm \det(A_B \mid I_B) = \pm 1$. Pokud by B nebyla báze, pak

$(A_B | I_B)$ by byla singulární a $\det(A') = \pm \det(A_B | I_B) = 0$. Schematicky:

$$A' = \begin{pmatrix} \mathbf{a}'_{11} & \cdots & \mathbf{a}'_{1k} \\ \vdots & \ddots & \vdots \\ \mathbf{a}'_{k1} & \cdots & \mathbf{a}'_{kk} \end{pmatrix}, \quad (A_B | I_B) = \left(\begin{array}{ccc|cccccc} a_{1b_1} & \cdots & a_{1b_k} & 1 & 0 & 0 & 0 & 0 & 0 \\ \vdots & & \ddots & 0 & \ddots & 0 & 0 & 0 & 0 \\ a_{ib_1} & \cdots & a_{ib_k} & 0 & 0 & 1 & 0 & 0 & 0 \\ \mathbf{a}'_{11} & \cdots & \mathbf{a}'_{1k} & 0 & 0 & 0 & 0 & 0 & 0 \\ \vdots & \ddots & \vdots & 0 & 0 & 0 & 0 & 0 & 0 \\ \mathbf{a}'_{k1} & \cdots & \mathbf{a}'_{kk} & 0 & 0 & 0 & 0 & 0 & 0 \\ a_{jb_1} & \cdots & a_{jb_k} & 0 & 0 & 0 & 1 & 0 & 0 \\ \vdots & \ddots & \vdots & 0 & 0 & 0 & 0 & \ddots & 0 \\ a_{mb_1} & \cdots & a_{mb_k} & 0 & 0 & 0 & 0 & 0 & 1 \end{array} \right) \quad \square$$

Z hlediska celočíselného programování je stěžejní následující věta, která říká, že úlohy s (totálně) unimodulární maticí můžeme řešit lineárním programováním a celočíselnost řešení je zaručena.

Věta 1.9 (Hoffman & Kruskal, 1956).

- (1) Buď $A \in \mathbb{Z}^{m \times n}$ hodnosti m . Pak $M = \{x \in \mathbb{R}^n; Ax = b, x \geq 0\}$ má všechny vrcholy celočíselné pro každé $b \in \mathbb{Z}^m$ právě tehdy, když A je unimodulární.
- (2) Buď $A \in \mathbb{Z}^{m \times n}$. Pak $M' = \{x \in \mathbb{R}^n; Ax \leq b, x \geq 0\}$ má všechny vrcholy celočíselné pro každé $b \in \mathbb{Z}^m$ právě tehdy, když A je totálně unimodulární.

Důkaz. (1) „ $\Rightarrow$ “ Buď B přípustná báze. Chceme dokázat, že $\det(A_B) = \pm 1$, neboli $\det(A_B^{-1}) = \det(A_B)^{-1} = \pm 1$. To je ekvivalentní s tvrzením, že A_B^{-1} má celočíselné složky: Je-li $A_B^{-1} \in \mathbb{Z}^{m \times m}$, pak z rovnosti $\det(A_B) \det(A_B^{-1}) = 1$ a celočíselnosti obou determinantů plyne $\det(A_B^{-1}) = \pm 1$. Naopak, je-li $\det(A_B^{-1}) = \pm 1$, tak adjungovaná matice $\text{adj}(A_B)$ je celočíselná a tedy $A_B^{-1} = \frac{1}{\det(A_B)} \text{adj}(A_B)$ jakbysmet.

Celočíselnost A_B^{-1} ukážeme po sloupcích. Uvažme i -tý sloupec $A_B^{-1}e_i$ a definujme $x := y + A_B^{-1}e_i \geq 0$, kde $y \geq 0$ je celočíselný a dostatečně velký vektor (například lze volit $y = |A_B^{-1}e_i|$). Nyní definujme $b := A_B x = A_B(y + A_B^{-1}e_i) = A_B y + e_i \in \mathbb{Z}^m$. Definujme vektor x' tak, že $x'_B = x$ a $x'_N = 0$. Nyní x' je vrchol polyedru M a podle předpokladu je celočíselný. Tudíž $A_B^{-1}e_i = x - y$ je také celočíselný.

„ $\Leftarrow$ “ Buď x vrchol M , příslušný bázi B . Označme $N := \{1, \dots, m\} \setminus B$. Pak $x_N = 0$ a $x_B = A_B^{-1}b$, neboli $A_B x_B = b$. Protože $\det(A_B) = \pm 1$, podle Cramerova pravidla jsou složky x_B celočíselné.

- (2) Přepíšeme M' do tvaru $M' = \{x \in \mathbb{R}^n; Ax + Ix' = b, x, x' \geq 0\}$ a použijeme tvrzení 1.8 s bodem (1). □

Celočíselné programování s (totálně) unimodulárními maticemi je tedy výpočetně jednoduché. Celočíselnost můžeme relaxovat a lineární program automaticky najde celočíselné optimum. Zde je potřeba pouze podotknout, že pokud není optimum jednoznačné, ale tvoří jej celá stěna, pak je celočíselným optimem libovolný vrchol této stěny.

Důsledek 1.10. Je-li matice A totálně unimodulární a vektor $b \in \mathbb{Z}^m$, pak úloha celočíselného programování

$$\max c^T x \quad \text{za podm. } Ax \leq b, x \geq 0, x \in \mathbb{Z}^n$$

je ekvivalentní s úlohou lineárního programování

$$\max c^T x \quad \text{za podm. } Ax \leq b, x \geq 0, x \in \mathbb{R}^n.$$

Ukážeme si dvě speciální třídy úloh, kde matice v omezeních je automaticky totálně unimodulární.

Definice 1.11. Buď $G = (V, E)$ neorientovaný graf. Pak matice incidence grafu G je matice $A_G \in \{0, 1\}^{|V| \times |E|}$ s prvky $(A_G)_{ve} = 1$ pro $v \in e$, a s nulami jinak.

Buď $G = (V, E)$ orientovaný graf. Pak matice incidence grafu G je matice $A_G \in \{0, \pm 1\}^{|V| \times |E|}$ s prvky $(A_G)_{ve} = -1$ pro $e = (v, u)$, $(A_G)_{ve} = 1$ pro $e = (u, v)$, a s nulami jinak.

Věta 1.12. *Buď A matice incidence neorientovaného grafu G . Pak A je totálně unimodulární právě tehdy, když G je bipartitní graf.*

Důkaz. „ $\Rightarrow$ “ Bez újmy na obecnost předpokládejme, že G neobsahuje izolovaný vrchol, protože nulový řádek neovlivní unimodularitu matice A ani bipartitnost grafu G . Podle věty 1.9 má polyedr $\{x \in \mathbb{R}^{|V|}; A^T x = e, x \geq 0\}$ celočíselné vrcholy. Protože je neprázdný (obsahuje vektor $\frac{1}{2}e$) a omezený (leží v nezáporném oktantu a součtem rovnic dostaneme omezení $a^T x = |V|$, kde $a > 0$), obsahuje celočíselné řešení x^* . Pak indexy s $x_i^* = 1$ odpovídají jedné partitě a ostatní té druhé.

„ $\Leftarrow$ “ Matematickou indukci podle velikosti podmatice. Podmatice řádu 1 mají zřejmě determinant 0 nebo 1. Nyní uvažujme podmatici A' řádu k . Jsou-li v každém sloupci dvě jedničky, pak součet řádků, odpovídajících první partitě, se rovná součtu řádků druhé partity, a tudíž $\det(A') = 0$. V opačném případě obsahuje matice A' sloupec s maximálně jednou jedničkou, na ten aplikujeme Laplaceův rozvoj a použijeme indukční předpoklad. $\square$

Příklad 1.13. Příkladem úlohy, kde se vyskytuje matice incidence bipartitního grafu je např. *dopravní problém*

$$\min \sum_{i=1}^m \sum_{j=1}^n c_{ij} x_{ij} \text{ za podm. } \sum_{j=1}^n x_{ij} = a_i \forall i, \sum_{i=1}^m x_{ij} = b_j \forall j, x \geq 0,$$

kde a_i , $i = 1, \dots, m$ jsou kapacity výrobců, b_j , $j = 1, \dots, n$ kapacity spotřebitelů, a c_{ij} cena přepravy jednotky zboží od i -tého výrobce k j -tému spotřebiteli. Zde se ještě navíc předpokládá, aby $\sum_i a_i = \sum_j b_j$. Nazývá se podmínkou ekonomické rovnováhy, a je přirozenou podmínkou, neboť bez ní by úloha neměla řešení.

Setřídíme-li proměnné $x_{11}, \dots, x_{1n}, x_{21}, \dots, x_{2n}, \dots, x_{m1}, \dots, x_{mn}$, potom matice soustavy rovnic z omezení

$$\begin{pmatrix} 1 & \dots & 1 & & & & & & \\ & & & 1 & \dots & 1 & & & \\ & & & & & & \dots & & \\ & & & & & & & 1 & \dots & 1 \\ 1 & & & & 1 & & & & 1 & \\ & \ddots & & & & \ddots & & \dots & & \ddots \\ & & & 1 & & & 1 & & & 1 \end{pmatrix}$$

je totálně unimodulární. $\square$

Následující pozorování prý nahlédl už Poincaré roku 1900.

Věta 1.14. *Matice incidence orientovaného grafu je totálně unimodulární.*

Důkaz. Analogicky důkazu věty 1.12. $\square$

Příklad 1.15. Příkladem úloh, kde se vyskytuje matice incidence orientovaného grafu jsou *toky v sítích*. Klasická úloha nalezení maximálního toku v síti (orientovaném grafu) $G = (V, E)$

$$\begin{aligned} \max v \text{ za podm. } & \sum_{j:(i,j) \in E} x_{ij} - \sum_{j:(j,i) \in E} x_{ji} = 0, \forall i \in V \setminus \{s, t\} \\ & \sum_{j:(s,j) \in E} x_{sj} - \sum_{j:(j,s) \in E} x_{js} = v, \\ & \sum_{j:(t,j) \in E} x_{tj} - \sum_{j:(j,t) \in E} x_{jt} = -v, \\ & 0 \leq x \leq u, \end{aligned}$$

kde $u: E \rightarrow \mathbb{R}_+$ jsou kapacity hran, $s \in V$ je počáteční vrchol a $t \in V$ koncový. Přidáme-li do grafu symbolickou hranu (t, s) , pak úlohu maximálního toku můžeme formulovat jako hledání maximálního

toku skrze přidanou hranu za podmínek, že pro každý vrchol platí podmínka rovnováhy (přítok = odtok). Je-li A matice incidence grafu G , pak tato úloha má tvar

$$\max x_{ts} \text{ za podm. } Ax = 0, 0 \leq x \leq u.$$

Podobnou úlohou je nalezení nejlevnějšího toku (*minimum-cost flow*)

$$\min \sum_{(i,j) \in E} c_{ij}x_{ij} \text{ za podm. } \sum_{j:(i,j) \in E} x_{ij} - \sum_{j:(j,i) \in E} x_{ji} = b_i \forall i \in V, \ell \leq x \leq u,$$

kde $\ell, u: E \rightarrow \mathbb{R}_+$ je dolní a horní mez na tok v jednotlivých hranách, c_{ij} cena přepravy jednotky zboží po hraně $(i, j) \in E$, a $b \in \mathbb{R}^{|V|}$ je vyrovnávací člen nabídky/poptávky. Maticově můžeme úlohu zapsat jako

$$\min \sum_{(i,j) \in E} c_{ij}x_{ij} \text{ za podm. } Ax = b, \ell \leq x \leq u,$$

kde A je opět matice incidence daného grafu. Této formulaci vyhovuje i situace, kdy je v grafu více zdrojů a stoků (počátečních a koncových vrcholů). $\square$

Příklad 1.16. Jiným příkladem úloh, kde se vyskytuje matice incidence orientovaného grafu je hledání *nejkratší cesty v grafu*. Uvažujme orientovaný graf $G = (V, E)$. Nechť $c: E \rightarrow \mathbb{R}_+$ jsou váhy na hranách a nechť $s \in V$ je počáteční vrchol a $t \in V$ koncový. Úlohu můžeme formulovat jako hledání nejlevnějšího toku velikosti 1

$$\begin{aligned} \min \sum_{(i,j) \in E} c_{ij}x_{ij} \text{ za podm. } & \sum_{j:(i,j) \in E} x_{ij} - \sum_{j:(j,i) \in E} x_{ji} = 0, \forall i \in V \setminus \{s, t\}, \\ & \sum_{j:(s,j) \in E} x_{sj} - \sum_{j:(j,s) \in E} x_{js} = 1, \\ & \sum_{j:(t,j) \in E} x_{tj} - \sum_{j:(j,t) \in E} x_{jt} = -1, \\ & 0 \leq x \leq 1. \end{aligned}$$

Poznamenejme, že pro neorientované grafy je situace trochu jiná v tom, že pokud povolíme záporné váhy na hranách nebo pokud hledáme nejdelší cestu, tak se problém stává NP-těžkým. Pro nejdelší cestu totiž nestačí změnit „min“ na „max“ ve formulaci úlohy, protože pak program nenajde nejdelší cestu, ale nejdelší tah. $\square$

Z hlediska maticových operací je totální unimodularita matice invariantní vůči transpozici, výměně řádků, mazání řádků, zdvojování řádků, násobení -1 , přidávání jednotkových a nulových řádků, konkatenace do blokově diagonálních matic atp.

Na konec této sekce si položíme otázku jak je těžké rozhodnout, zda daná matice $A \in \{0, \pm 1\}^{m \times n}$ je totálně unimodulární? Tuto otázku lze rozhodnout v polynomiálním čase metodou Seymourovy dekompozice z roku 1980 [Schrijver, 1998]. Ta je založená na faktu, že A je totálně unimodulární právě tehdy, když lze vytvořit z tzv. síťových matic (zobecnění matice incidence orientovaného grafu) a několika pevných matic pomocí výše zmíněných úprav a jím podobných.

Cvičení

1.6. Buďte $A, B \in \mathbb{Z}^{n \times n}$ unimodulární matice. Které z matic A^T , $A + B$, $A \cdot B$ jsou také unimodulární? Platí stejné vlastnosti pro transpozici, součet a součin totálně unimodulárních matic?

1.7. Buď $A \in \mathbb{Z}^{n \times n}$.

(a) Je-li A unimodulární, je také A^2 unimodulární?

(b) Je-li A totálně unimodulární, je také A^2 totálně unimodulární?

1.8. Buď $A \in \mathbb{Z}^{n \times n}$.

- (a) Je-li A unimodulární, je také A^{-1} unimodulární?
 (b) Je-li A regulární a totálně unimodulární, je také A^{-1} totálně unimodulární?
- 1.9. Ukažte, že pro testování totální unimodularity nestačí uvažovat pouze čtvercové podmatice, které tvoří po sobě následující řádky/sloupce dané matice.
- 1.10. Kolik je všech čtvercových podmatic matice $A \in \mathbb{R}^{n \times n}$, které v definici totální unimodularity uvažujeme?
- 1.11. Jsou následující matice (totálně) unimodulární?

$$A = \begin{pmatrix} -1 & 0 & 0 \\ 1 & 1 & 0 \\ 0 & -1 & -1 \end{pmatrix}, \quad B = \begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & -1 & 1 & 0 \\ 1 & 2 & 0 & -1 \\ 1 & 1 & -1 & 2 \end{pmatrix}, \quad C = \begin{pmatrix} 1 & 1 & 0 & -1 \\ -1 & 1 & 1 & 0 \\ 0 & -1 & -1 & 1 \\ 1 & 0 & 0 & 1 \end{pmatrix}$$

- 1.12. Je následující matice totálně unimodulární?

$$\begin{pmatrix} 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 \end{pmatrix}$$

- 1.13. Matice $A \in \{0, 1\}^{m \times n}$ je v intervalovém tvaru, pokud mají všechny řádky tvar

$$(0, \dots, 0, 1, \dots, 1, 0, \dots, 0),$$

tj. všechny hodnoty 1 následují za sebou v právě jednom souvislém intervalu. Ukažte, že každá matice v intervalovém tvaru je totálně unimodulární. (*Nápověda: Využijte elementární sloupcové úpravy.*)

- 1.14. Přepavní problém (transshipment problem) je modifikací klasického dopravního problému, ve které existují vnitřní uzly umožňující překládku zboží na cestě od dodavatelů ke spotřebitelům.
- (a) Formulujte přepravní problém jako úlohu toku v síti.
 (b) Formulujte přepravní problém jako dopravní problém.

1.4 Složitost podrobně

Nejprve se změříme na přípustnost celočíselného programování, tedy otázku rozhodnout, zda existuje (a případně najít) celočíselné přípustné řešení.

Věta 1.17 (Cook, 1971). *Celočíselné programování je silně NP-těžké.*

Důkaz. Na celočíselné programování lze převést řada NP-těžkých problémů, např.:

- Vrcholové pokrytí grafu $G = (V, E)$ lze zformulovat jako

$$\min \sum_{i \in V} x_i \quad \text{za podm.} \quad x_i + x_j \geq 1 \quad \forall \{i, j\} \in E, \quad x_i \in \{0, 1\} \quad \forall i \in V.$$

- SAT, splnitelnost booleovských formulí v konjunktivní normální formě. Zde logickým proměnným přísluší binární proměnné, a každou klauzuli přepíšeme jako lineární nerovnici. Např. klauzuli $V_1 \vee V_2 \vee \neg V_3$ přepíšeme na $v_1 + v_2 + (1 - v_3) \geq 1$. $\square$

Naším dalším cílem bude ukázat, že celočíselné programování je NP-úplný problém, tedy náleží do třídy NP. To je kupodivu ta těžší část důkazu, a musíme si pro to nejprve připravit půdu. Nebude to však jednoúčelová práce, neboť dostaneme v důsledku i jiné pěkné výsledky.

Definice 1.18 (Velikost zápisu). Buď $r \in \mathbb{Q}$, $v \in \mathbb{Q}^n$ a $A \in \mathbb{Q}^{m \times n}$, a necht' $r = \frac{p}{q}$ je v základním tvaru, tj. p, q nesoudělné a $p \in \mathbb{Z}$, $q \in \mathbb{N}$. Pak velikost jejich zápisu definujeme:

$$\begin{aligned}\sigma(r) &:= 1 + \lceil \log_2(|p| + 1) \rceil + \lceil \log_2(q + 1) \rceil, \\ \sigma(v) &:= n + \sum_i \sigma(v_i), \\ \sigma(A) &:= mn + \sum_{i,j} \sigma(a_{ij}) = \sum_i \sigma(A_{i*}) = \sum_j \sigma(A_{*j}).\end{aligned}$$

Velikost zápisu odpovídá počtu bitů, které jsou potřeba k zakódování daného racionálního čísla, vektoru resp. matice. Závisí samozřejmě na konkrétním výpočetním modelu a existují i jiné definice velikosti zápisu, ale liší se pouze o aditivní či multiplikativní konstantu. Tím pádem výsledky, které odvodíme, zůstávají v platnosti (s jinými koeficienty).

Věta 1.19 (Vlastnosti velikosti zápisu).

- (1) $\sigma(z) = 2 + \lceil \log_2(|z| + 1) \rceil$, $z \in \mathbb{Z}$,
- (2) $\sigma(a + b) \leq \sigma(a) + \sigma(b)$, $a, b \in \mathbb{Z}$,
- (3) $\sigma(ab) \leq \sigma(a) + \sigma(b)$, $a, b \in \mathbb{Q}$,
- (4) $\sigma(a/b) \leq \sigma(a) + \sigma(b)$, $a, b \in \mathbb{Q}$, $b \neq 0$,
- (5) $a < 2^{\sigma(a)}$, $a \in \mathbb{Q}$.

Důkaz. Viz cvičení 1.15. □

Věta 1.20. Buď $A \in \mathbb{Z}^{n \times n}$. Pak $\sigma(\det(A)) < \sigma(A)$.

Důkaz. Pro $n = 1$ platí tvrzení triviálně, tak předpokládejme $n \geq 2$. Za použití odhadu

$$\begin{aligned}|\det(A)| &= \left| \sum_{p \in S_n} \operatorname{sgn}(p) a_{1,p(1)} \cdots a_{n,p(n)} \right| \\ &\leq \sum_{p \in S_n} |a_{1,p(1)}| \cdots |a_{n,p(n)}| \leq \prod_{i,j=1}^n (|a_{ij}| + 1)\end{aligned}$$

a věty 1.19(1) odvodíme

$$\begin{aligned}\sigma(\det(A)) &\leq 2 + \left\lceil \log_2(1 + \prod_{i,j=1}^n (|a_{ij}| + 1)) \right\rceil \leq 2 + \left\lceil \log_2(2 \prod_{i,j=1}^n (|a_{ij}| + 1)) \right\rceil \\ &\leq 3 + \left\lceil \sum_{i,j=1}^n \log_2(|a_{ij}| + 1) \right\rceil \leq 3 + \sum_{i,j=1}^n \lceil \log_2(|a_{ij}| + 1) \rceil \\ &< n^2 + \sum_{i,j=1}^n (2 + \lceil \log_2(|a_{ij}| + 1) \rceil) = \sigma(A).\end{aligned}$$
□

Věta 1.21.

- (1) Buď $A \in \mathbb{Z}^{n \times n}$ regulární a buď $b \in \mathbb{Z}^n$. Pak řešení x^* soustavy $Ax = b$ splňuje

$$\sigma(x_i^*) < 2\sigma(A | b), \quad \forall i.$$

- (2) Buď $A \in \mathbb{Z}^{m \times n}$ hodnosti m , a buď $m < n$. Pak soustava $Ax = 0$ má nenulové řešení x^* splňující

$$\sigma(x_i^*) < 2\sigma(A), \quad \forall i.$$

Důkaz.

- (1) Podle Cramerova pravidla je $x_i^* = \det(A_i^b) / \det(A)$, kde matice A_i^b vznikne z A nahrazením i -tého sloupce vektorem b . Podle věty 1.19(4) a věty 1.20 je $\sigma(x_i^*) \leq \sigma(\det(A_i^b)) + \sigma(\det(A)) < \sigma(A_i^b) + \sigma(A) \leq 2\sigma(A | b)$.
- (2) Soustavu $Ax = 0$ můžeme po vhodné permutaci proměnných vyjádřit jako $A_1 x_1 + A_2 x_2 = 0$, kde A_1 je regulární. Všechna řešení jsou tvaru $x_1 = -A_1^{-1} A_2 x_2$, kde $x_2 \in \mathbb{R}^{n-m}$ je libovolné. Zvolme $x_2^* := e_1$. Pak x_1^* je řešením soustavy $A_1 x_1 = -(A_2)_{*1}$, takže podle první části věty mají všechny složky velikost menší než $2\sigma(A)$. □

Poznámka 1.22. Nadále budeme bez újmy na obecnost předpokládat, že v úloze celočíselného programování (1.1) jsou A , b , c celočíselné.

Důkaz. Buď i libovolné pevné a φ velikost i -tého řádku matice $(A|b)$. Nejmenší společný násobek jmenovatelů čísel $a_{i1}, \dots, a_{in}, b_i$ je nanejvýš 2^φ , neboť označíme-li tyto jmenovatele $q_1, \dots, q_{n+1}$, tak podle věty 1.19(5)

$$q_1 \cdots q_{n+1} \leq 2^{\sigma(q_1)+\dots+\sigma(q_{n+1})} \leq 2^{\sigma(a_{i1})+\dots+\sigma(a_{in})+\sigma(b_i)} \leq 2^\varphi.$$

Tudíž přenásobením nejmenším společným násobkem se velikost zápisů jmenovatelů nezvětší a velikost zápisu každého čitatele zvětší maximálně o φ . Velikost zápisu i -tého řádku se tak zvětší maximálně $(n+2)$ -krát. Tím pádem i velikost zápisu celé soustavy se zvětší maximálně $(n+2)$ -krát.

Podobně pro vektor c účelové funkce. □

Nyní připomeneme známou Carathéodoryho větu o tom, že konvexní obal v prostoru $\mathbb{R}^n$ lze zkonstruovat konvexními kombinacemi nanejvýš $n+1$ bodů. Druhá část věty se týká konvexního kužele, generovaného množinou S a definovaného jako

$$\text{conv cone}(S) = \left\{ \sum_{i=1}^k \alpha_i x_i; x_i \in S, \alpha_i \geq 0, k \in \mathbb{N} \right\}.$$

Je to tedy nejmenší konvexní kužel obsahující množinu S .

Věta 1.23 (Carathéodory, 1911).

- (1) Je-li $S \subseteq \mathbb{R}^n$, $x \in \text{conv}(S)$, pak $x \in \text{conv}\{x_0, \dots, x_n\}$ pro určité $x_0, \dots, x_n \in S$.
- (2) Je-li $S \subseteq \mathbb{R}^n$, $x \in \text{conv cone}(S)$, pak $x \in \text{conv cone}\{x_1, \dots, x_n\}$ pro určité $x_1, \dots, x_n \in S$.

Důkaz.

- (1) Nechť $x = \sum_{i=0}^m \alpha_i x_i$, kde $\alpha_i > 0$, $\sum_i \alpha_i = 1$. Pokud je $m \leq n$, jsme hotovi. Jinak uvažme $x_1 - x_0, x_2 - x_0, \dots, x_m - x_0$. Těchto vektorů je m v n -dimenzionálním prostoru, tudíž jsou lineárně závislé a existují $\beta_1, \dots, \beta_m$ takové, že alespoň jedna β_i je nenulová a $\sum_{i=1}^m \beta_i (x_i - x_0) = 0$. Položme $\beta_0 = -\sum_{i=1}^m \beta_i$. Pak $\sum_{i=0}^m \beta_i x_i = 0$ a $\sum_{i=0}^m \beta_i = 0$. Proto platí

$$x = \sum_{i=0}^m \alpha_i x_i = \sum_{i=0}^m \alpha_i x_i - \lambda \sum_{i=0}^m \beta_i x_i = \sum_{i=0}^m (\alpha_i - \lambda \beta_i) x_i \quad \forall \lambda \in \mathbb{R}.$$

Nyní můžeme zvolit λ tak, aby $\alpha'_i = \alpha_i - \lambda \beta_i$ byla všechna nezáporná a alespoň jedno nulové. Pak stále platí $\sum_{i=0}^m \alpha'_i = 1$, ale α'_i nenulových je méně než nenulových α_i . Takto můžeme dále pokračovat, dokud je vektorů x_i více než $n+1$.

- (2) Dokáže se obdobně jako (1). □

Věta 1.24. Buď $A \in \mathbb{Z}^{m \times n}$, $b \in \mathbb{Z}^m$ a označme φ největší velikost zápisu řádku matice $(A|b)$. Existuje-li celočíselný bod v $M = \{Ax \leq b, x \geq 0\}$, pak existuje celočíselný bod x^* takový, že $\sigma(x_i^*) < \Phi$ pro každé $i = 1, \dots, n$, kde

$$\Phi := 6n^2\varphi. \tag{1.4}$$

Důkaz. Podobné jako ve větě 1.3 můžeme každý bod $x \in M \cap \mathbb{Z}^n$ vyjádřit jako

$$x = \sum_{i \in I} \alpha_i v_i + \sum_{j \in J} \beta_j h_j,$$

kde v_i , $i \in I$, jsou vrcholy M ; h_j , $j \in J$, jsou směry neomezených hran M ; $\alpha_i \geq 0$, $\sum_{i \in I} \alpha_i = 1$, $\beta_j \geq 0$. Navíc předpokládejme, že h_j jsou celočíselné.

Podle Carathéodoryho věty 1.23 lze předpokládat, že maximálně $n+1$ z čísel α_i je nenulových a maximálně n z čísel β_j je nenulových. Tedy bez újmy na obecnosti nechť $|I| = n+1$, $|J| = n$.

Podle věty 1.21 je velikost složek vektorů v_i a h_j nanejvýš $2n\varphi$. Protože jsme ale předpokládali, že h_j jsou celočíselné, tak vynásobením nejmenším společným násobkem jmenovatelů se může velikost zvýšit

nanejvýš o $2n^2\varphi$ (srov. Poznámka 1.22). Tudíž velikost složek vektorů v_i a h_j odhadneme shora hodnotou $4n^2\varphi$. Pro bod $x^* \in M \cap \mathbb{Z}^n$ definovaný

$$x^* := \sum_{i \in I} \alpha_i v_i + \sum_{j \in J} \{\beta_j\} h_j$$

pak podle věty 1.19(5) platí

$$|x_k^*| \leq \sum_{i \in I} |(v_i)_k| + \sum_{j \in J} |(h_j)_k| < (2n+1)2^{4n^2\varphi} = 2^{4n^2\varphi + \log_2(2n+1)} \leq 2^{5n^2\varphi}.$$

Tím pádem podle věty 1.19(1)

$$\sigma(x_k^*) = 2 + \lceil \log_2(|x_k^*| + 1) \rceil \leq 2 + \lceil 5n^2\varphi \rceil < 6n^2\varphi. \quad \square$$

Poznamenejme, že odhady byly dost hrubé a je možno dosáhnout těsnějších odhadů s menšími koeficienty. Pro naše účely to však postačuje.

Věta 1.25. *Problém rozhodnout zda $M \cap \mathbb{Z}^n \neq \emptyset$ je NP-úplný.*

Důkaz. Podle věty 1.17 je problém NP-těžký a podle věty 1.24 náleží do třídy NP, neboť máme garanci existence certifikátu polynomiální velikosti (to je onen celočíselný bod). $\square$

Poznámka 1.26 (Neomezenost M_I). Bez újmy na obecnosti můžeme nadále předpokládat, že M_I je omezený. Podle věty 1.3 a jejího důkazu platí $M_I = Q_I + C$, tudíž všechny vrcholy M_I se vyskytují v Q_I . Pokud tedy optimální řešení x^* úlohy (1.1) existuje, tak podle důkazu věty 1.24 splňuje $|x_i^*| < 2^\Phi$. Případnou neomezenost lze proto testovat takto:

Řešme omezenou úlohu celočíselného programování (viz obrázek 1.5)

$$\max c^T x \text{ za podm. } x \in M \cap \mathbb{Z}^n, |x_i| \leq 2^\Phi \forall i.$$

Je-li tato úloha nepřipustná, pak je nepřipustná i původní úloha. Je-li x^* optimální řešení, pak je buďto optimálním řešením původní úlohy, nebo je původní úloha neomezená. Neomezenost pak rozhodneme pomocí pomocné úlohy lineárního programování

$$\max c^T x \text{ za podm. } x \in M.$$

Tato úloha je omezená právě tehdy, když je původní úloha omezená, viz poznámka 1.5.

Poznámka 1.27. Celočíselný program není v NP, protože to není rozhodovací úloha. Odpovídající rozhodovací problém má tvar

$$M \cap \{x; c^T x \geq c_0\} \cap \mathbb{Z}^n \neq \emptyset, \quad (1.5)$$

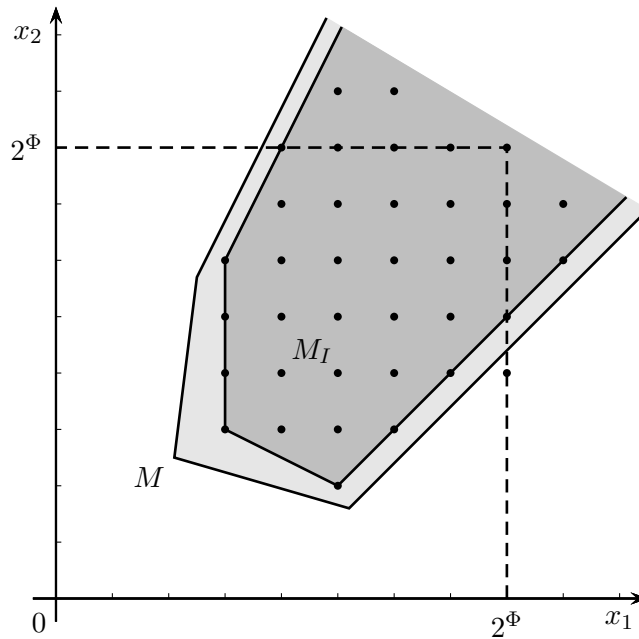
a vlastně se ptá, jestli existuje přípustné řešení s hodnotou účelové funkce aspoň c_0 . Tento problém už NP-úplný jest.

Vztah úloh (1.1) a (1.5) je úzký – kdybychom dokázali rozhodnout (1.5) v polynomiálním čase, tak dokážeme řešit rovněž celočíselné programování v polynomiálním čase. Důvod je následující. Protože podle (1.4) jsou složky optimálního řešení v absolutní hodnotě menší než 2^Φ , tak (celočíselná) optimální hodnota je ve vnitřku intervalu $[-\alpha, \alpha]$, kde $\alpha := 2^\Phi \sum_{i=1}^n |c_i|$. Tudíž pomocí binárního půlení parametru $c_0 \in [-\alpha, \alpha]$ a iterativním řešením rozhodovacích problémů (1.5) vyřešíme původní optimalizační úlohu po řádově $\log_2(2\alpha)$ iteracích, což je polynomiální hodnota vzhledem k velikosti vstupu.

Poznámka 1.28. Další NP-úplné problémy:

- Předchozí úlohy s polyedrem $M = \{x; Ax = b, x \geq 0\}$.
- Dáno $y \in \mathbb{Q}^n$ a M , patří $y \in M_I$?

Na druhou stranu, testování resp. nalezení celočíselného řešení soustavy rovnic $Ax = b$ již polynomiální problém je (tzv. diofantické rovnice). Další polynomiální případ je při pevné velikosti dimenze:

Obrázek 1.5: Neomezenost M_I , poznámka 1.26.

Věta 1.29 (Lenstra, 1983).

- (1) Pro každé $n \in \mathbb{N}$ pevné existuje polynomiální algoritmus, který rozhodne, zda polyedr $\{x \in \mathbb{R}^n; Ax \leq b\}$ obsahuje celočíselný bod, a pokud ho obsahuje, tak ho i najde.
- (2) Pro každé $n \in \mathbb{N}$ pevné existuje polynomiální algoritmus, který řeší úlohu celočíselného programování $\max \{c^T x; Ax \leq b, x \in \mathbb{Z}^n\}$. $\square$

Poznámka 1.30. Ne všechny celočíselné problémy jsou v třídě P či NP. Např. rozhodnout o celočíselné řešitelnosti polynomiální rovnice s celočíselnými koeficienty je nerozhodnutelný problém, známý jako 10. Hilbertův problém. Toto ukázal Matijasevič roku 1970.

Cvičení

- 1.15. Dokažte větu 1.19. Ukažte navíc, že vztah $\sigma(a + b) \leq \sigma(a) + \sigma(b)$ obecně neplatí pro $a, b \in \mathbb{Q}$.

1.5 Umění formulace celočíselných programů

Celočíselné programy mají velkou vyjadřovací schopnost. Pomocí nich můžeme zformulovat celou řadu netriviálních podmínek – logické či kvadratické podmínky, absolutní hodnotu, komplementaritu atp.

Příklad 1.31 (Min, max, absolutní hodnota aj.). Zformulujte pomocí lineárních celočíselných podmínek následující úlohy:

- (1) restrikce hodnot proměnných: $x \in \{v_1, \dots, v_m\}$, kde $v_1, \dots, v_m \in \mathbb{R}$ jsou dány,
- (2) $u = \min(x, y)$, kde $0 \leq x \leq K$, $0 \leq y \leq K$ jsou proměnné,
- (3) $u = |x|$, kde $0 \leq x \leq K$.

Řešení:

- (1) Přidáme m binárních proměnných $\alpha_1, \dots, \alpha_m$ a podmínku $x = \sum_{i=1}^m \alpha_i v_i$, kde $\sum_{i=1}^m \alpha_i = 1$.
- (2) Opět potřebujeme, aby hodnoty proměnných x, y byly omezené nějakou konstantou K . Přidáme proměnnou u , binární proměnnou δ a podmínky:

$$\begin{aligned} u &\leq x, & u &\geq x - K\delta, \\ u &\leq y, & u &\geq y - K(1 - \delta). \end{aligned}$$

(3) Přidáme binární proměnnou δ a podmínky:

$$\begin{aligned} u &\geq x, & u &\leq x + 2K\delta, \\ u &\geq -x, & u &\leq -x + 2K(1 - \delta), \end{aligned}$$

nebo převodem na předchozí případ transformací $|x| = -\min(x, -x)$. $\square$

Příklad 1.32 (Logické operace). Uvažujme úlohu investice kapitálu c mezi 7 projektů, kde i -tý projekt má výnos c_i a náklady a_i . Zformulujte úlohu jako celočíselný program s následujícími dodatečnými omezeními:

- (1) bez omezení,
- (2) chci investovat aspoň do 2, ale nanejvýš do 5 projektů,
- (3) do projektu 5 lze investovat právě tehdy, když do projektu 6,
- (4) investuji-li do projektu 3, musím i do projektu 4,
- (5) investuji-li do projektu 1, nesmím pak do projektu 2.

Řešení:

- (1) $\max c^T x$ za podm. $a^T x \leq c$, $x \in \{0, 1\}^7$.
- (2) $2 \leq e^T x \leq 5$,
- (3) $x_5 = x_6$,
- (4) $x_4 \geq x_3$,
- (5) $x_2 \leq 1 - x_1$.

$\square$

Příklad 1.33 (Po částech lineární funkce). Zformulujte pomocí lineárních celočíselných podmínek po částech lineární funkci $f(x)$ na intervalu $[x_0, x_m]$, přičemž body zlomů v $x_0, \dots, x_m \in \mathbb{R}$ a hodnoty ve zlomech jsou $a_0, \dots, a_m \in \mathbb{R}$.

Řešení: Zavedeme binární proměnné $y_1, \dots, y_m \in \{0, 1\}$, kde $y_i = 1$ říká, že x se nachází v intervalu $[x_{i-1}, x_i]$ a $y_i = 0$ říká, že nikoli. Dále zavedeme spojitě proměnné $\lambda_0, \dots, \lambda_m$, pomocí níz vytvoříme odpovídající konvexní kombinace. Hledaná formulace pro $z = f(x)$ pak je

$$\begin{aligned} x &= \sum_{i=0}^m \lambda_i x_i, & z &= \sum_{i=0}^m \lambda_i a_i, & \sum_{i=0}^m \lambda_i &= 1, \\ \lambda_0 &\leq y_1, & \lambda_m &\leq y_m, \\ \lambda_i &\leq y_{i-1} + y_i, & i &= 1, \dots, m-1, \\ \lambda_1, \dots, \lambda_m &\geq 0, \\ y_1, \dots, y_m &\in \{0, 1\}. \end{aligned}$$

Pozoruhodné na této formulaci je, že relaxováním celočíselnosti proměnných $y_1, \dots, y_m \in \{0, 1\}$ na $y_1, \dots, y_m \in [0, 1]$ dostaneme nejlepší možnou lineární relaxaci. Výraz $(x, z) = \sum_{i=0}^m \lambda_i (x_i, a_i)$ pak totiž popisuje konvexní kombinaci bodů $(x_0, a_0), \dots, (x_m, a_m)$. Projekcí do podprostoru proměnných x, z tak dostaneme konvexní obal bodů $(x_0, a_0), \dots, (x_m, a_m)$, což je nejlepší možný výsledek – lineární podmínky popisují konvexní polyedr a projekcí dostaneme zase konvexní polyedr, tudíž těsnější objekt nelze očekávat. $\square$

Příklad 1.34 (Sjednocení polyedrů). Zformulujte pomocí lineárních celočíselných podmínek sjednocení m omezených konvexních polyedrů.

Řešení 1: Pro $i = 1, \dots, m$ nechť polyedr P_i je popsán soustavou $A^i x \leq b^i$. Pak jejich sjednocení $\cup_{i=1}^m P_i$ je popsáno

$$\begin{aligned} A^i x &\leq b^i y_i, & i &= 1, \dots, m, \\ x &= \sum_{i=1}^m x^i, & \sum_{i=1}^m y_i &= 1, \\ x, x^1, \dots, x^m &\in \mathbb{R}^n, & y_1, \dots, y_m &\in \{0, 1\}. \end{aligned}$$

Zde se využívá faktu, že pro omezený polyedr nastane $Ax \leq 0$ právě tehdy, když $x = 0$. Binární proměnná y_i pak vyjadřuje, že x bude náležet do polyedru P_i . Přesně řečeno, $y_i = 1$ znamená, že ano, a $y_i = 0$ znamená, že nemusí.

Tato formulace má podobnou vlastnost s jakou jsme se setkali již v minulém Příkladu 1.33. Pokud relaxujeme binární proměnné na tvar $y_1, \dots, y_m \in [0, 1]$, soustava nabyde formy

$$\begin{aligned} A^i x^i &\leq b^i, \quad i = 1, \dots, m, \\ x &= \sum_{i=1}^m y_i x^i, \quad \sum_{i=1}^m y_i = 1, \\ x^i &\in \mathbb{R}^n, \quad y_1, \dots, y_m \in [0, 1], \end{aligned}$$

která vyjadřuje, že x je lineární kombinací bodů $x^i \in P_i$, čili $x \in \text{conv}(\cup_{i=1}^m P_i)$.

Řešení 2: Sjednocení $\cup_{i=1}^m P_i$ lze popsat i jako

$$\begin{aligned} A^i x &\leq b^i + (1 - y_i)Ke, \quad i = 1, \dots, m, \\ \sum_{i=1}^m y_i &= 1, \\ x &\in \mathbb{R}^n, \quad y_1, \dots, y_m \in \{0, 1\}. \end{aligned}$$

Zde K reprezentuje dost velké číslo takové, aby $A^i x \leq b^i + Ke$ pro všechna $x \in \cup_{i=1}^m P_i$. Potom $y_i = 1$ říká, že x náleží do P_i a $y_i = 0$, že nemusí. Výhoda této formulace je, že využívá menší počet proměnných. Zatímco v první formulaci jsme měli $n(m+1)$ spojitých a m binárních proměnných, nyní máme pouze n spojitých a m binárních proměnných. Na druhou stranu, druhá formulace nemusí být tak silná jako ta první. Kvalitu formulace lze zesílit co nejtěsnější volbou konstanty K , či ještě lépe vektor Ke nahradit co nejtěsnějším vektorem K^i pro $i = 1, \dots, m$.

Speciálně pro dva polyedry dostaneme formulaci

$$\begin{aligned} A^1 x &\leq b^1 + yKe, \\ A^2 x &\leq b^2 + (1 - y)Ke, \\ x &\in \mathbb{R}^n, \quad y \in \{0, 1\}. \end{aligned}$$

□

Příklad 1.35 (Pevné ceny). Přeformujte celočíselným programem účelová funkci ve tvaru

$$f(x) = \begin{cases} c^T x + d & \text{je-li } x > 0, \\ 0 & \text{je-li } x = 0. \end{cases}$$

Takto vypadají často poplatky v dopravních úlohách – za převoz žádného zboží neplatíme nic, ale za převoz určitého zboží platíme úměrně dle množství zboží plus nějaké další fixní výdaje či paušál.

Řešení: Vyjádříme $f(x) = c^T x + dy$, kde $y \in \{0, 1\}$ je binární proměnná, která určuje, zda $x > 0$. Podmínku $y = 1 \Leftrightarrow x > 0$ vyjádříme jako $x \leq yK$, $x \geq yK^{-1}$, kde K je dost velká konstanta. Poslední podmínka $x \geq yK^{-1}$ se často může vynechat, protože v praktických úlohách bývá $d > 0$. □

Příklad 1.36 (Problém lineární komplementarity). Přeformulujte jako celočíselný program úlohu lineární komplementarity, což je úloha přípustnosti systému bez účelové funkce

$$y = Mz + q, \quad y, sz \geq 0, \quad y^T z = 0,$$

s proměnnými $y, z \in \mathbb{R}^n$.

Řešení: Stačí přeformulovat podmínku komplementarity $y^T z = 0$. Tu rozepíšeme jako

$$y_i \leq \alpha b_i, \quad z_i \leq \alpha(1 - b_i), \quad i = 1, \dots, n,$$

kde $\alpha > 0$ je dostatečně velké (jak víme, s polynomiální velikostí) a $b_i \in \{0, 1\}$ jsou binární proměnné. □

Příklad 1.37 (Asymetrie). V řadě příkladů této sekce se ve formulaci vyskytují podmínky

$$\sum_{i=1}^m y_i = 1, \quad y \in \{0, 1\}^m.$$

Takovéto formulace pak působí potíže v algoritmech branch & bound, viz sekce 4.4. Proto je dobré přeformulovat podmínky jiným způsobem.

Uvažujme substituci $W := Ly$, neboli $y := L^{-1}w$, kde

$$L = \begin{pmatrix} 1 & \dots & \dots & 1 \\ 0 & \ddots & & \vdots \\ \vdots & \ddots & \ddots & \vdots \\ 0 & \dots & 0 & 1 \end{pmatrix}, \quad L^{-1} = \begin{pmatrix} 1 & -1 & 0 & 0 \\ 0 & \ddots & \ddots & 0 \\ \vdots & \ddots & \ddots & -1 \\ 0 & \dots & 0 & 1 \end{pmatrix}.$$

Tedy, $y_1 = w_1 - w_2, \dots, y_{m-1} = w_{m-1} - w_m, y_m = w_m$. Pokud například proměnná x má representovat diskrétní výběr jedné z hodnot $a_1, \dots, a_m$, pak tradiční formulaci

$$x = \sum_{i=1}^m a_i y_i, \quad \sum_{i=1}^m y_i = 1, \quad y \in \{0, 1\}^m$$

nahradíme formulací

$$x = \sum_{i=1}^{m-1} a_i (w_i - w_{i+1}) + a_m w_m, \quad 1 = w_1 \geq w_2 \geq \dots \geq w_m, \quad w \in \{0, 1\}^m,$$

neboli

$$x = a_1 w_1 + \sum_{i=2}^m (a_i - a_{i-1}) w_i, \quad 1 = w_1 \geq w_2 \geq \dots \geq w_m, \quad w \in \{0, 1\}^m. \quad \square$$

Další formulace viz například:

- MOSEK: <http://docs.mosek.com/modeling-cookbook/>
- Fico: <http://www.fico.com/en/node/8140?file=5125>

Cvičení

1.16. Buďte b_1, b_2, b_3 binární proměnné. Linearizujte podmínku $b_1 b_2 = b_3$.

1.17. Buď $0 < a < b$. Formulujte lineární celočíselnou podmínku pro omezení

$$x \in \{0\} \cup [a, b]$$

pro spojitou proměnnou $x \geq 0$.

1.18. Výroba produktu A nebo B implikují výrobu produktu C nebo D nebo E .

1.19. Zformulujte jako celočíselný program $u \in \{x \in \mathbb{Z}^n; Ax \leq b\} \setminus \{x^*\}$, kde $x^* \in \mathbb{R}^n$ je dáno a polyedr je omezený.

1.20. Zformulujte problém nalezení kružnice maximální váhy v grafu jako úlohu celočíselného programování. Váhy jsou dány na hranách a váha kružnice je součet vah jejích hran.

1.21. Formulace úloh z teorie grafů pomocí celočíselného programu. Použijte jak rozepsaný tvar, tak i vyjádření s maticí sousednosti nebo incidence.

- Určete velikost maximální kliky grafu.
- Najděte maximální nezávislou množinu grafu.
- Najděte vrcholové pokrytí grafu.

- (d) Určete barevnost grafu.
- (e) Najděte maximální párování v grafu.
- (f) Najděte maximální řez v grafu.
- 1.22. Pro $n \in \mathbb{N}$ sestavte celočíselný program, který vyřeší problém umístění n dam na šachovnici $n \times n$ tak, aby se navzájem neohrožovaly.
- 1.23. Dána velká krabice s rozměry k_1, k_2, k_3 a n malých krabic s rozměry m_1^i, m_2^i, m_3^i , $i = 1, \dots, n$. Napište celočíselný program na složení maximálního počtu malých krabic do velké. Podotýkáme, že krabice lze celé otáčet, ale nelze umisťovat šikmo.
- 1.24. Navrhněte lineární celočíselný program pro řešení Sudoku.
- 1.25. Karel, student optimalizace na Univerzitě Karlově, si chce v následujícím semestru zapsat některé z předmětů $p_1, \dots, p_5$. Pomozte Karlovi pomocí podmínek lineárního celočíselného programování modelovat tyto studijní požadavky:
- (a) musí si zapsat alespoň dva předměty,
- (b) zapíše-li si předmět p_1 , musí si také zapsat p_5 ,
- (c) zapíše-li si předmět p_2 , nemůže si zapsat p_4 ,
- (d) předmět p_3 si může zapsat, pouze pokud si zapíše také p_1 nebo p_2 ,
- (e) předmět p_4 si může zapsat, pouze pokud si zapíše také p_2 a p_3 ,
- (f) pokud si zapíše 2 nebo 3 předměty z množiny $\{p_3, p_4, p_5\}$, nemůže si zapsat p_2 .

Další omezení pro Karla:

- (a) Pokud si Karel zapíše alespoň k předmětů z množiny $\{p_1, \dots, p_n\}$, pak si musí také zapsat předměty q_1 a q_2 .
- (b) Karel si může zapsat předmět p_3 pouze tehdy, když si zapíše právě jeden z předmětů p_1, p_2 .
- 1.26. Firma Karlovy trubky s.r.o. skladuje kovové trubky standardní délky 100 cm, které následně řeže a prodává v kratších délkách. Firma obdržela následující objednávku:

délka (cm)	14	31	36	45
množství (ks)	211	395	610	97

Navrhněte model pro výpočet minimálního množství 100 cm trubek potřebných ke splnění objednávky a určení způsobu rozřezání trubek.

- 1.27. V nemocnici sv. Karla pracují ošetřovatelé v 8hodinových směnách, které začínají v 0:00, 4:00, 8:00, 12:00, 16:00 a 20:00. Navrhněte model lineárního celočíselného programování pro výpočet minimálního počtu ošetřovatelů zaměstnaných v nemocnici, pokud je počet potřebných ošetřovatelů v daném časovém intervalu určený následující tabulkou:

časový interval	minimální počet ošetřovatelů
00:00–04:00	3
04:00–08:00	8
08:00–12:00	10
12:00–16:00	12
16:00–20:00	14
20:00–00:00	8

- 1.28. Případová studie: Navrhněte (zjednodušený) model lineárního celočíselného programování pro rozvrhování výuky na vysoké škole. Určete:
- (a) omezující podmínky, které bychom měli do modelu zahrnout,

- (b) vhodnou účelovou funkci pro tento problém,
- (c) vstupní data potřebná pro vytvoření programu.

1.29. Případová studie: Máme 1 letiště na pevnině a n vrtných plošin na moři, h vrtulníků a každý vrtulník má kapacitu osob c a rychlost v . Dále známe vzdálenosti d_{ij} mezi i -tou a j -tou plošinou a vzdálenosti d_{0j} mezi letištěm j -tou plošinou. Koncem každého měsíce musíme obměnit osoby pracující na plošinách; na i -té plošině je jich p_i .

Navrhňte řešení tohoto problému, tj. navrhňte formulaci, diskutujte její vlastnosti, popř. navrhňte vylepšení (a jaké údaje byste k vylepšení potřebovali).

Kapitola 2

Technika reformulace a linearizace (RLT)

Tato kapitola vychází z knihy Sherali and Adams [1999] a ukazuje metodu relaxace množiny přípustných řešení. Ta povede na těsnější aproximaci na klasická celočíselná relaxace. Jak uvidíme, RLT dost velkého řádu dokonce vydá konvexní obal přípustných řešení.

Smíšená úloha s binárními proměnnými

Uvažujme podmínky se smíšenými binárními a spojitými proměnnými

$$Ax + By \leq c, \quad x \in \{0, 1\}^n, \quad y \in [0, 1]^m, \quad (2.1)$$

kde $A \in \mathbb{R}^{r \times n}$, $B \in \mathbb{R}^{r \times m}$ a $c \in \mathbb{R}^r$. U spojitých proměnných uvažujeme dolní a horní meze, z čehož přeškálováním dosáhneme rozsahu $[0, 1]$.

RLT řádu d . RLT řádu $d = 0$ je pouze celočíselná relaxace, což vede na konvexní polyedr

$$M_0 = \{(x, y) \in \mathbb{R}^{n+m}; Ax + By \leq c, \quad x \in [0, 1]^n, \quad y \in [0, 1]^m\}.$$

Buď $d > 0$. Základní myšlenka je přenásobovat nerovnice v (2.1) proměnnou $x_i \geq 0$ a výrazem $1 - x_i \geq 0$. S využitím toho, že $x_i^2 = x_i$, neboli $x_i(1 - x_i) = 0$, můžeme výsledný výraz zjednodušit. Zbylé monomy pak nahradíme novými proměnnými.

Označme $N := \{1, \dots, n\}$. Symbolem $(I, J)_d$ rozumíme dvojici indexových množin $I, J \subseteq N$ takovou, že $I \cap J = \emptyset$ a $|I \cup J| = d$. Uvažujme polynomy tvaru

$$f_d(I, J) = \prod_{i \in I} x_i \prod_{j \in J} (1 - x_j), \quad \forall (I, J)_d. \quad (2.2)$$

Těchto polynomů je $\binom{n}{d} 2^d$, protože je $\binom{n}{d}$ možností pro výběr $I \cup J$ a 2^d možností pro výběr zda dané $i \in I \cup J$ bude v I nebo v J . Pro pevnou hodnotu d je jich tedy polynomiálně mnoho.

Reformulace. Nyní vynásobíme každou nerovnici v (2.1) postupně všemi polynomy typu (2.2). Dostaneme nerovnice tvaru

$$\sum_{j \in I} (a_{ij} - c_i) f_d(I, J) + \sum_{j \notin I \cup J} a_{ij} \cdot f_{d+1}(I \cup \{j\}, J) + \sum_{j=1}^m b_{ij} \cdot y_j \cdot f_d(I, J) \leq 0, \quad \forall i, \forall (I, J)_d,$$

$$\begin{aligned} f_{d'}(I, J) &\geq 0, \quad \forall (I, J)_{d'}, \\ f_d(I, J) &\geq y_k \cdot f_d(I, J) \geq 0, \quad \forall (I, J)_d, \end{aligned}$$

kde $d' = \min(d + 1, n)$.

Linearizace. Nyní linearizujeme vzešlé polynomy s použitím substituce

$$v_J \equiv \prod_{j \in J} x_j, \quad w_{J,k} \equiv v_J y_k,$$

kdy $v_{\{j\}} \equiv x_j$, $v_{\emptyset} \equiv 1$ a $w_{\emptyset,k} \equiv y_k$. Linearizovaný polynom $f_d(I, J)$ označíme jako $g_d(I, J)$ a výsledná soustava pak má popis

$$\sum_{j \in I} (a_{ij} - c_i) g_d(I, J) + \sum_{j \notin I \cup J} a_{ij} \cdot g_{d+1}(I \cup \{j\}, J) + \sum_{j=1}^m b_{ij} \cdot y_j \cdot g_d(I, J) \leq 0, \quad \forall i, \forall (I, J)_d, \quad (2.3a)$$

$$g_{d'}(I, J) \geq 0, \quad \forall (I, J)_{d'}, \quad (2.3b)$$

$$g_d(I, J) \geq y_k \cdot g_d(I, J) \geq 0, \quad \forall (I, J)_d. \quad (2.3c)$$

Označme jako M_d polyedr popsaný touto soustavou a necht' M_d^P je jeho projekce do prostoru proměnných x, y . Naším cílem je ukázat vztah

$$M_0 = M_0^P \supseteq M_1^P \supseteq \dots \supseteq M_n^P = M_I, \quad (2.4)$$

kde jako v sekci 1.2 značíme $M_I = \text{conv}(M \cap (\{0, 1\}^n \times [0, 1]^m))$. Tento vztah ukazuje, že čím vyšší řád RLT, tím, lepší aproximace a pro n -tý řád RLT dostaneme konvexní obal přípustných bodů. Důkaz vyplyne z vět 2.4 a 2.6 dole. Nejprve ale ilustrujeme postup na příkladech a odvodíme pomocná tvrzení.

Příklad 2.1. Aplikujte RLT prvního řádu na podmínky

$$x + y \leq 2, \quad -x + y \leq 1, \quad 2x - 2y \leq 1, \quad x \in \{0, 1\}, \quad y \geq 0.$$

Řešení: Nerovnosti vynásobíme postupně hodnotami x a $1 - x$ a dostaneme sadu podmínek:

$$\begin{aligned} x^2 + xy &\leq 2x, & x(1-x) + y(1-x) &\leq 2(1-x), \\ -x^2 + xy &\leq x, & -x(1-x) + y(1-x) &\leq 1-x, \\ 2x^2 - 2xy &\leq x, & 2x(1-x) - 2y(1-x) &\leq 1-x, \\ xy &\geq 0, & y(1-x) &\geq 0. \end{aligned}$$

S využitím $x^2 = x$, neboli $x(1-x) = 0$ a substitucí $w := xy \geq 0$ se nerovnice přepíší na tvar

$$\begin{aligned} -x + w &\leq 0, & 2x + y - w &\leq 2, \\ -2x + w &\leq 0, & x + y - w &\leq 1, \\ x - 2w &\leq 0, & x - 2y + 2w &\leq 1, \\ w &\geq 0, & y - w &\geq 0. \end{aligned}$$

Fourier–Motzkinovou eliminací proměnné w dostaneme projekci do prostoru proměnných x, y

$$x \leq 2y, \quad 0 \leq x \leq 1, \quad y \leq 1.$$

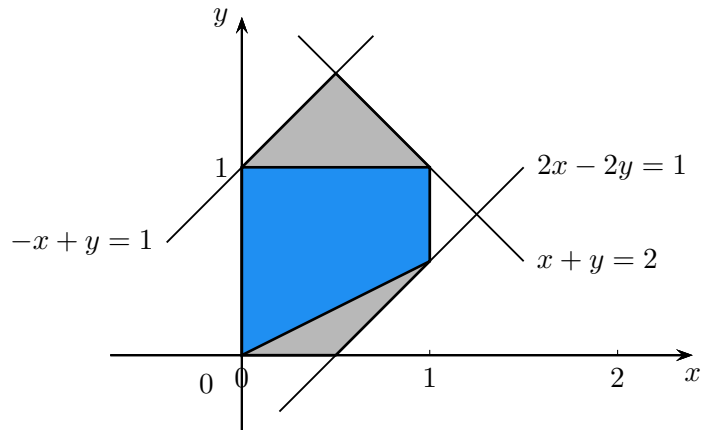
Podle vztahu (2.4) (tj. věty 2.6) tato množina popisuje konvexní obal množiny přípustných řešení, což je ilustrováno na obrázku 2.1. Tudíž již RLT prvního řádu vede na celočíselný polyedr. $\square$

Příklad 2.2. Aplikujte RLT prvního řádu na podmínky

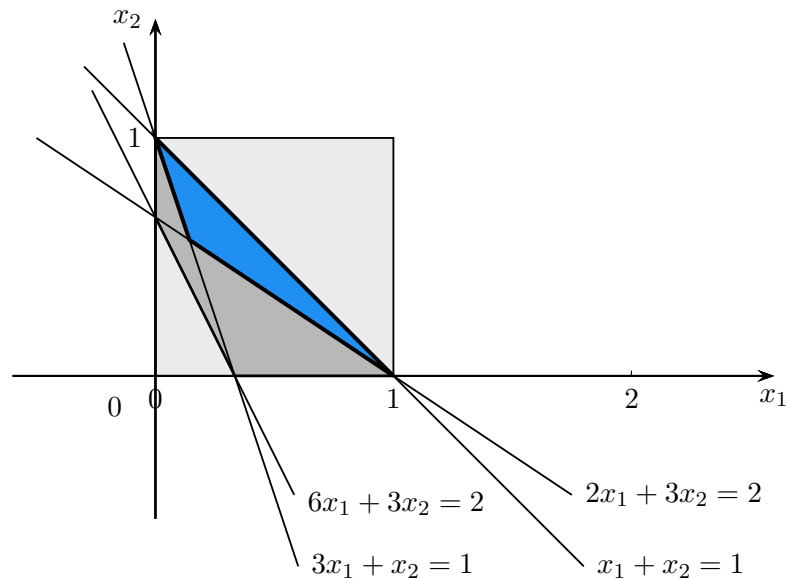
$$x_1 + x_2 \leq 1, \quad 6x_1 + 3x_2 \geq 2, \quad x \in \{0, 1\}^2.$$

Řešení: První nerovnost $x_1 + x_2 \leq 1$ vynásobíme postupně hodnotami $x_1, x_2, 1 - x_1, 1 - x_2$ a dostaneme sadu podmínek:

$$\begin{aligned} x_1^2 + x_1 x_2 &\leq x_1, \\ x_1 x_2 + x_2^2 &\leq x_2, \\ x_1(1 - x_1) + x_2(1 - x_1) &\leq 1 - x_1, \\ x_1(1 - x_2) + x_2(1 - x_2) &\leq 1 - x_2. \end{aligned}$$



Obrázek 2.1: (příklad 2.1) Relaxovaný polyedr (tmavě šedě) a polyedr po technice RLT prvního řádu (modře).



Obrázek 2.2: (příklad 2.2) Relaxovaný polyedr (tmavě šedě) a polyedr po technice RLT prvního řádu (modře).

S využitím $x_i^2 = x_i$ a substitucí $v := x_1x_2 \geq 0$ se nerovnice přepíší na

$$\begin{aligned} v &\leq 0, \\ x_1 + x_2 - v &\leq 1. \end{aligned}$$

Tudíž $v = 0$ a dostaneme původní nerovnici $x_1 + x_2 \leq 1$. Poznamenejme, že podmínku $v = x_1x_2 = 0$ jsme mohli využít rovnou ze znalosti úlohy, protože binární proměnné x_1, x_2 splňují $x_1 + x_2 \leq 1$.

Nyní postupujeme analogicky pro druhou podmínku $6x_1 + 3x_2 \geq 2$, kterou přenásobíme postupně x_1 , x_2 , $1 - x_1$, $1 - x_2$:

$$\begin{aligned} 6x_1^2 + 3x_1x_2 &\geq 2x_1, \\ 6x_1x_2 + 3x_2^2 &\geq 2x_2, \\ 6x_1(1 - x_1) + 3x_2(1 - x_1) &\geq 2(1 - x_1), \\ 6x_1(1 - x_2) + 3x_2(1 - x_2) &\geq 2(1 - x_2). \end{aligned}$$

Opět využijeme $x_i^2 = x_i$ a $v = x_1x_2 = 0$ a dostaneme

$$\begin{aligned} x_1 &\geq 0, \\ x_2 &\geq 0, \\ 2x_1 + 3x_2 &\geq 2, \\ 3x_1 + x_2 &\geq 1. \end{aligned}$$

Výsledný polyedr je tedy popsán soustavou

$$\begin{aligned} x_1 + x_2 &\leq 1, \\ 2x_1 + 3x_2 &\geq 2, \\ 3x_1 + x_2 &\geq 1, \\ x_1, x_2 &\geq 0 \end{aligned}$$

a je ilustrován na obrázku 2.2. □

Lemma 2.3. Pro každé $d \in \{0, 1, \dots, n-1\}$ nerovnice $g_{d+1}(I, J) \geq 0$ implikují nerovnice $g_d(I, J) \geq 0$. Podobně nerovnice $g_{d+1}(I, J) \geq y_k \cdot g_{d+1}(I, J) \geq 0$ implikují nerovnice $g_d(I, J) \geq y_k \cdot g_d(I, J) \geq 0$.

Důkaz. Pro všechna $i \notin I \cup J$ platí

$$f_d(I, J) = f_d(I, J)(x_i + 1 - x_i) = f_{d+1}(I \cup \{i\}, J) + f_{d+1}(I, J \cup \{i\}).$$

Tento vztah platí zřejmě i po linearizaci

$$g_d(I, J) = g_{d+1}(I \cup \{i\}, J) + g_{d+1}(I, J \cup \{i\}). \quad (2.5)$$

To dává první část znění tvrzení a druhá se ukáže analogicky. □

Věta 2.4. Platí

$$M_0 = M_0^P \supseteq M_1^P \supseteq \dots \supseteq M_n^P \supseteq M_I.$$

Důkaz. První rovnost platí z definice M_0 . Poslední inkluze platí z toho, že jsme původní podmínky relaxovali, a tedy popisují nadmnožinu původní množiny přípustných řešení.

Buď $d \in N$ a ukážeme $M_d \supseteq M_{d+1}$, z čehož pak ihned vyplývá $M_d^P \supseteq M_{d+1}^P$. Podle lemmatu 2.3 jsou podmínky (2.3b) a (2.3c) pro M_d^P implikovány analogickými podmínkami pro M_{d+1}^P . Zbývá porovnat podmínky (2.3a). Podle (2.5) je ale nerovnice získaná přenásobením $f_d(I, J)$ odvoditelná jako součet nerovnic získaných přenásobením $f_{d+1}(I \cup \{i\}, J)$ a $f_{d+1}(I, J \cup \{i\})$. □

Lemma 2.5. Uvažujme zobrazení $v_J \mapsto z_J$, $J \subseteq N$, definované

$$z_J = g_n(J, N \setminus J) = \sum_{I \subseteq N \setminus J} (-1)^{|I|} v_{I \cup J}.$$

Toto zobrazení je bijekcí a k němu inverzní zobrazení má předpis

$$v_J = \sum_{I \subseteq N \setminus J} z_{I \cup J}.$$

Podobně

$$z_J^k = y_k \cdot g_n(J, N \setminus J) = \sum_{I \subseteq N \setminus J} (-1)^{|I|} w_{I \cup J, k}$$

je bijekcí $w_{J, k} \mapsto z_J^k$. Speciálně

$$x_j = v_{\{j\}} = \sum_{J \ni j} z_J, \quad 1 = v_\emptyset = \sum_{J \subseteq N} z_J, \quad y_k = w_{\emptyset, k} = \sum_{J \subseteq N} z_J^k.$$

Důkaz. Upravíme

$$\begin{aligned} f_n(J, N \setminus J) &= \prod_{j \in J} x_j \prod_{j \notin J} (1 - x_j) = \prod_{j \in J} x_j \sum_{I \subseteq N \setminus J} (-1)^{|I|} \prod_{i \in I} x_i \\ &= \sum_{I \subseteq N \setminus J} (-1)^{|I|} \prod_{j \in I \cup J} x_j. \end{aligned}$$

Tudíž

$$z_J = g_n(J, N \setminus J) = \sum_{I \subseteq N \setminus J} (-1)^{|I|} v_{I \cup J}.$$

Nyní naopak, upravíme výraz

$$\sum_{I \subseteq N \setminus J} z_{I \cup J} = \sum_{I \subseteq N \setminus J} \sum_{K \subseteq N \setminus (I \cup J)} (-1)^{|K|} v_{I \cup K \cup J}.$$

Substitucí $H \equiv I \cup K$ dostaneme

$$\sum_{I \subseteq N \setminus J} z_{I \cup J} = \sum_{H \subseteq N \setminus J} \sum_{K \subseteq H} (-1)^{|K|} v_{H \cup J} = \sum_{H \subseteq N \setminus J} v_{H \cup J} \sum_{K \subseteq H} (-1)^{|K|} = v_J,$$

protože

$$\sum_{K \subseteq H} (-1)^{|K|} = (1 - 1)^{|H|} = \begin{cases} 0 & H \neq \emptyset, \\ 1 & H = \emptyset. \end{cases}$$

Tím jsme ukázali první část tvrzení, ostatní se dokážou analogicky. □

Věta 2.6. Platí $M_n^P = M_I$.

Důkaz. Je-li $M_n^P = \emptyset$, pak tvrzení zřejmě platí. Předpokládejme tedy $M_n^P \neq \emptyset$. Protože je konvexní polyedr M_n^P navíc omezený, stačí ukázat, že všechny jeho vrcholy mají x -složku binární. Jinými slovy, pro libovolné $p \in \mathbb{R}^n$ a $q \in \mathbb{R}^m$ má úloha lineárního programování

$$\max p^T x + q^T y \quad \text{za podm. } (x, y) \in M_n^P$$

optimální řešení s binárním x . Úloha má konkrétně tvar (podmínka (2.3b) je redundantní)

$$\begin{aligned} &\max p^T x + q^T y \quad \text{za podm.} \\ &\sum_{j \in J} (a_{ij} - c_i) g_n(J, N \setminus J) + \sum_{k=1}^m b_{ik} \cdot y_k \cdot g_n(J, N \setminus J) \leq 0, \quad \forall i, \forall J \subseteq N, \\ &g_n(J, N \setminus J) \geq y_k \cdot g_n(J, N \setminus J) \geq 0, \quad \forall J \subseteq N. \end{aligned}$$

S využitím substituce podle lemmatu 2.5 můžeme psát

$$\begin{aligned} &\max \sum_{j=1}^n p_j \sum_{J \ni j} z_J + \sum_{k=1}^m q_k \sum_{J \subseteq N} z_J^k \quad \text{za podm.} \\ &\sum_{j \in J} (a_{ij} - c_i) z_J + \sum_{k=1}^m b_{ik} z_J^k \leq 0, \quad \forall i, \forall J \subseteq N, \\ &\sum_{J \subseteq N} z_J = 1, \\ &z_J \geq z_J^k \geq 0, \quad \forall k, \forall J \subseteq N. \end{aligned}$$

Poznamenejme, že podmínka $\sum_{J \subseteq N} z_J = 1$ vznikla substitucí pro případ $J = \emptyset$. Označme $\tilde{p}_J := \sum_{j \in J} p_j$ a $h_{ij} := a_{ij} - c_i$. Nyní má úloha tvar

$$\begin{aligned} \max \quad & \sum_{J \subseteq N} \tilde{p}_J z_J + \sum_{J \subseteq N} \sum_{k=1}^m q_k z_J^k \quad \text{za podm.} \\ & \sum_{j \in J} h_{ij} z_J + \sum_{k=1}^m b_{ik} z_J^k \leq 0, \quad \forall i, \forall J \subseteq N, \\ & \sum_{J \subseteq N} z_J = 1, \quad z_J \geq z_J^k \geq 0, \quad \forall k, \forall J \subseteq N. \end{aligned}$$

Pro pevnou hodnotu z_J , $J \subseteq N$, je tato úloha separabilní. Tedy ji můžeme formulovat

$$\max \sum_{j=1}^n \tilde{p}_J z_J + \sum_{J \subseteq N} \gamma_J(z_J) \quad \text{za podm.} \quad \sum_{J \subseteq N} z_J = 1, \quad z_J \geq 0,$$

kde

$$\begin{aligned} \gamma_J(z_J) = \max \quad & \sum_{k=1}^m q_k z_J^k \quad \text{za podm.} \quad h_{ij} z_J + \sum_{k=1}^m b_{ik} z_J^k \leq 0, \quad \forall i, \\ & z_J \geq z_J^k \geq 0, \quad \forall k. \end{aligned}$$

Substitucí $\tilde{z}_J^k \equiv z_J^k / z_J$, $\tilde{\gamma}_J \equiv \gamma_J(z_J) / z_J$ přepíšeme podúlohu do tvaru

$$\begin{aligned} \tilde{\gamma}_J = \max \quad & \sum_{k=1}^m q_k \tilde{z}_J^k \quad \text{za podm.} \quad h_{ij} + \sum_{k=1}^m b_{ik} \tilde{z}_J^k \leq 0, \quad \forall i, \\ & 1 \geq \tilde{z}_J^k \geq 0, \quad \forall k, \end{aligned}$$

přičemž $\gamma_J = -\infty$ resp. $\tilde{\gamma}_J = -\infty$, pokud je podúloha nepřipustná. Díky $M_n^P \neq \emptyset$ je konečný aspoň jeden koeficient pro nějaké J . Původní úlohu lineárního programování tak můžeme přepsat do tvaru

$$\max \sum_{J \subseteq N} (\tilde{p}_J + \tilde{\gamma}_J) z_J \quad \text{za podm.} \quad \sum_{J \subseteq N} z_J = 1, \quad z_J \geq 0.$$

Maximalizujeme tedy lineární funkci na simplexu, tudíž optimum se nabyde v nějakém vrcholu. To znamená, že existuje J^* takové, že optimum je $z_{J^*} = 1$ a $z_J = 0$ pro $J \neq J^*$. Tomu odpovídá hodnota $x_j = 1$ pro $j \in J^*$ a $x_j = 0$ pro $j \notin J^*$. Hodnota y_k se dopočítá z podúlohy γ_{J^*} jako $y_k = \tilde{z}_{J^*}^k$. Proměnná x nabývá binárních hodnot, což dokazuje tvrzení. $\square$

Speciální tvary

Pro speciální tvary celočíselných podmínek lze pochopitelně odvodit speciálnější, a dost často silnější, výsledky. Uvažujme například binární proměnné $x_1, \dots, x_n$ a omezení $e^T x \leq 1$. Toto omezení ve tvaru nerovnosti či rovnosti se často vyskytuje v různých souvislostech (viz GUB na straně 60). V tomto případě namísto přenásobování výrazy

$$x_1, 1 - x_1, \dots, x_n, 1 - x_n$$

můžeme přenásobovat výrazy

$$x_1, \dots, x_n, 1 - e^T x.$$

Příklad 2.7. Aplikujte speciální RLT prvního řádu na podmínky z příkladu 2.2.

Řešení: Nové nerovnice dostaneme potenciálně přenásobením výrazem $1 - e^T x$. U první podmínky, která je tvaru $e^T x \leq 1$, pochopitelně dostaneme redundantní nerovnici, ale druhá vede na

$$(6x_1 + 3x_2 - 2)(1 - x_1 - x_2) \geq 2,$$

která po úpravě získá tvar $x_1 + x_2 \geq 1$. Čili odvodíme tím omezení $x_1 + x_2 = 1$, které dává přímo celočíselný polyedr, tj. konvexní obal přípustných řešení. Získali jsme tak silnější výsledek než pro standardní způsob RLT. $\square$

Obecnější úlohy

Myšlenka RLT lze rozšířit na smíšené úlohy celočíselného programování s celočíselnými proměnnými, které nejsou nutně binární. Rovněž lze využít technik RLT ke konstrukci řezů (viz kapitola 3) nelineárních úloh celočíselného programování. Dokonce obecně mezi první práce využívající myšlenky RLT patří Adams [1985]; Adams and Serali [1986, 1990], zabývající je celočíselným kvadratickým programováním.

Další relaxační hierarchie

Lasserrova hierarchie. RLT je založena na lineární relaxaci a vnoření do prostoru vyšší dimenze. Máme zde i hierarchii danou zvětšujícími se řády (věta 2.6). Koncepčně podobně funguje i technika založená na semidefinitních relaxacích, která vede na tzv. *Lasserrovu hierarchii*, viz např. Rothvoß [2013]. Ta, narozdíl od RLT, neuvažuje lineární relaxace, ale relaxuje podmínky jako semidefinitní program. V ostatních aspektech se hodně podobá RLT – opět zvyšující se řád relaxace dává lepší aproximaci až nakonec n -tý řád dává M_I , tedy konvexní obal přípustných řešení. Pro čistou úlohu 0-1 programování jsou Lasserrovy relaxace přesnější než RLT [Laurent, 2003].

Vysvětlíme jenom základní myšlenku Lasserrovy relaxace bez technických detailů. Lasserrov přístup využíval statistické interpretace a momentových matic, my však relaxaci popíšeme ryze algebraicky. Uvažujme úlohu ve tvaru

$$\min c^T x \text{ za podm. } Ax \leq b, x \in \{0, 1\}^n.$$

Pro $i = 1, \dots, m$ vytvořme nejprve matici

$$A_i := (b_i - A_{i*}x)ee^T$$

a potom tzv. momentovou matici

$$M_i := DA_iD,$$

kde $D = \text{diag}(1, x_1, \dots, x_n)$. Pro každé $x \in \mathbb{R}^n$ je M_i zjevně pozitivně semidefinitní. Podobně jako v RLT aplikujeme nejprve reformulaci, kdy opět využijeme vlastnosti $x_i^2 = x_i$ pro každé přípustné řešení x . Dalším krokem je linearizace, kdy každý monom nahradíme novou proměnnou, tedy $v_j \equiv \prod_{j \in J} x_j$. Tím dostaneme lineární podmínky plus podmínku na pozitivní semidefinitnost matice M_i .

Takto jsme zhruba popsali Lasserrovu relaxaci prvního řádu. Vyšší řády fungují analogicky s tím, že v relaxaci k -tého řádu matice D na diagonále obsahuje všechny monomy stupně nanejvýš k . Tedy například pro $k = 2$ máme

$$D = \text{diag}(1, x_1, \dots, x_n, x_1^2, \dots, x_n^2, x_1x_2, x_1x_3, \dots, x_1x_n, x_2x_3, \dots, x_{n-1}x_n).$$

Další. Kromě RLT a Lasserrovy hierarchie existují ještě další relaxační hierarchie, viz Laurent [2003] i pro jejich srovnání.

Kapitola 3

Metody sečných nadrovin aneb řeznictví pana Gomoryho

Uvažujme úlohu celočíselného programování ve tvaru

$$\max c^T x \text{ za podm. } x \in M, x_i \in \mathbb{Z} \forall i \in C \subseteq \{1, \dots, n\},$$

kde

$$M = \{x \in \mathbb{R}^n; Ax \leq b, x \geq 0\}.$$

Princip metod sečných nadrovin spočívá nejprve ve vyřešení lineární relaxace

$$\max c^T x \text{ za podm. } x \in M. \quad (3.1)$$

Pokud optimální řešení x^* splňuje podmínky celočíselnosti, jsme hotovi. Pokud ne, sestrojíme sečnou nadrovinu $r^T x = s$, která odřízne x^* od polyedru, ale žádné přípustné řešení neodřízne. To jest, musí platit

$$r^T x^* > s$$

a přitom

$$r^T x \leq s \text{ pro všechna } x \in M, x_i \in \mathbb{Z} \forall i \in C.$$

Podmínku $r^T x \leq s$ přidáme do omezení M a celý postup opakujeme.

Pro řešení jednotlivých lineárních programů je vhodná tzv. *ℓ-metoda*, neboli lexikografická duální simplexová metoda. Duální simplexová metoda totiž umožňuje jednoduše přidávat dynamicky nové podmínky a rychle přeoptimalizovat. Lexikografie pak je užitečná pro konvergenci algoritmu. Následující tři sekce jsou proto věnované stručnému popisu této metody na řešení lineárních programů, a teprve v dalších sekcích se setkáme s algoritmy na úlohy celočíselného programování. Konkrétně, sekce 3.4 popisuje první Gomoryho algoritmus na čistou úlohu a sekce 3.5 druhý Gomoryho algoritmus na smíšenou úlohu celočíselného programování.

3.1 Duální simplexová metoda

Mějme matici $A \in \mathbb{R}^{m \times n}$ a primární úlohu lineárního programování ve tvaru:

$$\begin{aligned} \max_{x \in M} c^T x, \quad \text{kde } M &= \{x \in \mathbb{R}^n; Ax \leq b, x \geq 0\} \\ &= \{x \in \mathbb{R}^n; Ax + Ix' = b, x, x' \geq 0\}. \end{aligned}$$

K ní přísluší duální úloha

$$\begin{aligned} \min_{y \in M_D} b^T y, \quad \text{kde } M_D &= \{y \in \mathbb{R}^m; A^T y \geq c, y \geq 0\} \\ &= \{y \in \mathbb{R}^m; A^T y - Iy' = c, y, y' \geq 0\}. \end{aligned}$$

V revidované formě (bez jednotkové matice) je tabulka simplexové metody pro primární úlohu

	nebázické x	
bázické x'	A	b
	$-c^T$	0

a pro duální úlohu

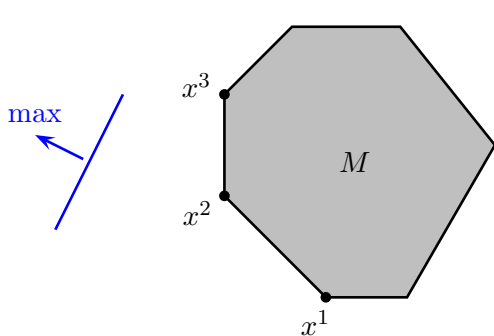
	nebázické y	
bázické y'	$-A^T$	$-c$
	b^T	0

Vidíme, že obě tabulky jsou v zásadě stejné až na překlopení a znaménko u matice A . Myšlenka duální simplexové metody je řešit duální úlohu za použití primární tabulky.

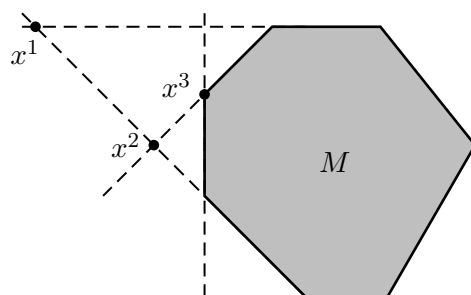
	nebáz. x , báz. y'	
báz. x' , nebáz. y	A	b
	$-c^T$	0

Zatímco primární simplexová metoda začíná s nezápornou pravou stranou b a postupnými iteracemi dospěje k výsledné tabulce s nezáporným kritériálním řádkem $-c^T$, duální metoda to dělá naopak: začne s nezáporným kritériálním řádkem a postupnými iteracemi dospěje k výsledné tabulce s nezápornou pravou stranou.

Z geometrického hlediska primární simplexová metoda prochází přípustná báze řešení řešení, tedy vrcholy polyedru M , se vzrůstající hodnotou účelové funkce tak dlouho, dokud nenarazí na optimum. Duální simplexová metoda naopak prochází přípustná řešení duální úlohy, která odpovídají nepřípustným báze primární úlohy. Postupně pak zlepšuje hodnotu účelové funkce duálu (a tedy zhoršuje hodnotu účelové funkce primáru), než získáme přípustné řešení primáru, které bude zároveň jeho optimum.



Primární simplexová metoda: postupná řešení
 x^1, x^2, x^3 jsou navazující vrcholy polyedru M .



Duální simplexová metoda: postupná řešení
 x^1, x^2, x^3 jsou nepřípustné báze.

3.2 Lexikografie

Definice 3.1. Buď $x, y \in \mathbb{R}^n$. Pak x je *lexikograficky větší* než y , psáno $x \succ y$, pokud $\exists i$ tak, že $x_1 = y_1, \dots, x_{i-1} = y_{i-1}$ a $x_i > y_i$. Dále, x je *lexikograficky větší nebo rovno* y , psáno $x \succeq y$, pokud $x \succ y$ nebo $x = y$.

Lexikografie je tedy úplné uspořádání na $\mathbb{R}^n$, každé dva vektory jsou porovnatelné.

Pro $x \in M$ označme $\tilde{x} := (c^T x, x) \in \mathbb{R}^{n+1}$ jako *rozšířené řešení*. Dále řekneme, že x^* je ℓ -optimální řešení úlohy $\max_{x \in M} c^T x$ pokud $\tilde{x}^* \succeq \tilde{x}$ pro všechna $x \in M$. Je zřejmé, že ℓ -optimální řešení je zároveň optimální.

Věta 3.2. Pro úlohu $\max_{x \in M} c^T x$ nechť je množina optimálních řešení M_{opt} neprázdná a omezená. Pak ℓ -optimální řešení existuje a je jednoznačné.

Důkaz. Protože M_{opt} je omezený neprázdný konvexní polyedr, každý bod $x \in M_{\text{opt}}$ se dá vyjádřit jako konvexní kombinace $x = \sum_{i \in I} \alpha_i v_i$, kde v_i , $i \in I$, jsou vrcholy M_{opt} a $\alpha_i \geq 0$, $\sum_{i \in I} \alpha_i = 1$. Uvažme lexikograficky největší vrchol v_{i^*} , tj., $v_{i^*} \succeq v_i$ pro všechna $i \in I$. Nyní

$$x = \sum_{i \in I} \alpha_i v_i \preceq \sum_{i \in I} \alpha_i v_{i^*} = v_{i^*},$$

tedy i pro bod $x \in M_{\text{opt}} \setminus \{x^*\}$ je $\tilde{x} \prec \tilde{v}_{i^*}$. Podobně pro libovolné $x \in M \setminus M_{\text{opt}}$ máme $c^T x < c^T v_{i^*}$, a proto $\tilde{x} \prec \tilde{v}_{i^*}$. Tudíž v_{i^*} je ℓ -optimální řešení. Jednoznačnost je zřejmá, neboť dva různé vektory jsou lexikograficky různé. $\square$

3.3 ℓ -metoda

Jak jsme již zmínili, lineární programy (3.1) budeme řešit ℓ -metodou. Ta pracuje s ℓ -tabulkou, která má na začátku tvar

$$\begin{array}{c|cc} & x_1 & x_n \\ \hline x_0 & -c^T & 0 \\ x_1 & & 0 \\ \vdots & & \vdots \\ x_n & -I_n & 0 \\ \hline x_{n+1} & & \\ \vdots & A & b \\ x_{m+n} & & \end{array} \quad \equiv \quad \begin{array}{c|cccc} & x_{N_1} & & x_{N_n} & \\ \hline x_0 & & & & \\ x_1 & & & & \\ \vdots & & & & \\ x_n & R_1 & \dots & R_n & R_0 \\ x_{n+1} & & & & \\ \vdots & & & & \\ x_{m+n} & & & & \end{array}$$

Na začátku jsou nebázické indexy $N := \{1, \dots, n\}$. V průběhu algoritmu si tabulku budeme značit pomocí matice R , a sloupceky zleva jako $R_1, \dots, R_n, R_0$. Každý řádek tabulky odpovídá (ne)rovnici z omezení tvaru

$$x_i = R_{i0} - \sum_{j=1}^n R_{ij} x_{N_j}, \quad \forall i = 1, \dots, m+n.$$

U počáteční tabulky speciálně máme $x = 0 - (-x)$ a pro doplňkové proměnné $x' := (x_{n+1}, \dots, x_{m+n})^T$ máme $x' = b - Ax$, což souhlasí.

Úvodní krok

ℓ -tabulka je ℓ -normální pokud $R_j \succ 0$, $j = 1, \dots, n$ (nestačí tedy pouze nezápornost prvního řádku). Pokud je tabulka ℓ -normální, můžeme jít na běžný krok algoritmu, jinak musíme provést úvodní krok.

Do tabulky přidáme podmínku $\sum_{j=1}^n x_i \leq L$, kde $L > 0$ je dost velké. Hodnotu L buďto umíme vyčíst za zadání, nebo ji určíme pomocným lineárním programem, anebo vystupuje jako parametr reprezentující libovolně velkou hodnotu. Nová podmínka je přípustná pokud je úloha omezená; v opačném případě neomezenost odhalíme během algoritmu.

Klíčový řádek (tj. ten, ve kterém se vybírá pivot) bude ten nový a klíčový sloupec ten lexikograficky nejmenší. Provedeme pivotizaci dle běžných simplexových pravidel tak, aby v klíčovém řádku byl opačný jednotkový vektor, tzn., že klíčový sloupec se dělí opačnou hodnotou pivota. Označme jako R' tabulku po jedné transformaci. Pak transformace s pivotem na pozici (k, ℓ) má tvar:

$$R'_\ell := \frac{1}{-R_{k\ell}} R_\ell, \quad (3.2a)$$

$$R'_j := R_j - \frac{R_{kj}}{R_{k\ell}} R_\ell, \quad j \neq \ell. \quad (3.2b)$$

Tvrzení 3.3. *Po jedné transformaci je tabulka ℓ -normální.*

Důkaz. Buď R_ℓ klíčový sloupec a označme R' tabulku po jedné transformaci. Protože $R_\ell \prec 0$, bude $R'_\ell = -R_\ell \succ 0$. Dále, z definice ℓ pro každé $j \neq \ell$ máme $R'_j = R_j - R_\ell \succ 0$. $\square$

Běžný krok

Volba klíčového řádku k pro pivota:

$$k := \arg \min \{i = 1, \dots, m+n; R_{i0} < 0\}.$$

Volba klíčového sloupce ℓ pro pivota:

$$\frac{R_\ell}{|R_{k\ell}|} := \text{lex min} \left\{ \frac{R_j}{|R_{kj}|}; j = 1, \dots, n, R_{kj} < 0 \right\}.$$

Transformace tabulky probíhá podle pravidla (3.2).

Tvrzení 3.4. *Po transformaci tabulka zůstane ℓ -normální.*

Důkaz. Pro klíčový sloupec platí

$$R'_\ell = \frac{1}{-R_{k\ell}} R_\ell \succ 0.$$

Pro ostatní sloupce $j \neq \ell$ platí

$$R'_j = R_j - \frac{R_{kj}}{R_{k\ell}} R_\ell.$$

Pokud $R_{kj} \geq 0$, tak triviálně $R'_j \succ 0$. Pokud $R_{kj} < 0$, tak z definice ℓ je $\frac{1}{-R_{k\ell}} R_j \succ \frac{1}{-R_{k\ell}} R_\ell$. Odtud $R_j \succ \frac{R_{kj}}{-R_{k\ell}} R_\ell$ a proto $R'_j \succ 0$. $\square$

Tvrzení 3.5. *Poslední sloupec tabulky se po každé transformaci lexikograficky zmenší.*

Důkaz. Pro poslední sloupec tabulky platí stejná transformační pravidla. Protože $R_{k0}, R_{k\ell} < 0$, dostáváme

$$R'_0 = R_0 - \frac{R_{k0}}{R_{k\ell}} R_\ell \prec R_0. \quad \square$$

Důsledek 3.6. *ℓ -metoda je konečná.*

Důkaz. Každá tabulka odpovídá jedné bázi, a těch je konečně mnoho. Protože se po každé transformaci poslední sloupec lexikograficky zmenší, algoritmus musí skončit po konečně mnoha iteracích. $\square$

Tvrzení 3.7 (kriterium optimality). *Je-li $x^* := (R_{1,0}, \dots, R_{m+n,0}) \geq 0$, pak x^* je ℓ -optimální řešení.*

Důkaz. Optimality x^* vyplývá z teorie duální simplexové metody.

Z důkazu tvrzení 3.2 víme, že ℓ -optimální řešení je vrcholem M , takže k důkazu ℓ -optimality x^* stačí ukázat, že přechodem k jinému optimálnímu bázičkému řešení (což lze učinit konečně mnoha přípustnými transformacemi ℓ -tabulky) si pouze lexikograficky pohoršíme vektor $R_0 = \tilde{x}^*$.

Uvažme transformaci dle pivota $R_{k\ell} \neq 0$. Předpokládejme, že $R_{k0} > 0$, neboť případ $R_{k0} = 0$ nevede ke změně vrcholu. Pokud $R_{k\ell} > 0$, tak $R'_0 = R_0 - \frac{R_{k0}}{R_{k\ell}} R_\ell \prec R_0$ a tedy si pohoršíme. Pro případ $R_{k\ell} < 0$ nechť $s := N_\ell$. Pak s -tý řádek tabulky je $-e_\ell$ a tudíž $R'_{s0} = R_{s0} - \frac{R_{k0}}{R_{k\ell}} R_{s\ell} = 0 + \frac{R_{k0}}{R_{k\ell}} < 0$. To znamená, že by řešení nebylo přípustné. $\square$

Tvrzení 3.8 (kriterium nepřípustnosti). *Nechť pro index $k > 0$ platí: $R_{k0} < 0$ a $R_{kj} \geq 0$, $j = 1, \dots, n$. Pak $M = \emptyset$.*

Důkaz. k -tý řádek ℓ -tabulky odpovídá rovnici $x_k = R_{k0} - \sum_{j=1}^n R_{kj} x_{N_j} < 0$, což nesplňuje žádný přípustný (a tedy nezáporný) vektor x . $\square$

Poznámka 3.9 (kriterium neomezenosti). Třetí, a poslední, možnost, která pro úlohu lineárního programování může nastat, je neomezenost. To znamená, že účelová funkce utíká do nekonečna podél některé neomezené hrany. Jak to poznáme z ℓ -metody? Protože uvažujeme nezápornost proměnných a přidávali jsme podmínku $\sum_{j=1}^n x_j \leq L$, omezili jsme tak vlastně množinu přípustných řešení. Pokud by tedy úloha byla neomezená, poznáme to tak, že doplňková proměnná, odpovídající přidané nerovnosti, zůstane v bázi a optimální řešení bude závislé na parametru L . Pokud i optimální hodnota bude závislá na parametru L , znamená to, že původní úloha je neomezená a z výsledné tabulky můžeme i vyčíst hranu neomezeného směru.

Poznámka 3.10. Jak je vidět, v ℓ -metodě závisí na pořadí proměnných a omezení. Různou volbou pořadí na začátku tedy můžeme ovlivnit průběh celého algoritmu.

Příklad 3.11. Řešte lineární program

$$\max x_1 + 2x_2$$

za podmínek

$$\begin{aligned} -x_1 + x_2 &\leq 1, & x_1 + 3x_2 &\leq 9, \\ 3x_1 - 4x_2 &\leq 6, & x_2 &\leq \frac{5}{2}, \\ x_1 + x_2 &\leq 4, & x &\geq 0. \end{aligned}$$

Řešení: Sestavíme tabulku a provádíme jednotlivé transformace. Pivoť je zakroužkovaný a v záhlaví jsou nebázické proměnné. Protože první tabulka není ℓ -normální, provádíme Úvodní krok s tím, že použijeme nerovnosti $x_1 + x_2 \leq 4$ ze zadání.

	x_1	x_2			x_1	x_5			x_3	x_5		
x_0	-1	-2	0		x_0	1	2	8	x_0	$\frac{1}{2}$	$\frac{3}{2}$	$\frac{13}{2}$
x_1	-1	0	0		x_1	-1	0	0	x_1	$-\frac{1}{2}$	$\frac{1}{2}$	$\frac{3}{2}$
x_2	0	-1	0		x_2	1	1	4	x_2	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{5}{2}$
x_3	-1	1	1	$\sim$	x_3	$\textcircled{-2}$	-1	-3	x_3	-1	0	0
x_4	3	-4	6		x_4	7	4	22	x_4	$\frac{7}{2}$	$\frac{1}{2}$	$\frac{23}{2}$
x_5	1	$\textcircled{1}$	4		x_5	0	-1	0	x_5	0	-1	0
x_6	1	3	9		x_6	-2	-3	-3	x_6	-1	-2	0
x_7	0	1	$\frac{5}{2}$		x_7	-1	-1	$-\frac{3}{2}$	x_7	$-\frac{1}{2}$	$-\frac{1}{2}$	0

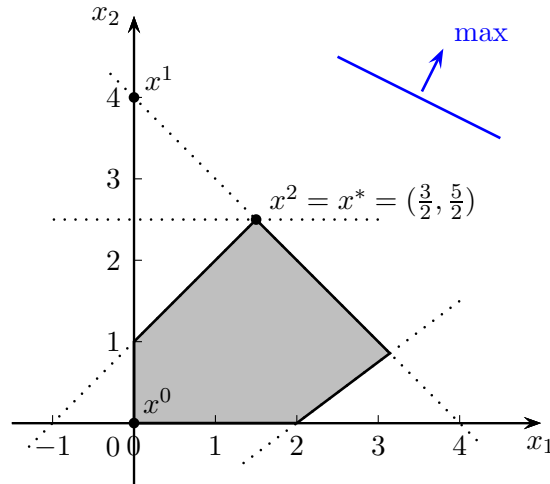
Z výsledné tabulky vyčteme optimální řešení $(x_1^*, x_2^*) = (\frac{3}{2}, \frac{5}{2})$ a optimální hodnotu $\frac{13}{2}$. Dále vidíme, že řešení je silně degenerované, neboť doplňkové proměnné $x_3^* = x_5^* = x_6^* = x_7^* = 0$ (pro nedegenerované řešení by pouze dvě doplňkové proměnné byly nulové).

Postup algoritmu znázorňuje obrázek 3.1. Začínáme v bodě $x^0 = (0, 0)$ a postupujeme skrze bod $x^1 = (0, 4)$ do výsledného optimálního řešení $x^2 = x^* = (\frac{3}{2}, \frac{5}{2})$.

Pro ilustraci ještě ukážeme, jak by metoda pracovala, kdybychom nerovnost $x_1 + x_2 \leq 4$ neuměli odvodit ze zadání. Použijeme nerovnost $x_1 + x_2 \leq L$, kde L reprezentuje libovolně velké číslo.

	x_1	x_2			x_1	x_5			x_3	x_5		
x_0	-1	-2	0		x_0	1	2	$2L$	x_0	$\frac{1}{2}$	$\frac{3}{2}$	$\frac{1}{2} + \frac{3}{2}L$
x_1	-1	0	0		x_1	-1	0	0	x_1	$-\frac{1}{2}$	$\frac{1}{2}$	$-\frac{1}{2} + \frac{1}{2}L$
x_2	0	-1	0		x_2	1	1	L	x_2	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{2} + \frac{1}{2}L$
x_3	-1	1	1	$\sim$	x_3	$\textcircled{-2}$	-1	$1 - L$	x_3	-1	0	0
x_4	3	-4	6		x_4	7	4	$6 + 4L$	x_4	$\frac{7}{2}$	$\frac{1}{2}$	$\frac{19}{2} + \frac{1}{2}L$
x_5	1	1	4		x_5	0	-1	$4 - L$	x_5	0	$\textcircled{-1}$	$4 - L$
x_6	1	3	9		x_6	-2	-3	$9 - 3L$	x_6	-1	-2	$8 - 2L$
x_7	0	1	$\frac{5}{2}$		x_7	-1	-1	$\frac{5}{2} - L$	x_7	$-\frac{1}{2}$	$-\frac{1}{2}$	$2 - \frac{1}{2}L$
x_5	1	$\textcircled{1}$	L									

S proměnnou L pracujeme jako s parametrem, což prakticky lze implementovat tak, že sloupec pravých stran se skládá ze dvou podsloupců – první reprezentuje absolutní členy a druhý koeficienty u L . $\square$

Obrázek 3.1: (Příklad 3.11) Průběh ℓ -metody.

3.4 První Gomoryho algoritmus

Uvažujme čistou úlohu celočíselného programování ve tvaru

$$\max c^T x \text{ za podm. } x \in M \cap \mathbb{Z}^n,$$

kde $A \in \mathbb{Z}^{m \times n}$, $b \in \mathbb{Z}^m$, $c \in \mathbb{Z}^n$ a

$$M = \{x \in \mathbb{R}^n; Ax \leq b, x \geq 0\}.$$

Schema prvního Gomoryho algoritmu¹⁾ [Gomory, 1958] je následující:

Algoritmus 3.12 (První Gomoryho algoritmus (1958)).

- 1: Vyřešíme lineární program $\max_{x \in M} c^T x$ pomocí ℓ -metody. Pokud optimální řešení neexistuje, tak ani původní úloha nemá řešení. Pokud je optimální řešení celočíselné, končíme.
- 2: Jinak zavedeme řez následujícím způsobem. Definujme první řádek s neceločíselnými hodnotami napravo v tabulce jako

$$k := \min\{i = 0, \dots, m+n; R_{i0} \notin \mathbb{Z}\}. \quad (3.3)$$

a nadrovinu

$$H := \{x \in \mathbb{R}^n; \sum_{j=1}^n \{R_{kj}\} x_{N_j} = \{R_{k0}\}\}.$$

K omezením z M přidáme další podmínku

$$x \in H^+ = \{x \in \mathbb{R}^n; \sum_{j=1}^n \{R_{kj}\} x_{N_j} \geq \{R_{k0}\}\}$$

a celý postup opakujeme.

Nyní ukážeme, že nadrovina odřízne aktuální optimální řešení x^* lineárního programu, ale neodřízne žádný celočíselný bod v M .

Věta 3.13 (přípustnost řezu). Platí $x^* \in \text{int } H^-$ a $M \cap \mathbb{Z}^n \subseteq H^+$, kde

$$\text{int } H^- = \{x \in \mathbb{R}^n; \sum_{j=1}^n \{R_{kj}\} x_{N_j} < \{R_{k0}\}\},$$

$$H^+ = \{x \in \mathbb{R}^n; \sum_{j=1}^n \{R_{kj}\} x_{N_j} \geq \{R_{k0}\}\}.$$

¹⁾Gomory původně bádá ve své disertaci na nelineárních diferenciálních rovnicích. K celočíselnému programování se dostal později, když pracoval pro americkou armádu.

Důkaz. 1) Vlastnost $x^* \in \text{int } H^-$ plyne dosazením x^* do nerovnice $\sum_{j=1}^n \{R_{kj}\}x_{N_j}^* < \{R_{k0}\}$, neboť $x_N^* = 0$.
 2) Buď $x \in M \cap \mathbb{Z}^n$ libovolný. Pak

$$x_k = R_{k0} - \sum_{j=1}^n R_{kj}x_{N_j} = \lfloor R_{k0} \rfloor - \sum_{j=1}^n \lfloor R_{kj} \rfloor x_{N_j} + \{R_{k0}\} - \sum_{j=1}^n \{R_{kj}\}x_{N_j}.$$

Z této rovnice vyplývá, že $\sum_{j=1}^n \{R_{kj}\}x_{N_j} - \{R_{k0}\} \in \mathbb{Z}$. Protože ale

$$\sum_{j=1}^n \underbrace{\{R_{kj}\}}_{\geq 0} \underbrace{x_{N_j}}_{\geq 0} - \underbrace{\{R_{k0}\}}_{< 1} > -1,$$

musí $\sum_{j=1}^n \{R_{kj}\}x_{N_j} - \{R_{k0}\} \geq 0$. □

Poznámka 3.14.

1. V algoritmu může nastat i situace $k = 0$, což je v pořádku.
2. Je-li $\{R_{kj}\} = 0$ pro každé $j \in \{1, \dots, n\}$, pak $M \cap \mathbb{Z}^n = \emptyset$. Tedy i když nadrovina není dobře definována, věta stále platí.
3. Přidání omezení $x \in H^+$ prakticky znamená, že do ℓ -tabulky přidáme další řádek

$$x_{m+n+1} \quad \begin{array}{|c|c|c|} \hline \vdots & \vdots & \vdots \\ \hline -\{R_{k1}\} & \dots & -\{R_{kn}\} & -\{R_{k0}\} \\ \hline \end{array}$$

odpovídající omezení $x_{m+n+1} = -\{R_{k0}\} - \sum_{j=1}^n \{R_{kj}\}x_{N_j}$. Pokračujeme ℓ -metodou s tím, že klíčový řádek bude ten poslední.

4. Řez je jednoduchý, ale nepřilíš hluboký.

Tvrzení 3.15. *Po provedení řezu a jedné transformace tabulky lze přidávaný řádek vynechat.*

Důkaz. Sloupce R_j , $j = 1, \dots, n$, tabulky zůstanou lineárně nezávislé, a proto jednoznačnost výběru klíčového sloupce přetrvá. Původní omezení z popisu polyedru M rovněž zůstanou zachovány, tudíž se zachová přípustnost řešení. Nevrátíme se ani zpět k již spočítanému řešení, protože ℓ -tabulka lexikograficky klesá. □

Nyní směřujeme k tomu ukázat, že První Gomoryho algoritmus je konečný. Označme jako R_0^r poslední sloupec výsledné ℓ -tabulky (tj. rozšířené ℓ -optimální řešení) úlohy v r -té iteraci

$$\max \quad c^T x \quad \text{za podm. } x \in M \cap H_1^+ \cap \dots \cap H_r^+.$$

Nechť $R_0^{r'}$ značí poslední sloupec po provedení řezu a jedné transformace tabulky.

Lemma 3.16. *Platí $R_0^r \succ R_0^{r'} \succeq R_0^{r+1}$.*

Důkaz. Plyne z tvrzení 3.5. Rovnost napravo nastane pokud po jedné transformaci tabulky je už $R_0^{r'}$ přímo ℓ -optimální. □

Lemma 3.17. *Existuje vektor $v \in \mathbb{Z}^{m+n+1}$ tak, že $R_0^r \geq v$ pro každé $r = 0, 1, \dots$.*

Důkaz. Stačí volit $v = (\alpha, 0, \dots, 0)^T$, kde α je celočíselná dolní mez na optimální hodnotu. □

Lemma 3.18. *Buď p libovolné takové, že $R_{p0}^r \notin \mathbb{Z}$. Pak*

$$(R_{00}^r, \dots, R_{p-1,0}^r, \lfloor R_{p,0}^r \rfloor) \succeq (R_{00}^{r'}, \dots, R_{p-1,0}^{r'}, R_{p,0}^{r'}). \quad (3.4)$$

Důkaz. Pro jednoduchost vynecháme horní index r a budeme psát R namísto R^r . Řez konstruujeme podle k -tého řádku tabulky a podle (3.3) je $k \leq p$. Definujme

$$q := \min\{i = 0, \dots, m+n; R_{i\ell} \neq 0\}$$

jako index první nenulové hodnoty v ℓ -tém sloupci. Protože $R_{k\ell} \neq 0$, máme $q \leq k \leq p$.

	$R_\ell \succ 0$	R_0
q	0 $R_{q\ell} > 0$	
k	$R_{k\ell} \neq 0$	$R_{k0} \notin \mathbb{Z}$
p		$R_{p0} \notin \mathbb{Z}$
	$-\{R_{k\ell}\}$	$-\{R_{k0}\} < 0$

Rozlišme dva případy:

- (a) Necht' $q < p$. Ze vztahu pro pravý sloupec tabulky po jedné transformaci $R'_0 = R_0 - \frac{-\{R_{k0}\}}{-\{R_{k\ell}\}}R_\ell$ dostáváme speciálně pro q -tou složku (předchozí hodnoty se nezmění)

$$R'_{q0} = R_{q0} - \frac{-\{R_{k0}\}}{-\{R_{k\ell}\}}R_{q\ell} < R_{q0}.$$

Tudíž $(R_{00}, \dots, R_{q0}) \succ (R'_{00}, \dots, R'_{q0})$ a proto i (3.4).

- (b) Necht' $q = k = p$. Ze vztahu pro k -tou složku pravého sloupce tabulky po jedné transformaci dostáváme

$$R'_{k0} = R_{k0} - \frac{-\{R_{k0}\}}{-\{R_{k\ell}\}}R_{k\ell} = R_{k0} - \frac{R_{k\ell}}{\{R_{k\ell}\}}\{R_{k0}\} \leq R_{k0} - \{R_{k0}\} = \lfloor R_{k0} \rfloor,$$

kde jsme využili $R_{k\ell} \geq \{R_{k\ell}\}$. Nyní tedy dostáváme (3.4).

□

Věta 3.19. *První Gomoryho algoritmus je konečný.*

Důkaz. Sporem předpokládejme, že algoritmus je nekonečný a vytvoří posloupnost postupných řešení R^r , $r = 0, 1, \dots$. Podle Lemmatu 3.16 je $R^0 \succ R^2 \succ \dots$. Tudíž dostáváme nerostoucí posloupnost

$$R_{00}^0 \geq R_{00}^1 \geq R_{00}^2 \geq \dots$$

Podle Lemmatu 3.17 je posloupnost zdola omezená. Může existovat index r_0 takový, že posloupnost je konstantní od r_0 -tého členu. Pak je nerostoucí posloupnost

$$R_{10}^{r_0} \geq R_{10}^{r_0+1} \geq R_{10}^{r_0+2} \geq \dots$$

Obecně, uvažujme minimální $s \in \{0, \dots, m+n\}$ takové, že posloupnost $R_{s0}^0, R_{s0}^1, R_{s0}^2, \dots$ se nikde neustálí. Pak od určitého členu je posloupnost nerostoucí, zdola omezená a proto má limitu. Z konvergence posloupnosti tedy existuje $z \in \mathbb{Z}$ takové, že od určitého členu jsou prvky posloupnosti v intervalu $(z, z+1)$. Formálně, existuje r_s takové, že pro všechna $r \geq r_s$ je $R_{s0}^r \in (z, z+1)$. Nyní použijeme Lemma 3.18 pro $p := s$, což je přípustné díky $R_{s0}^r \notin \mathbb{Z}$. Z lemmatu vyplývá, že

$$(R_{00}^r, \dots, R_{s-1,0}^r, \lfloor R_{s,0}^r \rfloor) \succeq (R_{00}^{r'}, \dots, R_{s-1,0}^{r'}, R_{s0}^{r'}),$$

neboli $R_{s0}^{r+1} \leq R_{s0}^{r'} \leq \lfloor R_{s0}^r \rfloor = z$, což je spor.

□

Poznámka 3.20. Pro důkaz konečnosti algoritmu jsme potřebovali konkrétní znalost řezu pouze v Lemmatu 3.18, část (b). Proto pro důkaz konečnosti jiných algoritmů sečných nadrovin stačí pouze adaptovat tuto část důkazu.

Příklad 3.21. Řešte celočíselný lineární program

$$\max x_1 - x_2$$

za podmínek

$$-\frac{1}{3}x_1 + x_2 \leq \frac{1}{3}, \quad x_1 - \frac{1}{3}x_2 \leq \frac{1}{3}, \quad x \geq 0, \quad x \in \mathbb{Z}^2.$$

Řešení: Omezení přepíšeme do tvaru s celočíselnými koeficienty

$$-x_1 + 3x_2 \leq 1, \quad 3x_1 - x_2 \leq 1, \quad x \geq 0, \quad x \in \mathbb{Z}^2.$$

Z nerovnic odvodíme vztah $x_1 + x_2 \leq 1$, který použijeme pro úvodní krok ℓ -metody. Sestavíme ℓ -tabulku a vyřešíme lineární relaxaci ℓ -metodou.

	x_1	x_2			x_5	x_2			x_4	x_2		
x_0	-1	1	0		x_0	1	2	1	x_0	$\frac{1}{3}$	$\frac{2}{3}$	$\frac{1}{3}$
x_1	-1	0	0		x_1	1	1	1	x_1	$\frac{1}{3}$	$-\frac{1}{3}$	$\frac{1}{3}$
x_2	0	-1	0	$\sim$	x_2	0	-1	0	x_2	0	-1	0
x_3	-1	3	1		x_3	1	4	2	x_3	$\frac{1}{3}$	$\frac{8}{3}$	$\frac{4}{3}$
x_4	3	-1	1		x_4	-3	-4	-2	x_4	-1	0	0
x_5	1	1	1									

Protože řešení $(\frac{1}{3}, 0)$ relaxované úlohy nesplňuje celočíselné podmínky, zavedeme řez. Volíme $k = 0$ a řezná nadrovina bude $\frac{1}{3}x_4 + \frac{2}{3}x_2 = \frac{1}{3}$. Přidáme novou podmínku do tabulky a přeoptimalizujeme. Jak vidíme dole, stačí jedna transformace. Protože řešení ještě stále nebude celočíselné, zavedeme druhý řez (pro $k = 1$) a po transformaci tabulky už získáme celočíselné optimální řešení.

	x_4	x_2			x_4	x_6			x_7	x_6		
x_0	$\frac{1}{3}$	$\frac{2}{3}$	$\frac{1}{3}$		x_0	0	1	0	x_0	0	1	0
x_1	$\frac{1}{3}$	$-\frac{1}{3}$	$\frac{1}{3}$		x_1	$\frac{1}{2}$	$-\frac{1}{2}$	$\frac{1}{2}$	x_1	1	-1	0
x_2	0	-1	0	$\sim$	x_2	$\frac{1}{2}$	$-\frac{3}{2}$	$\frac{1}{2}$	x_2	1	-2	0
x_3	$\frac{1}{3}$	$\frac{8}{3}$	$\frac{4}{3}$		x_3	-1	4	0	x_3	-2	5	1
x_4	-1	0	0		x_4	-1	0	0	x_4	-2	1	1
x_6	$-\frac{1}{3}$	$(-\frac{2}{3})$	$-\frac{1}{3}$		x_7	$(-\frac{1}{2})$	$-\frac{1}{2}$	$-\frac{1}{2}$				

Z výsledné tabulky vyčteme optimální celočíselné řešení $(x_1^*, x_2^*) = (0, 0)$ a optimální hodnotu 0.

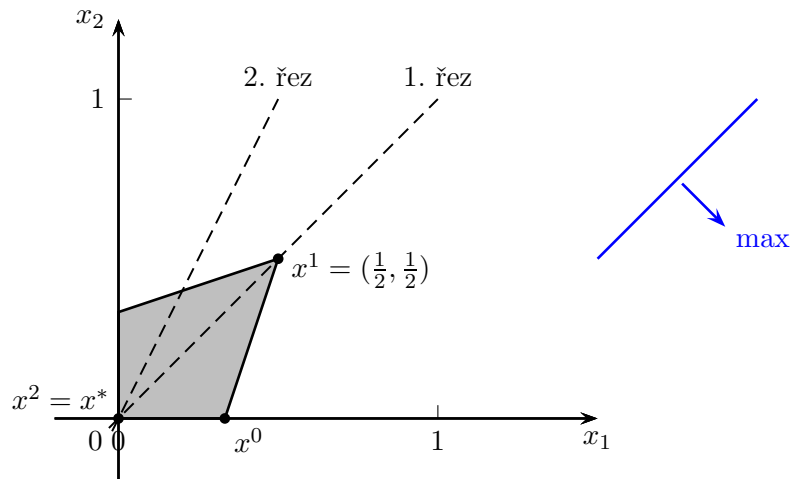
Je užitečné se podrobněji podívat na sečné nadroviny. První měla tvar $\frac{1}{3}x_4 + \frac{2}{3}x_2 = \frac{1}{3}$. Za proměnnou x_4 dosadíme ze čtvrtého řádku úplně první tabulky předpis $x_4 = 1 - 3x_1 + x_2$, čímž dostáváme tvar nadroviny $x_1 - x_2 = 0$ v proměnných x_1, x_2 . Podobně druhá sečná nadrovina měla tvar $\frac{1}{2}x_4 + \frac{1}{2}x_6 = \frac{1}{2}$. Dosazením $x_6 = -\frac{1}{3} + \frac{1}{3}x_4 + \frac{2}{3}x_2$ a $x_4 = 1 - 3x_1 + x_2$ dostaneme tvar $2x_1 - x_2 = 0$ sečné nadroviny.

Postup algoritmu a sečné nadroviny jsou znázorněny na obrázku 3.2. Optimum relaxované úlohy je $x^0 = (\frac{1}{3}, 0)$. Po prvním řezu dospějeme k řešení $x^1 = (\frac{1}{2}, \frac{1}{2})$ a po druhém řezu dojdeme konečně k celočíselnému optimu $x^2 = x^* = (0, 0)$.

Poznamenejme, že kdybychom na začátku nepřeváděli nerovnosti na tvar s celočíselnými koeficienty, tak by algoritmus nepracoval korektně a vrátil chybné řešení (zkuste to!). $\square$

Příklad 3.22. Najděte optimální řešení následujících celočíselných lineárních programů pomocí prvního Gomoryho algoritmu:

$$\begin{aligned} \max \quad & x_1 + x_2 \\ \text{za podm.} \quad & 3x_1 + 6x_2 \leq 10, \\ & x_1, x_2 \geq 0, \\ & x_1, x_2 \in \mathbb{Z}, \end{aligned} \tag{a}$$



Obrázek 3.2: (Příklad 3.21) Průběh prvního Gomoryho algoritmu.

$$\begin{aligned} \max \quad & x_2 \\ \text{za podm.} \quad & 2x_1 + x_2 \leq 7, \\ & -3x_1 + x_2 \leq -1, \\ & x_1, x_2 \geq 0, \\ & x_1, x_2 \in \mathbb{Z}, \end{aligned} \tag{b}$$

Řešení:

- (a) Sestavíme počáteční tabulku ℓ -metody pro nalezení optimálního řešení lineární relaxace. Počáteční tabulka není ℓ -normální (pro sloupce neplatí $R_j \succ 0$), přidáme proto podmínku $x_1 + x_2 \leq L$ a provedeme příslušnou transformaci. Řádek s přidanou podmínkou je klíčový, klíčovým sloupcem je lexikograficky minimální sloupec, tedy první. V dalším kroku je klíčovým řádkem k první řádek se zápornou pravou stranou R_{k0} a klíčový sloupec ℓ volíme podle pravidla

$$\frac{R_\ell}{|R_{k\ell}|} = \text{lex min} \left\{ \frac{R_j}{|R_{kj}|} : R_{kj} < 0, j = 1, \dots, n \right\}.$$

	x_1	x_2			x_4	x_2			x_3	x_2		
x_0	-1	-1	0	$\sim$	x_0	1	0	L	x_0	$\frac{1}{3}$	1	$\frac{10}{3}$
x_1	-1	0	0		x_1	1	1	L	x_1	$\frac{1}{3}$	2	$\frac{10}{3}$
x_2	0	-1	0		x_2	0	-1	0	x_2	0	-1	0
x_3	3	6	10		x_3	$\textcircled{-3}$	3	$10 - 3L$	x_3	-1	0	0
x_4	$\textcircled{1}$	1	L		x_4	-1	0	0	x_4	$-\frac{1}{3}$	-1	$-\frac{10}{3} + L$

Přidaný řádek s omezením můžeme po poslední úpravě vypustit, protože hodnota na pravé straně už bude vždy kladná a řádek tedy nebude klíčový. Poslední sloupec tabulky je nezáporný, optimálním řešením relaxace je $(\frac{10}{3}, 0)$. Toto řešení není celočíselné, zavedeme tedy řez podle prvního řádku s neceločíselnou hodnotou na pravé straně (tj. řádek s proměnnou x_0). Do tabulky přidáváme řez jako řádek ve tvaru

$$-\{R_{k1}\} \dots -\{R_{kn}\} \mid -\{R_{k0}\},$$

kde $\{r\} = r - \lfloor r \rfloor$. Tento řádek volíme jako klíčový.

	x_3	x_2			x_5	x_2	
x_0	$\frac{1}{3}$	1	$\frac{10}{3}$	$\sim$	x_0	1	3
x_1	$\frac{1}{3}$	2	$\frac{10}{3}$		x_1	1	2
x_2	0	-1	0		x_2	0	-1
x_3	-1	0	0		x_3	-3	0
x_5	$(-\frac{1}{3})$	0	$-\frac{1}{3}$		x_5	-1	0

Ve výsledné tabulce vidíme optimální řešení celočíselného programu $(x_1, x_2) = (3, 0)$ s hodnotou účelové funkce 3.

- (b) Opět vytvoříme počáteční tabulku, kterou jednou transformací upravíme na ℓ -normální tabulku a následně hledáme optimální řešení relaxace.

	x_1	x_2			x_1	x_5			x_1	x_3		
x_0	0	-1	0	$\sim$	x_0	1	1	L	x_0	2	1	7
x_1	-1	0	0		x_1	-1	0	0	x_1	-1	0	0
x_2	0	-1	0		x_2	1	1	L	x_2	2	1	7
x_3	2	1	7		x_3	1	$\textcircled{-1}$	$7 - L$	x_3	0	-1	0
x_4	-3	1	-1		x_4	-4	-1	$-1 - L$	x_4	$\textcircled{-5}$	-1	-8
x_5	1	$\textcircled{1}$	L		x_5	0	-1	0	x_5	-1	-1	$-7 + L$

Poslední řádek s přidáním omezením můžeme v tomto kroku opět vypustit. Na pravé straně je záporná hodnota v řádku s x_4 , provedeme proto ještě jednu transformaci s tímto klíčovým řádkem.

	x_1	x_3			x_4	x_3		
x_0	2	1	7	$\sim$	x_0	$\frac{2}{5}$	$\frac{3}{5}$	$\frac{19}{5}$
x_1	-1	0	0		x_1	$-\frac{1}{5}$	$\frac{1}{5}$	$\frac{8}{5}$
x_2	2	1	7		x_2	$\frac{2}{5}$	$\frac{3}{5}$	$\frac{19}{5}$
x_3	0	-1	0		x_3	0	-1	0
x_4	(-5)	-1	-8		x_4	-1	0	0

Získali jsme optimální řešení relaxace $(x_1, x_2) = (\frac{8}{5}, \frac{19}{5})$ s hodnotou účelové funkce $\frac{19}{5}$. Řešení není celočíselné, zavedeme tedy řez podle řádku s x_0 a provedeme další transformaci tabulky.

x_4 x_3			$\sim$	x_6 x_3				
x_0	$\frac{2}{5}$	$\frac{3}{5}$		$\frac{19}{5}$	x_0	1	0	3
x_1	$-\frac{1}{5}$	$\frac{1}{5}$		$\frac{8}{5}$	x_1	$-\frac{1}{2}$	$\frac{1}{2}$	2
x_2	$\frac{2}{5}$	$\frac{3}{5}$		$\frac{19}{5}$	x_2	1	0	3
x_3	0	-1		0	x_3	0	-1	0
x_4	-1	0		0	x_4	$-\frac{5}{2}$	$\frac{3}{2}$	2
x_6	$(-\frac{2}{5})$	$-\frac{3}{5}$		$-\frac{4}{5}$	x_6	-1	0	0

Výsledné řešení $(x_1, x_2) = (2, 3)$ už je celočíselné, je tedy optimálním řešením zadané úlohy. $\square$

3.5 Druhý Gomoryho algoritmus

Uvažujme smíšenou úlohu celočíselného programování ve tvaru

$$\max c^T x \text{ za podm. } x \in M_C,$$

kde $A \in \mathbb{Q}^{m \times n}$, $b \in \mathbb{Q}^m$, $c \in \mathbb{Q}^n$, $C \subseteq \{1, \dots, n\}$ a

$$M = \{x \in \mathbb{R}^n; Ax \leq b, x \geq 0\},$$

$$M_C = \{x \in M; x_i \in \mathbb{Z} \text{ pro } i \in C\}.$$

Pro důkaz konečnosti se navíc předpokládá, že $C = \{1, \dots, n_0\}$ pro nějaké $n_0 \leq n$ a $x_0 = c^T x \in \mathbb{Z}$ (druhá podmínka už není na újmu obecnosti).

Schema druhého Gomoryho algoritmu [Gomory, 1960] je následující:

Algoritmus 3.23 (Druhý Gomoryho algoritmus (1960)).

- 1: Vyřešíme lineární program $\max_{x \in M} c^T x$ pomocí ℓ -metody. Pokud optimální řešení neexistuje, tak ani původní úloha nemá řešení.

Pokud optimální řešení splňuje celočíselné podmínky, končíme.

- 2: Jinak zavedeme řez následujícím způsobem. Definujeme

$$k := \arg \min \{i \in C; R_{i0} \notin \mathbb{Z}\}.$$

a nadrovinu

$$H := \{x \in \mathbb{R}^n; \sum_{j=1}^n \gamma_j x_{N_j} = \{R_{k0}\}\},$$

kde γ_j , $j \in N$, jsou definovány

$$\gamma_j = \begin{cases} \{R_{kj}\} & N_j \in C, \{R_{k0}\} \geq \{R_{kj}\}, \\ \frac{\{R_{k0}\}}{1 - \{R_{k0}\}}(1 - \{R_{kj}\}) & N_j \in C, \{R_{k0}\} < \{R_{kj}\}, \\ R_{kj} & N_j \notin C, R_{kj} \geq 0, \\ \frac{\{R_{k0}\}}{1 - \{R_{k0}\}}(-R_{kj}) & N_j \notin C, R_{kj} < 0. \end{cases}$$

$$(3.5a)$$

$$(3.5b)$$

$$(3.5c)$$

$$(3.5d)$$

K omezením z M přidáme další podmínku

$$x \in H^+ = \{x \in \mathbb{R}^n; \sum_{j=1}^n \gamma_j x_{N_j} \geq \{R_{k0}\}\}$$

a celý postup opakujeme.

Nadrovina odřízne aktuální optimální řešení x^* lineárního programu, ale neodřízne žádný přípustný bod v M_C . Nejprve dokážeme pomocné lemma.

Lemma 3.24. *Uvažujme omezení*

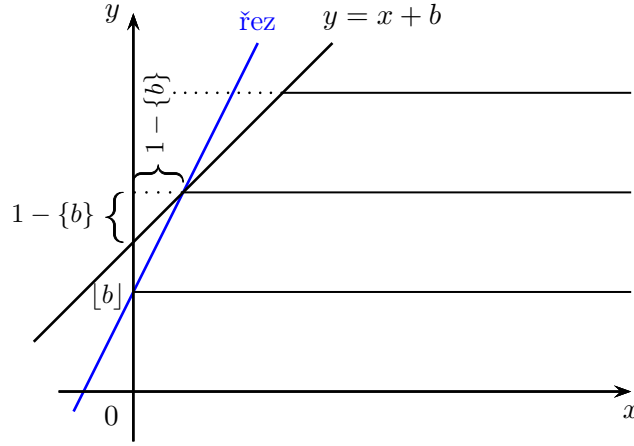
$$y \leq x + b, \quad x, y \geq 0, \quad x \in \mathbb{R}, \quad y \in \mathbb{Z},$$

kde $b \notin \mathbb{Z}$. Pak následující řezná nerovnost je přípustná

$$y \leq \frac{1}{1 - \{b\}}x + \lfloor b \rfloor.$$

Důkaz. Neformálně je vidět platnost řezu na obrázku 3.3. Řezná přímka musí procházet body $(0, \lfloor b \rfloor)$ a $(1 - \{b\}, \lfloor b \rfloor)$ a mít směrnici $\frac{1}{1 - \{b\}}$. Pro formální důkaz rozlišme dva případy:

- 1) Je-li $y \leq \lfloor b \rfloor$, pak je tvrzení jasné, neboť ve vyjádření nerovnice jako $y - \lfloor b \rfloor \leq \frac{1}{1 - \{b\}}x$ je levá strana nekladná a pravá nezáporná.



Obrázek 3.3: (Věta 3.24) Podstata řezné přímky.

2) Necht' $y > [b]$, to jest, $y \geq [b] + 1 = [b]$. Nyní upravme

$$\begin{aligned} y - b &\leq x, \\ y - [b] + (1 - \{b\}) &\leq x, \\ (1 - \{b\})(y - [b]) + (1 - \{b\}) &\leq x, \\ (1 - \{b\})(y - [b]) &\leq x, \end{aligned}$$

z čehož vyplývá výsledná řezná nerovnost. □

Tvrzení 3.25 (Přípustnost Druhého Gomoryho řezu). *Platí $x^* \in \text{int } H^-$ a $M_C \subseteq H^+$, kde*

$$\begin{aligned} H^- &= \{x \in \mathbb{R}^n; \sum_{j=1}^n \gamma_j x_{N_j} < \{R_{k0}\}\}, \\ H^+ &= \{x \in \mathbb{R}^n; \sum_{j=1}^n \gamma_j x_{N_j} \geq \{R_{k0}\}\}. \end{aligned}$$

Důkaz. Vlastnost $x^* \in \text{int } H^-$ plyne dosazením x^* do nerovnice $\sum_{j=1}^n \gamma_j x_{N_j}^* < \{R_{k0}\}$, neboť $x_N^* = 0$.

Nyní se zaměříme na vlastnost $M_C \subseteq H^+$. Buď $x \in M_C$ libovolný pevný. Podle k -tého řádku tabulky platí

$$x_k = R_{k0} - \sum_{j=1}^n R_{kj} x_{N_j}.$$

Rovnici přepíšeme jako dvě nerovnice

$$x_k + \sum_{j=1}^n R_{kj} x_{N_j} \leq R_{k0}, \quad (3.6)$$

$$-x_k - \sum_{j=1}^n R_{kj} x_{N_j} \leq -R_{k0}. \quad (3.7)$$

První nerovnici upravíme na

$$x_k + \sum_{(3.5a)} R_{kj} x_{N_j} + \sum_{(3.5b)} R_{kj} x_{N_j} + \sum_{(3.5c)} R_{kj} x_{N_j} + \sum_{(3.5d)} R_{kj} x_{N_j} \leq R_{k0}$$

a dále na

$$x_k + \sum_{(3.5a)} [R_{kj}] x_{N_j} + \sum_{(3.5b)} [R_{kj}] x_{N_j} + \sum_{(3.5c)} 0 \leq R_{k0} + \sum_{(3.5b)} (1 - \{R_{kj}\}) x_{N_j} - \sum_{(3.5d)} R_{kj} x_{N_j}.$$

Nyní použijeme Lemma 3.24, proměnné na pravé straně nerovnosti mají nezáporné koeficienty, proto můžeme lemma použít. Tím odvodíme platnou nerovnost

$$x_k + \sum_{(3.5a)} [R_{kj}] x_{N_j} + \sum_{(3.5b)} [R_{kj}] x_{N_j} + \sum_{(3.5c)} 0 \leq [R_{k0}] + \sum_{(3.5b)} \frac{1 - \{R_{kj}\}}{1 - \{R_{k0}\}} x_{N_j} - \sum_{(3.5d)} \frac{R_{kj}}{1 - \{R_{k0}\}} x_{N_j}.$$

K této nerovnici přičteme nerovnici (3.7) a dostaneme

$$\sum_{(3.5a)} -\{R_{kj}\}x_{N_j} + \sum_{(3.5b)} ?x_{N_j} + \sum_{(3.5c)} -R_{kj}x_{N_j} + \sum_{(3.5d)} ?x_{N_j} \leq -\{R_{k0}\}.$$

Koeficient v sumě odpovídající (3.5b) je

$$\lceil R_{kj} \rceil - R_{kj} - \frac{1 - \{R_{kj}\}}{1 - \{R_{k0}\}} = 1 - \{R_{kj}\} - \frac{1 - \{R_{kj}\}}{1 - \{R_{k0}\}} = \frac{-\{R_{k0}\}}{1 - \{R_{k0}\}}(1 - \{R_{kj}\}).$$

Koeficient v sumě odpovídající (3.5d) je

$$\frac{R_{kj}}{1 - \{R_{k0}\}} - R_{kj} = \frac{\{R_{k0}\}}{1 - \{R_{k0}\}} R_{kj}.$$

Odvodili jste tedy platnou nerovnost $\sum_{j=1}^n -\gamma_j x_{N_j} \leq -\{R_{k0}\}$. □

Poznámka 3.26. Proč jsou nutné 4 varianty (3.5a)–(3.5d) v popisu řezné nadroviny v druhém Gomoryho algoritmu? Nejprve musíme rozlišit diskrétní proměnné a spojité proměnné. Případy (3.5c)–(3.5d) pak rozlišujeme, protože pro použití Lemmatu 3.24 potřebujeme kladný koeficient u spojitě proměnné. Případy (3.5a)–(3.5b) rozlišujeme, aby byl řez co nejhlubší: Protože pro aktuální řešení x^* platí $x_N^* = 0$, je řez tím hlubší, čím je nadrovina H dále od počátku. Chceme tedy koeficienty γ_j co nejmenší. Kdy nastane (3.5a) ≤ (3.5b)? Právě tehdy, když

$$\{R_{kj}\} \leq \frac{\{R_{k0}\}}{1 - \{R_{k0}\}}(1 - \{R_{kj}\}),$$

neboli po úpravě

$$\{R_{kj}\}(1 - \{R_{k0}\}) \leq \{R_{k0}\}(1 - \{R_{kj}\}),$$

čili $\{R_{kj}\} \leq \{R_{k0}\}$. To je přesně podmínka na rozlišení případů (3.5a)–(3.5b).

Poznámka 3.27.

1. Po provedení řezu a jedné transformace tabulky lze přidáný řádek vynechat.
2. Řez je konstruován tak, aby fungoval pro smíšenou úlohu a aby byl co nejhlubší. Přesto hloubka řezu není nijak závratná.
3. Je-li $\gamma_j = 0$ pro všechna $j \in \{1, \dots, n\}$, pak $M_C = \emptyset$, tedy řez je přípustný, i když nadrovina sama o sobě není dobře definována.
4. Přidání omezení $x \in H^+$ prakticky znamená, že do ℓ -tabulky přidáme další řádek

$$x_{m+n+1} \quad \begin{array}{|c|c|} \hline \vdots & \vdots \\ \hline -\gamma_1 & \dots & -\gamma_n & -\{R_{k0}\} \\ \hline \end{array}$$

odpovídající omezení $x_{m+n+1} = -\{R_{k0}\} - \sum_{j=1}^n (-\gamma_j)x_{N_j}$. Pokračujeme ℓ -metodou s tím, že klíčový řádek bude ten poslední.

Příklad 3.28. Řešte příklad 3.21 pomocí druhého Gomoryho algoritmu.

Řešení: Uvažovaná úloha má tvar

$$\max x_1 - x_2 \quad \text{za podm.} \quad -\frac{1}{3}x_1 + x_2 \leq \frac{1}{3}, \quad x_1 - \frac{1}{3}x_2 \leq \frac{1}{3}, \quad x \geq 0, \quad x \in \mathbb{Z}^2.$$

Omezení nemusíme přepisovat do tvaru s celočíselnými koeficienty; ve stávajícím tvaru máme $C = \{0, 1, 2\}$. Dále využijeme toho, že z nerovnic vyplývá vztah $x_1 + x_2 \leq 1$, který použijeme pro úvodní krok ℓ -metody. Sestavíme ℓ -tabulku a vyřešíme lineární relaxaci ℓ -metodou.

	x_1	x_2			x_5	x_2			x_4	x_2		
x_0	-1	1	0		x_0	1	2	1	x_0	1	$\frac{2}{3}$	$\frac{1}{3}$
x_1	-1	0	0		x_1	1	1	1	x_1	1	$-\frac{1}{3}$	$\frac{1}{3}$
x_2	0	-1	0	$\sim$	x_2	0	-1	0	x_2	0	-1	0
x_3	$-\frac{1}{3}$	1	$\frac{1}{3}$		x_3	$\frac{1}{3}$	$\frac{4}{3}$	$\frac{2}{3}$	x_3	$\frac{1}{3}$	$\frac{8}{9}$	$\frac{4}{9}$
x_4	1	$-\frac{1}{3}$	$\frac{1}{3}$		x_4	$\textcircled{-1}$	$-\frac{4}{3}$	$-\frac{2}{3}$	x_4	-1	0	0
x_5	$\textcircled{1}$	1	1									

Protože řešení $(\frac{1}{3}, 0)$ relaxované úlohy nesplňuje celočíselné podmínky, zavedeme řez. Volíme $k = 0$ a řezná nerovnice má tvar $\sum_{j=1}^n \gamma_j x_{n_j} \leq -\{R_{k0}\} \leq -\frac{1}{3}$. Výpočet γ_j pro $j = 1, 2$:

1. Protože $N_1 = 4 \notin C$ a $R_{k1} = R_{01} = 1 \geq 0$, nastane třetí případ z definice γ_j a položíme $\gamma_1 = 1$.
2. Protože $N_2 = 2 \in C$ a $\frac{1}{3} = R_{k0} < R_{k2} = \frac{2}{3}$, nastane druhý případ z definice γ_j a položíme $\gamma_2 = \frac{1}{6}$.

Přidáme novou podmínku do tabulky a přeoptimalizujeme.

	x_4	x_2			x_6	x_2		
x_0	1	$\frac{2}{3}$	$\frac{1}{3}$	$\sim$	x_0	1	$\frac{1}{2}$	0
x_1	1	$-\frac{1}{3}$	$\frac{1}{3}$		x_1	1	$-\frac{1}{2}$	0
x_2	0	-1	0		x_2	0	-1	0
x_3	$\frac{1}{3}$	$\frac{8}{9}$	$\frac{4}{9}$		x_3	$\frac{1}{3}$	$\frac{5}{6}$	$\frac{1}{3}$
x_4	-1	0	0		x_4	-1	$\frac{1}{6}$	$\frac{1}{3}$
x_6	$\textcircled{-1}$	$-\frac{1}{6}$	$-\frac{1}{3}$					

Jak vidíme, stačí jedna transformace a dostaneme optimální celočíselné řešení $(x_1^*, x_2^*) = (0, 0)$ a optimální hodnotu 0.

Jak konkrétně vypadá řezná nadrovina? Podle předpisu je tvaru $-x_4 - \frac{1}{6}x_2 = -\frac{1}{3}$. Proměnnou x_4 vyjádříme z první tabulky jako $x_4 = \frac{1}{3} - x_1 + \frac{1}{3}x_2$ a dosazením dostáváme popis sečné nadroviny v původních proměnných jako $-\frac{1}{3} + x_1 - \frac{1}{3}x_2 - \frac{1}{6}x_2 = -\frac{1}{3}$, neboli $x_1 - \frac{1}{2}x_2 = 0$. Je to tedy stejný řez, který První Gomoryho algoritmus sestrojil až v druhé iteraci. $\square$

Poznámka 3.29. Historii Gomoryho řezů provází ve vědecké komunitě oscilace mezi přijetím a odmítáním. Nejprve byly považovány za zázračné, pak zcela odmítnuty ve prospěch metod branch & bound (kapitola 4) a nakonec opět povolány do boje s celočíselnými programy. Klíčovou novou myšlenkou, kromě řady jiných, bylo aplikovat více řezů najednou (viz sekce 4.3); najít vhodnou kombinaci řezů je však stále otevřený problém. Úspěšná byla i kombinovace s jinými přístupy a také se celkem podařilo zvládnout zásadní problémy se zaokrouhlováním v tabulce. Cornuéjols [2008] uvádí, že aplikací všech řezů ze simplexové tabulky snížilo vzdálenost k optimální hodnotě v průměru o 24 %.

3.6 Další řezy

Pěkný přehled různých řezů pro smíšenou úlohu celočíselného programování uvádí např. Cornuéjols [2008].

3.6.1 Integer rounding a Chvátalovy–Gomoryho řezy

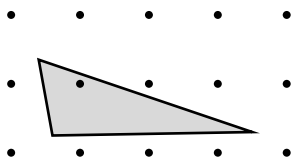
Příklad 3.30. Uvažme

$$M = \{x \in \mathbb{Z}^4; \frac{13}{11}x_1 + \frac{20}{11}x_2 + x_3 + \frac{6}{11}x_4 \geq \frac{72}{11}\}.$$

Zaokrouhlením koeficientů vlevo nahoru, a poté pravé strany rovněž nahoru dostaneme platnou nerovnost (splněnou pro každý bod z M)

$$2x_1 + 2x_2 + x_3 + x_4 \geq 7.$$

Tuto nerovnost odřízne např. bod $x = (0, 0, \frac{72}{11}, 0)^T$, který ta původní nerovnost splňuje. Pro lineární relaxaci má tedy smysl generovat takovéto řezy. $\square$



Obrázek 3.4: Ilustrace příkladu, kdy ani všechny Chvátalovy–Gomoryho řezy nezmenší M na M_I , a tedy Chvátalův rank je větší než 1. Právý vrchol trojúhelníku totiž neodřízneme více než o zhruba zlomkovou část a nedostaneme se mimo okolní celočíselné body.

Postup z příkladu můžeme zobecnit. Ve shodě se sekci 1.2 uvažme polyedr

$$M = \{x \in \mathbb{R}^n; Ax \leq b, x \geq 0\}$$

a konvexní obal celočíselných bodů

$$M_I := \text{conv}(M \cap \mathbb{Z}^n).$$

Pak pro libovolné $u \geq 0$ je $u^T Ax \leq u^T b$ platná nerovnost pro M , a

$$\lfloor u^T A \rfloor x \leq \lfloor u^T b \rfloor$$

je platná nerovnost pro M_I , neodřízne tedy žádný celočíselný bod z M . Tato nerovnice se nazývá Chvátalův–Gomoryho řez. Přidáním všech takovýchto nerovnic pro všechna možná $u \geq 0$ zmenšíme M na nějaký menší polyedr (není vidno na první pohled, že je to polyedr). Nemusíme ještě dostat M_I (viz obrázek 3.4), ale pokud celý postup opakujeme, po konečně mnoha iteracích (tzv. Chvátalův rank) dospějeme k M_I . Toto tvrdí slavná Chvátalova věta z r. 1973, a platí i pro reálná data.

Příklad 3.31.

1. Najděte Chvátalův–Gomoryho řez, který odřízne bod $(0, \frac{25}{6}, 0, 0, 0)$ od polyedru

$$M = \{x \in \mathbb{Z}^5; 9x_1 + 12x_2 + 8x_3 + 17x_4 + 13x_5 \geq 50, x \geq 0\}.$$

2. Najděte Chvátalův–Gomoryho řez, který odřízne bod $(20, 5, \frac{25}{8})$ od polyedru

$$M = \{(x, y) \in \mathbb{R}^2 \times \mathbb{Z}; x_1 + x_2 \leq 25, x_1 + x_2 \leq 8y, (x, y) \geq 0\}.$$

□

3.6.2 Generování logických nerovností

V 0-1 programování lze uvažovat i opačným směrem – generovat nové jednoduché omezení. Smysl těchto omezení je stejný jako u sečných nadrovin, nevyrábět redundantní nerovnice, ale omezení, která potenciálně odříznou část relaxovaného polyedru. Postup se nejlépe vysvětlí na příkladu.

Příklad 3.32. Uvažme celočíselný systém

$$\begin{aligned} 7x_1 + 3x_2 - 4x_3 - 2x_4 &\leq 1, \\ -2x_1 + 7x_2 + 3x_3 + x_4 &\leq 6, \\ -2x_2 - 3x_3 - 6x_4 &\leq -5, \\ -3x_1 + 2x_3 &\leq 1, \\ x &\in \{0, 1\}^4. \end{aligned}$$

Z první nerovnice můžeme odvodit podmínky:

- Je-li $x_1 = 1$, pak $x_3 = 1$ (neboli $x_1 \leq x_3$).

- Je-li $x_1 = 1$, pak $x_4 = 1$ (neboli $x_1 \leq x_4$).
- Nemohou obě x_1, x_2 být zároveň rovny 1 (neboli $x_1 + x_2 \leq 1$).

Z druhé nerovnice podobně odvodíme podmínky:

- $x_1 \geq x_2, x_2 + x_3 \leq 1$.

Z třetí nerovnice odvodíme podmínky:

- $x_2 + x_4 \geq 1, x_3 + x_4 \leq 1$.

Z poslední nerovnice pak odvodíme podmínku:

- $x_1 \geq x_3$.

Zkombinováním podmínek dostaneme:

- $x_1 \geq x_3, x_1 \leq x_3$, tedy $x_1 = x_3$.
- $x_1 + x_2 \leq 1, x_1 \geq x_2$, tedy $x_2 = 0$.
- $x_2 + x_4 \geq 1, x_2 = 0$, tedy $x_4 = 1$.

Tudíž v zásadě bez výpočtu jsme odvodili $x_1 = x_3, x_2 = 0, x_4 = 1$, což skutečně dává všechna přípustná řešení $(1, 0, 1, 1)$ a $(0, 0, 0, 1)$. $\square$

3.6.3 Perfektní párování a květinové nerovnosti

Edmondsův algoritmus [Edmonds, 1965] na nalezení perfektního párování je založen na lineárním programování, dualitě a sečných nadrovinách. Sečné nadroviny generujeme z faktu, že z množiny vrcholů liché velikosti musí vycházet aspoň jedna hrana. Navíc se dá ukázat, že všechny takovéto nerovnosti stačí k tomu, abychom původní relaxovaný polyedr ořízli na konvexní obal perfektních párování. S pomocí duality pak získáme polynomiální algoritmus.

Problém maximálního párování v grafu $G = (V, E)$ má formulaci

$$\max \sum_{e \in E} x_e \text{ za podm. } Ax \leq e, x \in \{0, 1\}^m,$$

kde $x_e = 1$ odpovídá hraně v párování a $x_e = 0$ jinak. Platná sečná nadrovina pak pro libovolnou množinu vrcholů $T \subseteq V$ liché velikosti má tvar $\sum_{e \in E(T)} x_e \leq \frac{1}{2}(|T| - 1)$, kde $E(T)$ jsou hrany grafu G s oběma vrcholy uvnitř T . I když je těchto nerovnic exponenciálně mnoho, chytrým výběrem lze dostat polynomiální algoritmus.

Cvičení

- 3.1. Vyřešte příklad 3.21 pomocí prvního a druhého Gomoryho algoritmu a *primární* simplexové metody.
- 3.2. (a) Jakým způsobem lze při použití ℓ -metody rozpoznat neomezené lineární programy?
(b) Jak můžeme rozpoznat nepřipustnou lineární relaxaci, případně nepřipustný celočíselný program při řešení úlohy Gomoryho algoritmem?
- 3.3. V prvním Gomoryho algoritmu po zavedení sečné nadroviny a provedení transformace tabulky podle nového řádku pak tento řádek vynecháme. Může se stát, že se někdy později dostaneme zase na druhou stranu této sečné nadroviny? Obecně, jak ovlivní vynechání řádku průběh algoritmu?
- 3.4. Adaptujte první Gomoryho algoritmus tak, abychom se vyhnuli řezům účelové funkce (tj. $k > 0$). Jak to bude s konečností? Dokážeme se jednoduše vyhnout i řezům pro doplňkové proměnné?
- 3.5. Zobecněte první Gomoryho algoritmus pro smíšenou úlohu. Vodítkem vám může být druhý Gomoryho algoritmus a důkaz správnosti sečné nadroviny.
- 3.6. Pro druhý Gomoryho algoritmus dokažte konečnost (za vhodných předpokladů). Stačí analogii Lemmatu 3.18 z prvního Gomoryho algoritmu.
Dále, diskutujte konečnost pro případ, že účelová funkce nemusí nabývat celočíselných hodnot.

- 3.7. Ovlivní nějak průběh prvního resp. druhého Gomoryho algoritmu to, že soustavu nerovnic předem přenásobím nějakým kladným (a např. celým) číslem?
- 3.8. Najděte platnou nerovnost (vzhledem k dané množině X), která odřízne bod x^* :
- (a) množina $X = \{(x_1, x_2) \in \mathbb{Z} \times \{0, 1\} : 2x_1 - x_2 \leq 2\}$ a bod $x^* = (1.5, 1)^T$,
 - (b) množina $X = \{(x_1, x_2) \in \mathbb{R}_+ \times \mathbb{Z}_+ : x_1 \leq 9, x_1 \leq 4x_2\}$ a bod $x^* = (9, \frac{9}{4})^T$.
- 3.9. Dokažte, že sečnou nadrovinu čisté úlohy celočíselného programu lze definovat např. také následujícím způsobem: Buď $d_{k0} \notin \mathbb{Z}$, tj. výsledná l -tabulka nemá napravo celé číslo v k -tém řádku. Sečná nadrovina je pak tvaru $\sum_{j \in I} x_{N_j} = 1$, kde $I = \{j = 1, \dots, n; d_{kj} \notin \mathbb{Z}\}$ (množina indexů prvků v k -tém řádku, které nejsou celočíselné).
- 3.10. Buď $f > 0$ dáno. Najděte minimální popis konvexního obalu množiny

$$M = \{(x, y) \in \mathbb{Z} \times \mathbb{R}; x - y \leq f, y \geq 0\}$$

pomocí soustavy lineárních nerovnic.

Kapitola 4

Metody branch & bound

Myšlenka metody branch & bound („větvení a mezí“) byla nezávisle dotknuta několika autory během 50. let 20. století. První úplný algoritmus byl formulován v legendárním článku Land and Doig [1960]¹⁾, který myšlenku proslavil, ale první rozumná implementace se objevuje až v Dakin [1965]. Název „branch & bound“ pochází od Little et al. [1963], kteří jej vymysleli pro rekurzivní metodu na řešení problému obchodního cestujícího.

Uvažujme smíšenou úlohu celočíselného programování ve tvaru

$$\max c^T x \text{ za podm. } x \in M_C,$$

kde $A \in \mathbb{Q}^{m \times n}$, $b \in \mathbb{Q}^m$, $c \in \mathbb{Q}^n$, $C \subseteq \{1, \dots, n\}$ a

$$M = \{x \in \mathbb{R}^n; Ax \leq b, x \geq 0\},$$
$$M_C = \{x \in M; x_i \in \mathbb{Z} \text{ pro všechna } i \in C\}.$$

Schema metod založených na branch & bound je následující, viz obrázek 4.1.

Algoritmus 4.1 (Branch & bound).

1: Vyřešíme lineární program $\max_{x \in M} c^T x$.

Pokud optimální řešení neexistuje, tak ani původní úloha nemá řešení.

Pokud je optimální řešení $x^0 \in M_C$, končíme.

2: Jinak zvol $k \in C$ takové, že $x_k^0 \notin \mathbb{Z}$ a definuj dva konvexní polyedry

$$M_1 := \{x \in M; x_k \leq \lfloor x_k^0 \rfloor\},$$

$$M_2 := \{x \in M; x_k \geq \lfloor x_k^0 \rfloor + 1\}.$$

Rekurzivně vyřeš dvě podúlohy na polyedrech M_1 resp. M_2 a vyber lepší řešení.

Pro řešení lineárních relaxací je opět výhodné použít duální simplexovou metodu (např. ℓ -metodu), protože potřebujeme rychle přeoptimalizovat po přidání nové podmínky $x_k \leq \lfloor x_k^0 \rfloor$ resp. $x_k \geq \lfloor x_k^0 \rfloor + 1$. Pro $k \in C$ takové, že $x_k^0 = R_{k0} \notin \mathbb{Z}$, k -tý řádek ℓ -tabulky má tvar

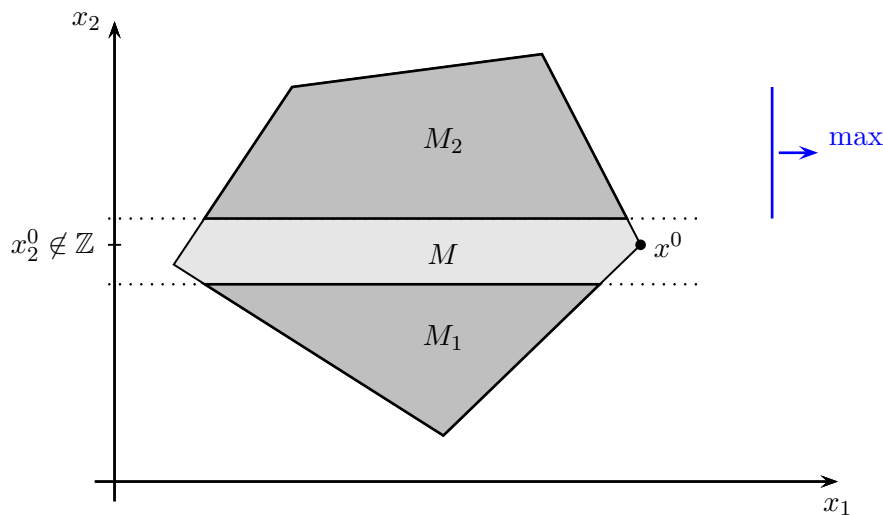
$$x_k = R_{k0} - \sum_{j=1}^n R_{kj} x_{N_j}.$$

Podmínka $x_k \leq \lfloor x_k^0 \rfloor$ pak vede na novou nerovnici

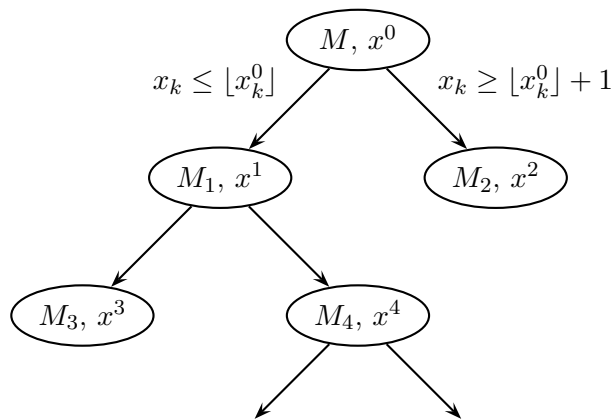
$$-\sum_{j=1}^n R_{kj} x_{N_j} \leq -\{R_{k0}\}, \quad (4.1)$$

a podobně podmínka $x_k \geq \lfloor x_k^0 \rfloor + 1$ vede na novou nerovnici

$$\sum_{j=1}^n R_{kj} x_{N_j} \leq \{R_{k0}\} - 1.$$



Obrázek 4.1: Myšlenka metody branch & bound.



Obrázek 4.2: Strom reprezentující volání podúloh metody branch & bound.

Čili do ℓ -tabulky přidáme koeficienty jedné z těchto nerovnic.

Poznámka 4.2 (Meze). Algoritmus 4.1 popisuje schema větvení. Metoda tedy prochází binární strom reprezentující volání podúloh, viz obrázek 4.2. Pro účinnou implementaci, abychom neprocházeli zbytečně všechny uzly stromu, potřebujeme efektivně zohlednit také „meze“. Nejjednodušším způsobem je pamatovat si aktuálně nejlepší přípustné řešení $x^* \in M_C$ (jakmile nějaké objevíme) a příslušnou hodnotu účelové funkce $z^* := c^T x^*$, která dává globální *dolní* mez na optimální hodnotu f^* . Naopak, horní mez na optimální hodnotu podúlohy dává příslušná lineární relaxace. Proto můžeme zavést test:

Je-li optimální hodnota lineární relaxace dané podúlohy menší nebo rovna z^* , pak podúlohu nemusím řešit (podstrom ve výpočetním stromu odřízneme).

Poznámka 4.3 (Základní vlastnosti).

- Jednoduše řeší smíšené úlohy, zejména je-li málo celočíselných proměnných.
- Je-li M omezený (srov. Poznámka 1.26), pak algoritmus je konečný. Každá proměnná totiž nabývá konečně mnoho hodnot a tudíž se podle ní mohou větvit konečně mnohokrát.
- Dá se snadno paralelizovat.

¹⁾Ailsa Landová a Alison Doigová, matematicky z Londýnské školy ekonomie.

- Narozdíl od sečných nadrovin není tak citlivý na zaokrouhlovací chyby.
- Dá se jednoduše použít i pro nelineární úlohy celočíselného programování.
- Na druhou stranu, může být hodně pomalé a náročné na paměť.

Poznámka 4.4 (0-1 programování). Pro 0-1 programování (sekce 1.1) mají podúlohy tvar

$$M_1 := \{x \in M; x_k = 0\},$$

$$M_2 := \{x \in M; x_k = 1\}.$$

S každou hladinou zanoření ve výpočetním stromu se tedy zafixuje jedna proměnná a dimenze úlohy (= počet proměnných) klesá.

Příklad 4.5. Tento příklad ukazuje, že metoda branch & bound může být časově náročná (exponenciální) i pro jednoduchou úlohu. Pro n liché uvaž

$$\max \sum_{i=1}^n 2x_i \quad \text{za podm.} \quad \sum_{i=1}^n 2x_i \leq n, \quad x \in \{0, 1\}^n.$$

Vidíme, že optimální řešení je např. prvních $\frac{1}{2}(n-1)$ proměnných rovno jedné, a ostatní nula; optimální hodnota je $n-1$. Na druhou stranu, lineární relaxace dá optimální hodnotu n pro každý uzel stromu až do hladiny $\frac{1}{2}(n-1)$. Musíme tudíž projít strom až do hloubky $\frac{1}{2}(n+1)$ a navštívit $2^{\frac{1}{2}(n+1)}$ uzlů. $\square$

Větvení jde uvažovat ještě v řadě jiných variant. Např. původní algoritmus Land and Doig [1960] uvažoval nejprve dvě podúlohy se zafixováním $x_k := \lfloor x_k^0 \rfloor$ resp. $x_k := \lfloor x_k^0 \rfloor + 1$, a teprve potom v případě potřeby další hodnoty proměnné x_k .

Příklad 4.6. Řešte příklad 3.21 pomocí metody branch & bound.

Řešení: Uvažovaná úloha má tvar

$$\max x_1 - x_2 \quad \text{za podm.} \quad -\frac{1}{3}x_1 + x_2 \leq \frac{1}{3}, \quad x_1 - \frac{1}{3}x_2 \leq \frac{1}{3}, \quad x \geq 0, \quad x \in \mathbb{Z}^2.$$

Podobně jako v příkladu 3.28 vyřešíme nejprve lineární relaxaci ℓ -metodou.

	x_1	x_2			x_5	x_2			x_4	x_2		
x_0	-1	1	0		x_0	1	2	1	x_0	1	$\frac{2}{3}$	$\frac{1}{3}$
x_1	-1	0	0		x_1	1	1	1	x_1	1	$-\frac{1}{3}$	$\frac{1}{3}$
x_2	0	-1	0	$\sim$	x_2	0	-1	0	x_2	0	-1	0
x_3	$-\frac{1}{3}$	1	$\frac{1}{3}$		x_3	$\frac{1}{3}$	$\frac{4}{3}$	$\frac{2}{3}$	x_3	$\frac{1}{3}$	$\frac{8}{9}$	$\frac{4}{9}$
x_4	1	$-\frac{1}{3}$	$\frac{1}{3}$		x_4	$\textcircled{-1}$	$-\frac{4}{3}$	$-\frac{2}{3}$	x_4	-1	0	0
x_5	$\textcircled{1}$	1	1									

Protože řešení $x^0 = (\frac{1}{3}, 0)$ relaxované úlohy nesplňuje celočíselné podmínky, rozdělíme úlohu na dvě podúlohy. Celočíselnost porušuje pouze první proměnná, proto zvolíme $k = 1$. První podúloha vznikne přidáním podmínky $x_k \leq \lfloor x_k^0 \rfloor$, jež má nyní podobu $x_1 \leq 0$. Pro přidání do ℓ -tabulky je vhodnější vyjádření (4.1), jež má tvar $-x_4 + \frac{1}{3}x_2 \leq -\frac{1}{3}$. Přidáme tuto podmínku do ℓ -tabulky a přeoptymalizujeme.

	x_4	x_2			x_6	x_2		
x_0	1	$\frac{2}{3}$	$\frac{1}{3}$	$\sim$	x_0	1	0	0
x_1	1	$-\frac{1}{3}$	$\frac{1}{3}$		x_1	1	0	0
x_2	0	-1	0		x_2	0	-1	0
x_3	$\frac{1}{3}$	$\frac{8}{9}$	$\frac{4}{9}$		x_3	$\frac{1}{3}$	1	$\frac{1}{3}$
x_4	-1	0	0		x_4	-1	$\frac{1}{3}$	$\frac{1}{3}$
x_6	$\textcircled{-1}$	$\frac{1}{3}$	$-\frac{1}{3}$		x_6	-1	0	0

Stačila tedy pouze jedna transformace a dostali jsme optimální celočíselné řešení $x^1 = (0, 0)$ v této větvi.

V druhé větvi řešíme podúlohu s přidanou podmínkou $x_k \geq \lfloor x_k^0 \rfloor + 1$, která nabyde tvaru $x_1 \geq 1$. V řeci nebážíckých proměnných má nerovnice tvar $x_4 - \frac{1}{3}x_2 \leq -\frac{2}{3}$. Přidáme tuto podmínku do ℓ -tabulky a přeoptimalizujeme.

	x_4	x_2			x_4	x_6		
x_0	1	$\frac{2}{3}$	$\frac{1}{3}$	$\sim$	x_0	3	2	-1
x_1	1	$-\frac{1}{3}$	$\frac{1}{3}$		x_1	0	-1	1
x_2	0	-1	0		x_2	-3	-3	2
x_3	$\frac{1}{3}$	$\frac{8}{9}$	$\frac{4}{9}$		x_3	3	$\frac{8}{3}$	$-\frac{4}{3}$
x_4	-1	0	0		x_4	-1	0	0
x_6	1	$\left(-\frac{1}{3}\right)$	$-\frac{2}{3}$		x_6	0	-1	0

Protože ve třetím řádku tabulky je napravo záporná hodnota, ale nalevo samé kladné, nelze vybrat žádného pivotu, a proto je množina přípustných řešení prázdná. V této větvi výpočetního stromu se tedy optimum nenalézá, a proto je optimálním řešením $x^1 = (0, 0)$. $\square$

Příklad 4.7. Řešte pomocí metody branch & bound úlohu

$$\begin{aligned} \max \quad & 13x_1 + 8x_2 \\ \text{za podm.} \quad & x_1 + 2x_2 \leq 10, \\ & 5x_1 + 2x_2 \leq 20, \\ & x_1, x_2 \in \mathbb{Z}_+. \end{aligned}$$

Řešení: Opět začneme sestavením ℓ -normální tabulky a vyřešením lineární relaxace.

	x_1	x_2			x_5	x_2			x_3	x_2		
x_0	-13	-8	0	$\sim$	x_0	13	5	$13L$	x_0	13	18	130
x_1	-1	0	0		x_1	1	1	L	x_1	1	2	10
x_2	0	-1	0		x_2	0	-1	0	x_2	0	-1	0
x_3	1	2	10		x_3	$\textcircled{-1}$	1	$10 - L$	x_3	-1	0	0
x_4	5	2	20		x_4	-5	-3	$20 - 5L$	x_4	-5	$\textcircled{-8}$	-30
x_5	$\textcircled{1}$	1	L		x_5	-1	0	0	x_5	-1	-1	$-10 + L$

V následujícím kroku získáme optimální řešení lineární relaxace $(x_1, x_2) = (\frac{5}{2}, \frac{15}{4})$. Řešení není celočíselné, rozvětvíme proto výpočet na dvě podúlohy podle některé celočíselné proměnné s aktuálně neceločíselnou hodnotou, zvolíme např. proměnnou x_1 . V podúlohách přidáváme do programu omezení $x_1 \leq \lfloor \frac{5}{2} \rfloor = 2$, resp. $x_1 \geq \lfloor \frac{5}{2} \rfloor + 1 = 3$. Do příslušné tabulky můžeme tato omezení přidat ve tvaru

$$-\sum_{j=1}^n R_{kj}x_{N_j} \leq -\{R_{k0}\}, \quad \sum_{j=1}^n R_{kj}x_{N_j} \leq \{R_{k0}\} - 1.$$

Následně tedy řešíme podúlohu s přidaným řádkem I., který reprezentuje první nerovnost, a druhou podúlohu s přidaným řádkem II. pro druhou nerovnost.

	x_3	x_4		(I)	x_3	x_6		(II)	x_7	x_4		
x_0	$\frac{7}{4}$	$\frac{18}{8}$	$\frac{125}{2}$		x_0	4	9	58	x_0	7	4	59
x_1	$-\frac{1}{4}$	$\frac{1}{4}$	$\frac{5}{2}$		x_1	0	1	2	x_1	-1	0	3
x_2	$\frac{5}{8}$	$-\frac{1}{8}$	$\frac{15}{4}$		x_2	$\frac{1}{2}$	$-\frac{1}{2}$	4	x_2	$\frac{5}{2}$	$\frac{1}{2}$	$\frac{5}{2}$
x_3	-1	0	0		x_3	-1	0	0	x_3	-4	-1	2
x_4	0	-1	0		x_4	-1	-4	2	x_4	0	-1	0
(I)	$\frac{1}{4}$	$(-\frac{1}{4})$	$-\frac{1}{2}$		x_6	0	-1	0	x_7	-1	0	0
(II)	$(-\frac{1}{4})$	$\frac{1}{4}$	$-\frac{1}{2}$						(III)	$(-\frac{5}{2})$	$-\frac{1}{2}$	$-\frac{1}{2}$
									(IV)	$\frac{5}{2}$	$\frac{1}{2}$	$-\frac{1}{2}$

Po jedné transformaci dostaneme v úloze (I) celočíselné optimální řešení $(x_1, x_2) = (2, 4)$ s hodnotou 58. V úloze (II) dostaneme neceločíselné řešení $(3, \frac{5}{2})$ s lepší hodnotou účelové funkce 59, musíme tedy tento uzel dále rozvést. Volíme proměnnou x_2 a přidáním podmínek $x_2 \leq 2$ a $x_2 \geq 3$ vytvoříme podúlohy (III) a (IV).

Úloha (III) vede na relaxaci s optimální hodnotou 57.6, která je horší než aktuální nejlepší hodnota pro celočíselné řešení, tuto větev můžeme tedy ukončit. Dále lze snadno nahlédnout, že úloha (IV) je nepřipustná, jelikož obsahuje podmínky $x_1 \geq 3$, $x_2 \geq 3$ a $5x_1 + 2x_2 \leq 20$.

Optimálním řešením zadané celočíselné úlohy je tedy řešení $(2, 4)$. □

4.1 Procházení výpočetního stromu

Binární výpočetní strom lze procházet různými způsoby. Nabízí se základní postupy jako prohledávání do šířky (seznam podúloh je fronta) a do hloubky (seznam podúloh je zásobník). Druhá varianta se zdá perspektivnější, protože rychleji najde přípustné řešení, které v důsledku pomůže ořezat neplodné větve v stromu.

Sofistikovanější možnost je v daném kroku vybrat podúlohu s maximálním horním odhadem na optimální hodnotu (tedy s největší optimální hodnotou relaxace). Jaká datová struktura se hodí pro seznam podúloh? Protože potřebujeme rychle najít maximum a odstranit ho, a umět přidávat další prvky, tak se hodí *halda*, která operace vykonává v logaritmickém čase vzhledem k velikosti seznamu. Teoreticky pak vychází nejlépe *Fibonacciho halda*, kde vkládání trvá amortizovaně konstantní čas.

4.2 Volba proměnné pro větvení

Pro efektivitu procházení stromu má velký vliv také volba celočíselné proměnné x_k , podle které vytváříme podúlohy.

Volba má vesměs podobu

$$k := \arg \max_{i \in C: x_i^0 \notin \mathbb{Z}} \min(d_i^-, d_i^+),$$

kde d_i^- , d_i^+ jsou určité míry vhodnosti dané proměnné pro větvení.

1. Nejjednodušší volbou je míra neceločíselnosti

$$d_i^- := \{x_i^0\}, \quad d_i^+ := 1 - \{x_i^0\}.$$

Volíme tedy proměnnou, která odpovídá nejméně celočíselné složce relaxovaného optima.

2. Další možností je použít pseudoderivace

$$d_i^- = d_i^+ := |c_i|.$$

Koeficienty účelové funkce udávají míru změny účelové funkce při změně proměnné, tedy větší vliv na změnu relaxované optimální hodnoty mají proměnné s větší hodnotou $|c_i|$.

3. Kombinací obou předchozích dostáváme

$$d_i^- := \{x_i^0\}|c_i|, \quad d_i^+ := (1 - \{x_i^0\})|c_i|,$$

což dává odhad, o kolik se odříznutím sníží optimální hodnota relaxace.

4. O něco dražší, ale přesnější, je pro každého kandidáta použít zaokrouhlení nahoru i dolů a podívat se jak se změní (přesně či aspoň přibližně) hodnota účelové funkce pro provedení jednoho kroku simplexové metody. Pro dané $i \in C$ takové, že $x_i^0 \notin \mathbb{Z}$, i -tý řádek simplexové tabulky říká

$$x_i = R_{i0} - \sum_{j=1}^n R_{ij}x_{N_j}.$$

Podmínka $x_i \leq \lfloor x_i^0 \rfloor$ vede na novou nerovnici

$$-\sum_{j=1}^n R_{ij}x_{N_j} \leq -\{R_{i0}\},$$

a účelová hodnota po jedné transformaci tabulky klesne aspoň o

$$d_i^- := \min_{j: R_{ij} > 0} \frac{R_{0j}}{R_{ij}} \{R_{i0}\}.$$

Analogicky, podmínka $x_i \geq \lfloor x_i^0 \rfloor + 1$ vede na novou nerovnici

$$\sum_{j=1}^n R_{ij}x_{N_j} \leq -(1 - \{R_{i0}\}),$$

a účelová hodnota po jedné transformaci tabulky klesne aspoň o

$$d_i^+ := \min_{j: R_{ij} < 0} \frac{R_{0j}}{-R_{ij}} (1 - \{R_{i0}\}).$$

4.3 Další varianty

Branch & cut

Branch & cut [Padberg and Rinaldi, 1987] je kombinací obou hlavních metod sečných nadrovin a branch & bound. V zásadě se jedná o branch & bound s tím, že v určitých uzlech výpočetního stromu osekáme relaxovaný vrchol pomocí sečných nadrovin. Většinou se řezy používají u kořene a v blízkých uzlech, abychom měli dobré relaxované řešení hned v počátku.

Nic nám nebrání přidat víc sečných nadrovin najednou. Není moc výhodné přidávat všechny myslitelné řezy, protože řada z nich bude redundantní a akorát navýší paměťovou a tím i časovou náročnost. Achterberg [2009] doporučuje vybrat několik řezů, které jsou na sebe co nejkolmější. Zvolíme-li threshold $\varepsilon > 0$, pak stačí postupovat hladovým algoritmem a přidávat takové sečné nadroviny, jejichž normály (po znormování) mají skalární součin v absolutní hodnotě nanejvýš ε se všemi již vybranými řezy.

Branch & price

Jedná se o kombinaci branch & bound a metody *column generation* (kapitola 7) pro řešení velkých úloh s obrovským počtem proměnných. Protože bázecké optimální řešení lineárního programu má nebázecké složky nulové, metoda *column generation* nejprve vybere jen proměnné do báze a ostatní zafixuje na nulu. Poté iterativně generuje sloupceky tabulky, tj. uvolňuje další proměnné podle *reduced cost*, tj. podle nejvíce porušné podmínky na nezápornost kritériálního řádku.

4.4 Implementační záležitosti

Preprocessing

Uvažme úlohu

$$\max c^T x \text{ za podm. } Ax \leq b, \ell \leq x \leq u.$$

Před vlastním řešením úlohy lze provést následující zjednodušující operace pro každou nerovnici $a^T x \leq b_i$ ze soustavy $Ax \leq b$:

- *Utáhnutí mezí.* Pro každé k

$$\begin{aligned} a_k > 0 &\Rightarrow x_k \leq \frac{1}{a_k} \left(b_i - \sum_{j \neq k: a_j > 0} a_j \ell_j - \sum_{j \neq k: a_j < 0} a_j u_j \right), \\ a_k < 0 &\Rightarrow x_k \geq \frac{1}{a_k} \left(b_i - \sum_{j \neq k: a_j > 0} a_j u_j - \sum_{j \neq k: a_j < 0} a_j \ell_j \right). \end{aligned}$$

- *Test redundance.* Daná nerovnost je redundantní pokud

$$\sum_{j: a_j > 0} a_j u_j + \sum_{j: a_j < 0} a_j \ell_j \leq b_i.$$

- *Test nepřipustnosti.* Systém je nepřipustný pokud pro nějaké i

$$\sum_{j: a_j > 0} a_j \ell_j + \sum_{j: a_j < 0} a_j u_j > b_i.$$

- *Fixace proměnných.* Pro každé k

- (1) pokud $c_k \geq 0$ a $a_{ik} \leq 0$ pro všechna i , pak můžeme zafixovat $x_k := u_k$,
- (2) pokud $c_k \leq 0$ a $a_{ik} \geq 0$ pro všechna i , pak můžeme zafixovat $x_k := \ell_k$.

Tyto předzpracující operace fungují dobře i pro lineární programování. V celočíselném programování můžeme navíc při utáhnutí mezí nové meze zaokrouhlit – horní odhad dolů a dolní odhad nahoru. Operace navíc můžeme opakovat, protože výsledek jedné operace ovlivňuje ty ostatní.

Příklad 4.8. Uvažme celočíselný program

$$\begin{aligned} \max \quad & 2x_1 + x_2 - x_3 \text{ za podm. } 5x_1 - 2x_2 + 8x_3 \leq 15, \\ & 8x_1 + 3x_2 - 1x_3 \geq 9, \\ & x_1 + x_2 + x_3 \leq 6, \\ & 0 \leq x_1 \leq 3, \\ & 0 \leq x_2 \leq 1, \\ & 1 \leq x_3, \quad x \in \mathbb{Z}^3. \end{aligned}$$

Aplikujte jednotlivé operace preprocessingu:

□

Další

Komerční systémy často obsahují

- *Priority.* Možnost zadat priority proměnným, podle kterých se pak řídí větvení ve výpočetním stromu.
- *User cut-offs.* Možnost zadat preference uživatele na *simplex strategies* (zda simplexová metoda či metoda vnitřních bodů), *strong branching* (zda věnovat větší čas dobrému výběru větve, tj. proměnné x_k), atp.

- *GUB (generalized upper bound) branching* nebo též *SOS1 branching* (Beale & Tomlin, 1970). Řada úloh obsahuje podmínky typu: $\sum_{i=1}^n x_i = 1$, $x \in \{0,1\}^n$, to jest, právě jedna z proměnných x_i je jedna, ostatní nulové. Takovéto podmínky se vyskytují v řadě případů, jako je například problém obchodního cestujícího (10.1), facility location problem (11.2), různé typy přiřazovacích a rozvrhovacích úloh nebo úlohy typu uvedené v příkladu 1.34.

Sestavovat podúlohy popsáním způsobem není efektivní (dvě větve stromu jsou značně nevyvážené), lepší je uvažovat podúlohy:

- (1) $x_i = 0$ pro všechna $i \in C_1$,
- (2) $x_i = 0$ pro všechna $i \in C_2$,

kde $C_1 \cup C_2 = \{1, \dots, n\}$ je disjunktní rozklad proměnných. Rozklad můžeme provést sekvenčně jako $C_1 = \{1, \dots, r\}$, $C_2 = \{r+1, \dots, n\}$, nebo po vhodné permutaci. Každopádně, obě podúlohy mají výrazně menší dimenzi.

Pokud bychom použili formulaci z příkladu 1.37, tak relaxovaná úloha pak má typicky řešení $1 = w_1 = \dots = w_{k-1} > w_k > w_{k+1} = \dots = w_n = 0$, tudíž větvení podle neceločíselné proměnné w_k rozdělí úlohu na dvě srovnatelně velké podúlohy, které mají automaticky zafixovaných k resp. $n - k + 1$ binárních proměnných.

Více o tom jak řešit podobné „symetrické“ podmínky viz Margot [2010].

Následující tabulka uvádí klíčové faktory na zrychlení praktických implementací během období kolem roku 2000; od té doby ale heuristiky získaly důležitější roli.

faktor	koeficient zrychlení
řezy	54
preprocessing	11
volba proměnné na dělení	3
heuristiky	1.5

4.5 Aktuální trendy

Aktuální trendy výzkumu (k roku 2015) zahrnují např.

- *Zobecněné větvení* podle obecných nadrovin. Čili místo podle nadroviny $x_i = x_i^0$ rozdělují podúlohu podle nadroviny $a^T x = b$.
- *Výběr LP solveru* na řešení jednotlivých relaxovaných podúloh výrazně ovlivňuje rychlost celého algoritmu.

4.6 Software

Open-source (nekomerční) solvery:

- *SCIP* (Solving Constraint Integer Programs), <http://scip.zib.de/>
- *GLPK* (GNU Linear Programming Kit), <http://www.gnu.org/software/glpk/>

Komerční solvery (výběr):

- *CPLEX*, <http://www.cplex.com/>
- *Gurobi*, <http://www.gurobi.com/>

Modelovací systémy zahrnující výběr z výše zmíněných solverů:

- *GAMS* (General Algebraic Modeling System), <http://www.gams.com/>
- *NEOS* (Network Enabled Optimization Server), <http://neos.mcs.anl.gov/neos/> – zdarma na dálku
- *AIMMS* (Advanced Interactive Multidimensional Modeling System), <http://www.aimms.com/>
- *AMPL* (A Mathematical Programming Language), <http://ampl.com/>

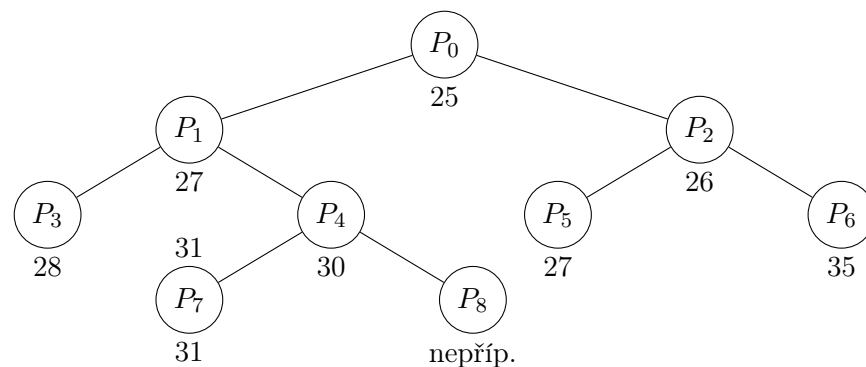
Další informace viz např. Atamtürk and Savelsbergh [2005]; Bussieck and Vigerske [2011].

Cvičení

- 4.1. Dokažte, že při použití metody branch & bound na smíšenou úlohu s jedinou celočíselnou proměnnou navštívíme při procházení větvícího stromu nejvýše 3 uzly.
- 4.2. Upravte metodu branch & bound tak, aby našla „dostatečně dobré“ přípustné řešení s hodnotou účelové funkce v toleranci $p\%$ od optimální hodnoty (pro dané p).
- 4.3. Aplikujte techniku „utáhnutí mezí“ k výpočtu co nejtěsnějších mezí pro celočíselné proměnné $x_1, \dots, x_6$ za podmínek

$$\begin{aligned} 2x_1 + 7x_2 - 3x_3 + 6x_4 - 9x_5 + x_6 &\leq -12, \\ x_1 - 2x_2 + x_3 + 4x_4 + 2x_5 - 3x_6 &\leq 13, \\ x_1 \in [1, 4], x_2 \in [0, 7], x_3 \in [4, 10], x_4 \geq 2, x_5 \in [0, 2], x_6 \geq 0, \\ x_1, \dots, x_6 &\in \mathbb{Z}. \end{aligned}$$

- 4.4. Uvažujme minimalizační úlohu celočíselného programování. Následující výpočetní strom reprezentuje průběh metody branch & bound (číslo pod uzlem označuje optimální hodnotu příslušné lineární relaxace, číslo nad uzlem optimální hodnotu s celočíselným řešením):



Najděte co nejtěsnější meze pro optimální hodnotu úlohy. Které uzly stromu můžeme odříznout, a které musíme dále rozvětvit a prohledat?

Kapitola 5

Heuristiky

Heuristiky jsou jednoduché metody, které mají za cíl rychle najít dobré přípustné řešení, což zvyšuje rychlost metod typu branch & bound. Pozitivní výsledek nemáme garantovaný, takže heuristiky mohou selhat. Význam dobrých heuristik ale je, že v mnoha případech fungují a efektivně pomáhají při řešení tím, že dávají globální dolní mez na optimální hodnotu (viz poznámka 4.2). Pěkný přehled heuristik najdeme v Berthold [2006], a heuristiky používané programem SCIP pak v Berthold [2007].

5.1 Heuristiky pro nalezení přípustného řešení

Relax & fix

Uvažujme úlohu ve tvaru

$$\max c^T x + d^T y \text{ za podm. } Ax + By \leq b, x, y \geq 0, x \in \mathbb{Z}^n, y \in \mathbb{Z}^m.$$

Zde, rozdělení proměnných do dvou skupin vychází z podstaty problému, neboť x jsou „důležitější“ proměnné, u kterých požadujeme, aby byly blíže k optimu.

Heuristika sestává ze dvou kroků. V prvním řešíme relaxovanou úlohu s $x \in \mathbb{Z}^n, y \in \mathbb{R}^m$ (tedy relaxují celočíselnost proměnných y). Nechť získáme optimální řešení x^*, y^* . V druhém kroku zafixujeme $x := x^*$ a dořešíme původní úlohu. Pokud dostaneme přípustné řešení, je dobrým přípustným řešením původní úlohy. V této heuristice řešíme tedy 2 menší celočíselné úlohy.

Cut & fix

Uvažujme úlohu ve tvaru

$$\max c^T x + d^T y \text{ za podm. } Ax + By \leq b, x, y \geq 0, x \in \mathbb{Z}^n, y \in \mathbb{Z}^m.$$

Tato heuristika nejprve k omezením přidá několik sečných nadrovin a spočítáme relaxované řešení $x^* \in \mathbb{R}^n, y^* \in \mathbb{R}^m$. Nyní, pro předem zvolenou malou konstantu $\varepsilon > 0$, přidáme podmínky

$$\lfloor x^* + \varepsilon e \rfloor \leq x \leq \lceil x^* - \varepsilon e \rceil$$

a dořešíme do celočíselného optima. Heuristické řešení získám rychleji, protože přidané podmínky sníží trochu dimenzi úlohy. Pokud totiž x_i^* je dostatečně blízko celému číslu (nanejvýš o ε), pak

$$\lfloor x_i^* + \varepsilon \rfloor = \lceil x_i^* - \varepsilon \rceil,$$

což zafixuje i -tou proměnnou x_i .

Feasibility pump

Základní idea: Najdi optimální řešení relaxované úlohy, zaokrouhli a najdi nejbližší přípustný bod relaxované úlohy v součtové normě. Postup iteruj. Detaily viz Achterberg and Berthold [2007].

5.2 Heuristiky pro nalezení dobrého řešení

Nyní se zaměříme na úlohy, kde je snadné najít přípustná řešení, ale je velmi těžké najít optimum. Následující heuristiky se proto snaží najít co nejlepší přípustné řešení (bez garance optimality) a jsou adaptací z heuristik pro úlohy spojitě optimalizace s velkou dimenzí.

Lokální prohledávání

Tato heuristika funguje tak, že dané přípustné řešení se snaží lokálně vylepšit. Prohledá sousední řešení a pokud najde lepší, změní aktuální přípustné řešení na toto nalezené a postup opakuje.

Konkrétní postup se dost odvíjí od konkrétní úlohy. V problému obchodního cestujícího může jít o tzv. dvoj-výměny, kdy danou cestu na dvou místech přestřihneme a spojíme opačné konce (viz sekce 10.4). Heuristiky pro úlohu Facility location problem jsou v sekci 11.3.

Následující heuristiky se snaží vyvarovat toho, abychom uvízli v lokálním maximu.

Simulované žíhání

Metoda, kdy procházíme mezi přípustnými řešeními. Na začátku je symbolická teplota T vysoká a dovoluje nám přejít i k horšímu řešení. Postupem času teplota klesá a nedovolí nám příliš si pohoršit účelovou hodnotu. Až nakonec smíme přecházet jen k lepším řešením.

Formálněji, je-li Δ rozdíl účelových hodnot aktuálního řešení a náhodně vybraného sousedního, tak k sousednímu přejdu vždy pokud $\Delta \geq 0$ (tj., sousední řešení není horší) a s pravděpodobností $e^{-\Delta/T}$, kde T je čas, pokud $\Delta < 0$ (tj., sousední řešení je horší).

Tabu search

Opět procházíme mezi přípustnými řešeními. Pokud se nemohu přemístit k lepšímu řešení, mohu přejít i k horšímu. K zabránění vrácení se zpět k již nalezeným lokálně optimálním řešením slouží tzv. *tabu list*, což je seznam několik předchozích lokálně optimálních řešení, které již nesmím navštívit.

Genetické algoritmy

Jejich princip spočívá v tom, že máme seznam několika přípustných řešení, nazývaný *výběr z populace*. Každý jedinec je nějak kódován. Tyto jedince pak měním podle pravidel

- *mutace* – změním trochu kód jedince,
- *křížení* – zkombinuji kódy dvou jedinců,
- *přírodní výběr* – vybírám co nejlepší jedince.

Implementace těchto pravidel závisí na konkrétním typu úlohy.

Například pro problém obchodního cestujícího, viz sekce 10.4.

Cvičení

5.1. Pomocí heuristiky relax & fix aproximujte optimální řešení úlohy

$$\begin{array}{ll} \max & 13x + 8y \\ \text{za podm.} & x + 2y \leq 10, \\ & 5x + 2y \leq 20, \\ & x, y \in \mathbb{Z}_+. \end{array}$$

Kapitola 6

Dekompozice

Uvedeme si dva typy dekompozic. První, Bendersova, rozděluje proměnné do dvou částí a určitým způsobem redukuje problém na několik menších. Druhá, Lagrangeova, naopak rozděluje do dvou skupin podmínky v omezeních.

6.1 Bendersova dekompozice

Bendersova dekompozice [Benders, 1962, 2005] funguje v zásadě i pro lineární či speciální nelineární program, ale pro naše účely uvažujeme smíšenou úlohu celočíselného programování

$$\max c^T x + d^T y \text{ za podm. } Ax + By \leq b, x \geq 0, y \in Y, \quad (6.1)$$

kde množina Y zahrnuje lineární a celočíselné podmínky jako např. $y \geq 0, y \in \mathbb{Z}^n$ atp. Budeme předpokládat $Y \neq \emptyset$. Potom můžeme úlohu přepsat jako

$$\max_{y \in Y} \{d^T y + \max\{c^T x; Ax \leq b - By, x \geq 0\}\},$$

nebo pomocí duálního lineárního programu jako

$$\max_{y \in Y} \{d^T y + \min\{(b - By)^T u; A^T u \geq c, u \geq 0\}\}.$$

Speciálně, primární a duální lineární program označíme jako

$$\begin{aligned} \max\{c^T x; Ax \leq b - By, x \geq 0\}, & \quad P(y) \\ \min\{(b - By)^T u; A^T u \geq c, u \geq 0\}. & \quad D(y) \end{aligned}$$

Abychom mohli použít dualitu, budeme ještě předpokládat, že množina přípustných řešení duální úlohy

$$U := \{u; A^T u \geq c, u \geq 0\} \neq \emptyset.$$

Neprázdnost U snadno ověříme zvlášť, a pokud by neplatila, tak původní úloha je neomezená či prázdná. Pro praktické úlohy je to vesměs splněno, jinak lze teoreticky použít nástroje popsané např. v Poznámce 1.26.

Buďte $v_1, \dots, v_V$ vrcholy a $h_1, \dots, h_H$ směry neomezených hran konvexního polyedru U . Pro pevné y , pokud v úloze $D(y)$ nastane $(b - By)^T h_j < 0$ pro jisté j , pak $D(y)$ je neomezená a tudíž $P(y)$ nepřipustná. Proto se stačí omezit na situace kdy $(b - By)^T h_j \geq 0$ pro všechna $j = 1, \dots, H$. Nyní můžeme úlohu (6.1) ekvivalentně vyjádřit ve tvaru

$$\max (d^T y + \min_{i=1, \dots, V} (b - By)^T v_i) \text{ za podm. } (b - By)^T h_j \geq 0 \forall j, y \in Y,$$

neboli

$$\max z \text{ za podm. } z \leq d^T y + (b - By)^T v_i \forall i, (b - By)^T h_j \geq 0 \forall j, y \in Y. \quad (6.2)$$

Tímto jsme zmenšili dimenzi původní úlohy, protože namísto spojitých proměnných x zde máme již jen jednu spojitou proměnnou z . Nicméně, celočíselné proměnné zůstávají stejné.

Abychom nemuseli enumerovat všechny vrcholy a neomezené hrany U , tak je budeme generovat postupně dle potřeby podle následujícího iterativního algoritmu. Pro indexové množiny I, J si označme

$$\max z \quad \text{za podm. } z \leq d^T y + (b - By)^T v_i \quad \forall i \in I, \quad (b - By)^T h_j \geq 0 \quad \forall j \in J, \quad y \in Y. \quad P_{I,J}$$

Algoritmus 6.1 (Bendersova dekompozice). Na začátku polož $I := \emptyset, J := \emptyset$. V k -té iteraci postupujeme

1: Vyřeš úlohu $P_{I,J}$. Má-li úloha optimum (y^*, z^*) , jdi na krok 2.

Je-li úloha neomezená, tak (y^*, z^*) zvol jako vrchol z něhož vychází neomezená hrana a jdi na krok 2.

Je-li nepřipustná, je nepřipustná i (6.1). Konec.

2: Vyřeš úlohu $D(y)$ s $y := y^*$.

Nepřipustnost nenastane díky $U \neq \emptyset$.

Je-li neomezená dle hrany h_{j^*} vycházející z vrcholu v_{i^*} , tak přiřaď $I := I \cup \{i^*\}, J := J \cup \{j^*\}$ a jdi na krok 1.

Má-li optimální řešení ve vrcholu v_{i^*} , tak rozlišme dva případy:

(a) Je-li $z^* \leq d^T y^* + (b - By^*)^T v_{i^*}$, pak platí $z^* \leq d^T y^* + (b - By^*)^T v_i$ pro všechna $i = 1, \dots, V$ a proto je (y^*, z^*) optimum úlohy (6.2). Tudíž je z^* optimální hodnota i původní úlohy (6.1), y^* je optimální řešení a druhou část optima x^* dopočítáme z podmínek duality.

(b) Jinak, přiřaď $I := I \cup \{i^*\}$, a jdi na krok 1.

Konečnost algoritmu je zaručena, počet iterací je maximálně $V + H$.

Navíc, algoritmus lze kdykoli ukončit, a máme dolní a horní odhady na optimální hodnoty. Jako horní odhad optimální hodnoty slouží z^* (postupem iterací se zmenšuje). Naopak, pokud úloha $D(y)$ má optimum v kroku 2 Algoritmu 6.1, tak $z^\circ := d^T y^* + (b - By^*)^T v_{i^*}$ je dolní mezí optimální hodnoty (neboť (y^*, z°) je přípustné řešení).

Bendersova dekompozice se nevyplatí pro každou úlohu, ale pro některé typy úloh je vhodná. Ukážeme si její použití v úloze UFLP (sekce 11.4).

Poznámka 6.2. Bendersovou dekompozicí dostaneme úlohu se stejným počtem celočíselných proměnných, ale s pouze jednou spojitou proměnnou z . Protože čistě celočíselné úlohy se řeší snadněji, může být výhodné proměnnou z uvažovat jako celočíselnou a pak binárním půlením aproximovat spojitou hodnotu.

Příklad 6.3. Bendersovou dekompozicí vyřešte úlohu (viz obrázek 6.1)

$$\begin{aligned} \max \quad & -5x + 3y \quad \text{za podm.} \\ & -2x + y \leq 0, \\ & -x + 3y \leq 13, \\ & y \leq 10, \\ & x, y \geq 0, \quad y \in \mathbb{Z}. \end{aligned}$$

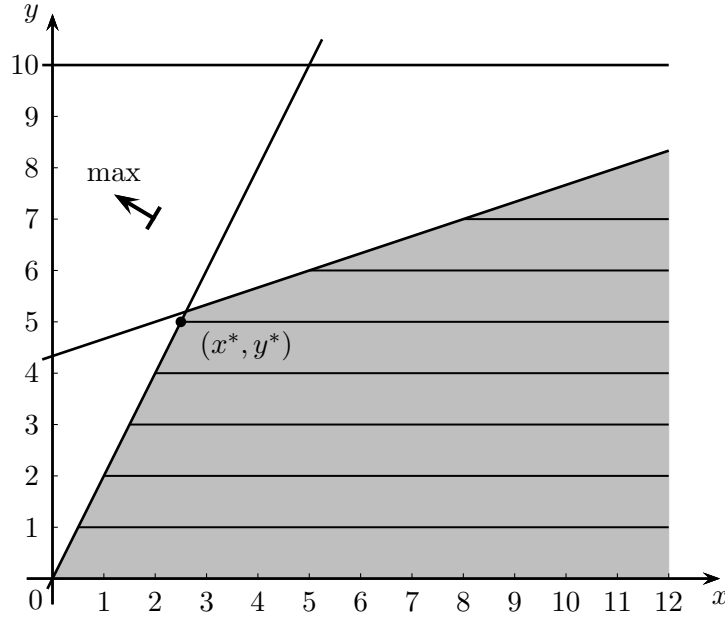
Řešení: V této úloze je

$$c = (-5), \quad d = (3), \quad A = \begin{pmatrix} -2 \\ -1 \end{pmatrix}, \quad B = \begin{pmatrix} 1 \\ 3 \end{pmatrix}, \quad b = \begin{pmatrix} 0 \\ 13 \end{pmatrix},$$

a množina U reprezentuje trojúhelník s vrcholy $(0, 0)^T$, $(0, 5)^T$ a $(2.5, 0)^T$:

$$U = \{u \in \mathbb{R}^2; -2u_1 - u_2 \geq -5, \quad u \geq 0\}.$$

Na začátku nastavíme $I := \emptyset, J := \emptyset$. Jednotlivé iterace jsou:



Obrázek 6.1: Množina přípustných řešení, účelová funkce a optimální řešení (x^*, y^*) příkladu 6.3.

1. Nejprve řešíme úlohu

$$\max z \text{ za podm. } y \in Y = \{y \leq 10, y \geq 0, y \in \mathbb{Z}\}.$$

Úloha je neomezená, volíme $y^* = 0^T$, $z^* = \infty$. Nyní řešíme úlohu

$$\min_{u \in U} (b - By)^T u = \min_{u \in U} 0u_1 + 13u_2.$$

Úloha má optimum v bodě $v_1 = (0, 0)^T$. Aktualizujeme $I = \{1\}$.

2. Řešíme úlohu

$$\max z \text{ za podm. } y \in Y, z \leq d^T y + (b - By)^T v_1,$$

čili

$$\max z \text{ za podm. } y \in Y, z \leq 3y.$$

Úloha má optimum $y^* = 10$, $z^* = 30$. Nyní řešíme úlohu

$$\min_{u \in U} -10u_1 - 17u_2.$$

Úloha má optimum ve vrcholu $v_2 = (0, 5)^T$. Aktualizujeme $I = \{1, 2\}$.

3. Řešíme úlohu

$$\max z \text{ za podm. } y \in Y, z \leq 3y, z \leq 65 - 12y.$$

Úloha má optimum $y^* = 4$, $z^* = 12$. Úloha

$$\min_{u \in U} -4u_1 + u_2$$

má optimum ve vrcholu $v_3 = (2.5, 0)^T$. Aktualizujeme $I = \{1, 2, 3\}$.

4. Řešíme úlohu

$$\max z \text{ za podm. } y \in Y, z \leq 3y, z \leq 65 - 12y, z \leq 0.5y.$$

Úloha má optimum $y^* = 5$, $z^* = 2.5$. Úloha

$$\min_{u \in U} -5u_1 - 2u_2$$

má optimum ve vrcholu $v_3 = (2.5, 0)^T$ a platí

$$2.5 = z^* \leq d^T y^* + (b - By^*)^T v_3 = 2.5.$$

Tudíž máme optimum původní úlohy: Optimální hodnota je 2.5, optimální řešení je $y^* = 5$ a $x^* = 2.5$ (to zjistíme vyřešením lineárního programu po dosazení y^* do původní úlohy). $\square$

Cvičení

6.1. Bendersovou dekompozicí vyřešte úlohu

$$\begin{aligned} \max \quad & -2x_1 - 2x_2 - 3y_1 - 4y_2; \\ & x_1 - x_2 - 3y_1 + 1y_2 \leq -6, \\ & -x_1 + 3x_2 - 2y_1 - 2y_2 \leq -5, \\ & y_1 \leq 2, \\ & x, y \geq 0, \quad y_1, y_2 \in \mathbb{Z}. \end{aligned}$$

6.2 Lagrangeova relaxace a dekompozice

Lagrangeova relaxace

Cílem Lagrangeovy relaxace (z roku 1963) je získat lepší relaxaci a tím i horní odhad na optimální hodnotu než obyčejnou celočíselnou relaxací.

Uvažujme celočíselný program ve tvaru

$$\max c^T x \text{ za podm. } Ax \leq b, Dx \leq d, x \geq 0, x \in \mathbb{Z}^n, \quad (P)$$

kde $A \in \mathbb{R}^{m \times n}$, $D \in \mathbb{R}^{\ell \times n}$, $b \in \mathbb{R}^m$, $d \in \mathbb{R}^\ell$. Připomeňme, že standardní celočíselná relaxace má tvar

$$\max c^T x \text{ za podm. } Ax \leq b, Dx \leq d, x \geq 0, x \in \mathbb{R}^n. \quad (CR)$$

Pro účely Lagrangeovy relaxace předpokládejme, že podmínky $Ax \leq b$ jsou „nehezké“, a ty ostatní jsou „hezké“ v tom smyslu, že na množině

$$Q := \{x \in \mathbb{Z}^n; Dx \leq d, x \geq 0\}$$

se snadno optimalizuje (například, že matice D je totálně unimodulární). Rozdělení podmínek pak často vychází z podstaty daného problému.

Definujme úlohu Lagrangeovsky relaxující podmínku $Ax \leq b$ jako

$$\max c^T x + u^T(b - Ax) \text{ za podm. } x \in Q. \quad (P_u)$$

Tvrzení 6.4. Platí $(P) \leq (P_u)$ pro každé $u \geq 0$.

Důkaz. Buď x přípustné řešení úlohy (P) a buď $u \geq 0$. Pak x je přípustným řešením úlohy (P_u) a navíc $c^T x \leq c^T x + u^T(b - Ax)$. $\square$

Tedy úloha (P_u) je relaxací úlohy (P) . Podobně relaxujeme i úlohy v jiném tvaru, např. pro rovnice není nutná nezápornost u .

Protože (P_u) relaxuje (P) pro všechna $u \geq 0$, je přirozené vzít to nejlepší možné $u \geq 0$, což nás vede na *Lagrangeovu relaxaci*

$$\min (P_u) \text{ za podm. } u \geq 0. \quad (LR)$$

Je to vlastně Lagrangeova duální úloha z teorie nelineární optimalizace.

Jaká je síla Lagrangeovy relaxace? Vždy je aspoň tak dobrá jako celočíselná relaxace:

Věta 6.5. Platí $(LR) = \max\{c^T x; Ax \leq b, x \in \text{conv}(Q)\}$.

Důkaz. Bez újmy na obecnost (srov. poznámku 1.26) nechť Q je omezený a skládá se z bodů $x_1, \dots, x_q$. Pak Lagrangeova relaxace (P_u) má tvar

$$\begin{aligned} \min_{u \geq 0} (P_u) &= \min_{u \geq 0} \max_{i=1, \dots, q} c^T x_i + u^T(b - Ax_i) \\ &= \min z \text{ za podm. } z \geq c^T x_i + u^T(b - Ax_i) \quad \forall i, u \geq 0, z \in \mathbb{R}. \end{aligned}$$

Toto je přípustná úloha lineárního programování, tudíž ji můžeme nahradit duální úlohou

$$\max \sum_{i=1}^q (c^T x_i) y_i \text{ za podm. } \sum_{i=1}^q (b - Ax_i) y_i \geq 0, \quad e^T y = 1, \quad y \geq 0.$$

Výraz $\sum_{i=1}^q y_i x_i$ vyjadřuje konvexní kombinaci bodů z Q . Takže substitucí $x := \sum_{i=1}^q y_i x_i$ dostaneme úlohu

$$\max c^T x \text{ za podm. } Ax \leq b, \quad x \in \text{conv}(Q). \quad \square$$

Důsledek 6.6. Platí $(P) \leq (LR) \leq (CR)$.

V řetízku nerovností mohou nastat všechny možné kombinace kdy se nějaká nerovnost nabyde jako rovnost či jako ostrá nerovnost. Ukážeme postačující podmínky pro tyto situace.

Důsledek 6.7. Je-li $\text{conv}(Q) = \{x \in \mathbb{R}^n; Dx \leq d, x \geq 0\}$, pak $(LR) = (CR)$.

Lagrangeova relaxace je tudíž stejná jako klasická celočíselná relaxace pokud je polyedr $\{x \in \mathbb{R}^n; Dx \leq d, x \geq 0\}$ celočíselný. To nastane například, pokud matice D je totálně unimodulární. Tudíž aby Lagrangeova relaxace byla silnější než celočíselná relaxace, nesmí se na množině Q optimalizovat až příliš snadno.

Naopak, následující postačující podmínka nám zaručuje $(LR) = (P)$.

Tvrzení 6.8. Bud' $u \geq 0$, bud' x^* optimální řešení (P_u) takové, že $Ax^* \leq b$ a navíc $(Ax^*)_i = b_i$ pro všechna $i : u_i > 0$. Pak x^* je optimálním řešením (P) .

Důkaz. Podle předpokladu je x^* přípustné řešení úlohy (P) . Podle tvrzení 6.4 je $(P) \leq c^T x^* + u^T (b - Ax^*) = c^T x^*$. Tudíž x^* je optimálním řešením (P) . $\square$

Subgradientní algoritmus

Jak řešit úlohu (LR) ? Ukážeme si efektivnější metodu než prostou enumeraci z důkazu věty 6.5.

Z důkazu věty 6.5 můžeme úlohu (LR) vyjádřit jako $\min_{u \geq 0} f(u)$, kde

$$f(u) := \max_{i=1, \dots, q} (b - Ax_i)^T u + c^T x_i, \quad (6.3)$$

a $x_1, \dots, x_q$ jsou prvky množiny Q . Tato funkce je konvexní a po částech lineární. Tudíž nemůžeme použít metody konvexní (hladké) optimalizace a musíme na to jít jinak. Konkrétně, využijeme subgradientů.

Definice 6.9. Subgradientem konvexní funkce $f: \mathbb{R}^n \rightarrow \mathbb{R}$ v bodě u^* je libovolný vektor $d \in \mathbb{R}^n$ takový, že $f(u) \geq f(u^*) + d^T(u - u^*)$ pro všechna $u \in \mathbb{R}^n$.

Geometricky, $f(u)$ musí ležet nad nadrovinou $f(u^*) + d^T(u - u^*)$, která se f dotýká v bodě u^* . Zřejmě, je-li f diferencovatelná, tak subgradientem je (a musí být) gradient $d = \nabla f(u^*) = \left(\frac{\partial f}{\partial u_1}(u^*), \dots, \frac{\partial f}{\partial u_n}(u^*) \right)^T$.

Subgradient naší funkce (6.3) spočítáme podle následujícího pozorování.

Tvrzení 6.10. Bud' x_{u^*} optimální řešení (P_u) s $u := u^*$, to jest

$$f(u^*) := (b - Ax_{u^*})^T u^* + c^T x_{u^*}.$$

Pak funkce $f(u)$ má v bodě u^* subgradient $d := b - Ax_{u^*}$.

Důkaz. Bud' $u \geq 0$. Máme ukázat $f(u) \geq f(u^*) + d^T(u - u^*)$, neboli

$$\max_{i=1, \dots, q} (b - Ax_i)^T u + c^T x_i \geq (b - Ax_{u^*})^T u^* + c^T x_{u^*} + (b - Ax_{u^*})^T (u - u^*).$$

To se zjednodušením pravé strany upraví na

$$\max_{i=1, \dots, q} (b - Ax_i)^T u + c^T x_i \geq (b - Ax_{u^*})^T u + c^T x_{u^*},$$

což zjevně platí. $\square$

Základní myšlenka subgradientního algoritmu, podobně jako u gradientních metod, je iterativně se posouvat v množině přípustných řešení ve směru klesání $-d$.

Algoritmus 6.11 (Subgradientní algoritmus). Zvol počáteční u^1 a nastav $k = 1$. V k -té iteraci provedeme:

- 1: Řeš úlohu (P_u) s $u := u^k$, a necht' má optimum x_{u^k} .
- 2: $u^{k+1} := \max(u^k - \alpha_k(b - Ax_{u^k}), 0)$,
- 3: $k := k + 1$.

Druhý krok zaručuje $u^{k+1} \geq 0$.

Parametr $\alpha_k > 0$ určuje délku kroku ve směru klesajícího subgradientu. Možné volby:

- Volíme α_k tak, aby $\alpha_k \rightarrow_{k \rightarrow \infty} 0$ a $\sum_{k=1}^{\ell} \alpha_k \rightarrow_{\ell \rightarrow \infty} \infty$. Toto zaručuje konvergenci. Typickou volbou je $\alpha_k = 1/k$, prakticky je to ale pomalé.
- Hodnota α_k je po dobu p iterací konstantní, a pak se po každých p iteracích násobí parametrem $\rho \in (0, 1)$. Konvergence je zaručena pro α_1 a ρ dostatečně velké.
- Další volby viz Wolsey [1998].

Ukončovací podmínkou může být např. $d = 0$, překročení limitu počtu iterací, nebo pro celočíselná data podmínka $f(u^k) - z < 1$, kde z je účelová hodnota nejlepšího dosud nalezeného přípustného řešení (tedy je to optimum). Poznamenejme, že algoritmus můžeme kdykoliv ukončit a podle věty 6.4 dává (P_u) horní odhad na optimální hodnotu; tedy v principu není nutné řešit (LR) do optima za každou cenu.

Lagrangeova dekompozice

Lagrangeova dekompozice ve své podstatě je speciálním případem Lagrangeovy relaxace, kdy relaxací určité podmínky se úloha rozpadne na dvě (ne zcela nezávislé) podúlohy. Přepíšme (P) do tvaru

$$\max c^T x \text{ za podm. } Ax \leq b, Dy \leq d, x = y, x, y \in \mathbb{Z}_+^n.$$

Lagrangeovou relaxací podmínky $x = y$ dostaneme

$$\begin{aligned} & \min_{u \in \mathbb{R}^n} \max\{(c - u)^T x + u^T y; Ax \leq b, Dy \leq d, x, y \in \mathbb{Z}_+^n\} \\ &= \min_{v, w: c = v + w} (\max\{v^T x; Ax \leq b, x \in \mathbb{Z}_+^n\} + \max\{w^T y; Dy \leq d, y \in \mathbb{Z}_+^n\}). \end{aligned} \quad (LR')$$

Takto dostaneme ještě lepší horní mez než přímou Lagrangeovou relaxací.

Tvrzení 6.12. Platí $(LR') = \max\{c^T x; x \in \text{conv}\{x \in \mathbb{Z}_+^n; Ax \leq b\} \cap \text{conv}\{x \in \mathbb{Z}_+^n; Dx \leq d\}\}$.

Lagrangeova relaxace pro speciální úlohy

Lagrangeova relaxace je výhodná pro řadu speciálních úloh. V Sekci 10.3 ukážeme použití pro problém obchodního cestujícího a v Sekci 11.5 pro úlohu UFLP.

Cvičení

- 6.2. Diskutujte obrácenou implikaci v Důsledku 6.7. To jest, pokud $(LR) = (CR)$, platí $\text{conv}(Q) = \{x \in \mathbb{R}^n; Dx \leq d, x \geq 0\}$?
- 6.3. Aplikujte subgradientní algoritmus pro Lagrangeovu relaxaci na následující celočíselný program:

$$\begin{aligned} & \max && 16x_1 + 10x_2 && + 4x_4 \\ & \text{za podm.} && 8x_1 + 2x_2 + x_3 + 4x_4 \leq 10, \\ & && x_1 + x_2 \leq 1, \\ & && x_3 + x_4 \leq 1, \\ & && x_1, x_2, x_3, x_4 \in \{0, 1\}. \end{aligned}$$

6.4. S využitím Lagrangeovy relaxace řešte následující celočíselný program:

$$\begin{array}{ll}\max & 9x_1 + 4x_2 + 15x_3 \\ \text{za podm.} & 3x_1 + 2x_2 + 4x_3 \leq 5, \\ & x_1, x_2, x_3 \in \{0, 1\}.\end{array}$$

Určete optimální hodnotu relaxace (P_u) pro všechny možné hodnoty $u \geq 0$ a najděte nejlepší možný horní odhad na optimální hodnotu programu, který z těchto relaxací můžeme získat (tj. optimum pro Lagrangeovu relaxaci).

Kapitola 7

Column generation

Tato technika byla vyvinuta pro celočíselné a spojité lineární programy s velkým počtem proměnných [Cochran et al., 2011; Wolsey, 1998].¹⁾ Uvažujme nejprve lineární program

$$\min c^T x \text{ za podm. } Ax = b, x \geq 0, \quad (7.1)$$

kde počet proměnných n je velké. Buď $I \subset \{1, \dots, n\}$. Z matice A vybereme sloupce indexované množinou I , a označíme ji A_I . Jinými slovy, zafixujeme proměnné $x_i = 0, i \notin I$. Dostaneme menší podúlohu

$$\min c_I^T x_I \text{ za podm. } A_I x_I = b, x_I \geq 0. \quad (7.2)$$

Buď x_I^* optimum této podúlohy. Z podmínek duality snadno získáme i optimum y^* duální úlohy

$$\max b^T y \text{ za podm. } A_I^T y \leq c_I.$$

Doplněním nul vektoru x_I^* dostaneme přípusté řešení x^* původní úlohy (7.1). Potom x^* je optimem, pokud y^* splňuje všechny podmínky duální úlohy k (7.1)

$$\max b^T y \text{ za podm. } A^T y \leq c.$$

Pokud y^* nesplňuje všechny podmínky, tak vyber tu nejvíce porušenou a přidej její index do seznamu I a přeoptymalizuj podúlohu. Pokud řešíme podúlohu (7.2) primární simplexovou metodou, tak úlohu s přidaným sloupцем snadno přeoptymalizujeme rozšířením simplexové tabulky o jeden sloupec a pivotizací v tomto sloupci.

Pokud v úloze (7.1) máme navíc podmínky na celočíselnost $x \in \mathbb{Z}^n$, tak metodu můžeme zkombinovat s branch & bound.

Příklad 7.1. Uvažujme úlohu ve tvaru

$$\min c_1^T x_1 + c_2^T x_2 \text{ za podm. } A_1 x_1 + A_2 x_2 = b, x_1, x_2 \geq 0, x_i \in X_i, i \in \{1, 2\},$$

kde $X_i = \{x_i \in \mathbb{Z}^{n_i}; C_i x_i \leq d_i\}, i \in \{1, 2\}$. Buďte $v_1^i, \dots, v_{m_i}^i$ všechna přípustná řešení X_i . Pak použijeme substituci

$$x_i = \sum_{k=1}^{m_i} \lambda_k^i v_k^i, \lambda_k^i \geq 0, \sum_{k=1}^{m_i} \lambda_k^i = 1.$$

Tím dostaneme lineární program s potenciálně velmi mnoha proměnnými v_k^i .

Jaká je síla takové transformace? Přepisem vlastně řešíme úlohu

$$\min c_1^T x_1 + c_2^T x_2 \text{ za podm. } A_1 x_1 + A_2 x_2 = b, x_1, x_2 \geq 0, x_i \in \text{conv}(X_i).$$

Tedy určitě je relaxace aspoň tak silná jako standardní celočíselná relaxace, na druhou stranu nemáme garanci, že řešení bude celočíselné. $\square$

¹⁾Poprvé aplikoval Gilmore & Gomory (1961), ale název patrně zavedl Appelgren (1969).

Příklad 7.2 (Boland and Surendonk, 2001). Uvažujme úlohu plánování rozvozu zboží zákazníkům pro období 12 následujících měsíců. Standardní model by byl

$$\min \sum_{i,s,t} c_{i,s} x_{i,s,t},$$

kde $c_{i,s}$ je cena za přepravu zboží zákazníkovi i lodí s . Proměnná $x_{i,s,t} \in \{0, 1\}$ říká, jestli loď s v čase t obslouží zákazníka i . Podmínky jsou:

$$\sum_s z_s x_{i,s,t} \geq d_{i,t}, \quad \forall i, t,$$

kde z_s je kapacita lodě s , a $d_{i,t}$ je požadavek zákazníka i v měsíci t . Dále,

$$\ell_s \leq \sum_{i,t} x_{i,s,t} \leq u_s, \quad \forall s,$$

kde ℓ_s a u_s je minimální a maximální počet použití lodě s za celou dobu rozvrhu. Loď s může být využita maximálně jednou během sekvence m_s měsíců, což vede na podmínky

$$\sum_i (x_{i,s,t} + x_{i,s,t+1} + \dots + x_{i,s,t+m_s-1}) \leq 1, \quad \forall t, s.$$

Konkrétní problém s 16 zákazníky a 30 loděmi nebyl v Cplexu vůbec dořešen do optima a nejlepší nalezené řešení bylo 16% od optima. Celočíslná relaxace je totiž velmi slabá.

Zkusme problém přeformulovat. Zavedeme proměnné $y_{i,k,t} \in \{0, 1\}$, které říkají zda požadavek zákazníka i v měsíci t bude uspokojen kombinací lodí k . Úloha má nyní tvar

$$\min \sum_{i,k,t} \left(\sum_{s \in k} c_{i,s} \right) y_{i,k,t},$$

za podmínek

$$\ell_s \leq \sum_{i, k \ni s, t} y_{i,k,t} \leq u_s, \quad \forall s,$$

$$\sum_{i, k \ni s} (y_{i,k,t} + y_{i,k,t+1} + \dots + y_{i,k,t+m_s-1}) \leq 1, \quad \forall t, s.$$

Počet proměnných je nyní velmi vysoký, a proto je vhodné použít metodu Column generation. Vybírání a přidávání sloupce zde odpovídá (pro daného zákazníka i a měsíc t) nalezení kombinace lodí k , která nejvíce zlepší účelovou hodnotu. Pro výše zmíněný konkrétní problém byla úloha vyřešena za 2 minuty s tím, že bylo vygenerováno 1700 sloupců (tj., kombinací lodí). Navíc řešení bylo rovnou celočíselné. $\square$

Cvičení

7.1. Aplikujte metodu column generation pro řešení řezného problému:

- firma Karlovy trubky, s.r.o, (viz cvičení 1.26) skladuje kovové trubky délky 218 cm,
- zákazník zadal objednávku na 44 ks trubek délky 81 cm, 3 ks trubek délky 70 cm a 48 ks trubek délky 68 cm.

Část II

Speciální případy

Kapitola 8

Problém batohu

Uvažujme problém batohu ve tvaru

$$\max c^T x \text{ za podm. } a^T x \leq b, x \in \{0, 1\}^n, \quad (8.1)$$

kde $a, c \in \mathbb{Z}^n$ a $b \in \mathbb{Z}$. Interpretace může být taková, že chceme maximálně naplnit pomyslný batoh abychom nepřekročili jeho nosnost, přičemž každý předmět má svou hmotnost a užitek.

Předpoklad. Budeme předpokládat, že $a > 0$, $c > 0$. Tento předpoklad je bez újmy na obecnost.

Důkaz. „Předpoklad $a > 0$.“ Je-li $a_i = 0$, pak zafixujeme $x_i := (\text{sgn}(c_i) - 1)/2$. Je-li $a_i < 0$, pak zavedeme substituci $x'_i := 1 - x_i \in \{0, 1\}$. Koeficienty se pak změni takto: $a'_i := -a_i$, pravá strana nerovnosti $b' := b - a_i$ a koeficient v účelové funkci na $c'_i := -c_i$. Celková cena se zvětší o c_i .

„Předpoklad $c > 0$.“ Je-li $c_i \leq 0$, pak zafixujeme $x_i := 0$. □

Předpoklad 2. Budeme předpokládat, že $\max_{i=1, \dots, n} a_i \leq b$. Tento předpoklad je bez újmy na obecnost.

Důkaz. Kdyby $a_i > b$ pro nějaké i , tak mohu zafixovat $x_i := 0$. □

Další typy. Kromě (8.1) lze uvažovat ještě řadu dalších variant, např.

$$\max c^T x \text{ za podm. } a^T x \leq b, x \geq 0, x \in \mathbb{Z}^n, \quad (8.2)$$

$$\max c^T x \text{ za podm. } a^T x = b, x \in \{0, 1\}^n. \quad (8.3)$$

Některé uvedené výsledky fungují analogicky i pro tyto varianty, jiné nikoli. Na některé případné odlišnosti upozorňujeme v textu, jiné necháváme čtenáři na rozmyšlenou.

Příklad 8.1 (Investování kapitálu). Uvažujme jednoduchý model investování kapitálu b jednotek peněz mezi m projektů, přičemž i -tý projekt vyžaduje počáteční investici a_i peněz a čistý výnos je c_i peněz. Úloha najít nejvýnosnější portfolio pak vede přímo na tvar (8.1). □

Příklad 8.2 (Problém dělení materiálu). Představme si materiál vyráběný ve velkých kusech (role textilu, ocelové trubky aj.) délky b , ale využítujeme pouze díly o délkách a_i , $i = 1, \dots, n$. Chceme-li maximálně využít materiál a mít tak minimální odpad, řešíme úlohu (8.2)

$$\max a^T x \text{ za podm. } a^T x \leq b, x \geq 0, x \in \mathbb{Z}^n. \quad \square$$

Preprocessing I.

Je-li $\sum_i a_i \leq b$, pak optimum je $x = 1$. Je-li $a_i > b$ pro nějaké i , pak můžeme zafixovat $x_i := 0$.¹⁾

¹⁾Tento případ nenastane u původní úlohy díky Předpokladu 2, ale může se uplatnit u podúlohy při řešení metodou branch & cut – sekce 8.4.

8.1 Složitost

Věta 8.3 (Korte & Schrader, 1981). *Problém batohu je NP-úplný (všechny varianty).* $\square$

Přestože je problém batohu je NP-úplný, není silně NP-úplný. To protože existuje pseudopolynomiální algoritmus, který běží polynomiálně při unárním kódování vstupu. Jinými slovy, je polynomiální vzhledem k velikosti vstupu (při standardním kódování) a největšímu číslu na vstupu (což v našem případě můžeme brát b). Důsledkem je, že pro malé b poběží pseudopolynomiální algoritmus rychle.

Pseudopolynomiálních algoritmů pro problém batohu existuje povícero, ukážeme si jeden z nich využívajícího principu dynamického programování.

Algoritmus 8.4 (Pseudopolynomiální algoritmus I.). Mějme pole S délky $b+1$, indexováno od 0, udávající v $S[\alpha]$ nejlepší cenu $\sum_{i=1}^k c_i x_i$ pro daný součet hmotností $\alpha := \sum_{i=1}^k a_i x_i$. Na počátku nastav $S[0] := 0$, ostatní jsou -1 .

Algoritmus sestává ze základního cyklu o n iteracích. V k -té iteraci přidáváme k -tou proměnnou a prozkoumáme jestli můžeme dosáhnout dalšího součtu hmotností a/nebo lepší ceny. To provedeme tak, že pro každé $i = b - a_k, \dots, 1, 0$ takové, že $S[i] \geq 0$, zjistíme, co způsobí přidání dalšího předmětu. Jestliže $S[i] + c_k > S[i + a_k]$, dosadíme $S[i + a_k] := S[i] + c_k$.

Nakonec z pole S vybereme největší hodnotu, která udává optimální hodnotu úlohy. Pokud si u jednotlivých nezáporných složek pole S pamatujeme i index k , můžeme jednoduše zrekonstruovat jak jsme největší hodnotu dostali a jaké (resp. jaká všechna) optimální řešení jí odpovídá.

Příklad 8.5. Adaptujte algoritmus i na ostatní varianty problému batohu (8.2) a (8.3).

Řešení: Pro (8.2) bude jediná změna v testu přidání k -tého předmětu, který mohu přidávat vícekrát. Čili nový test bude probíhat pro $i = 0, 1, \dots, b - a_k$, protože to umožní opakovaně přidávat jeden předmět.

Pro (8.3) bude naopak jediná změna v nalezení optimální hodnoty, která se nachází v $S[b]$. Je-li $S[b] = -1$, pak úloha nemá přípustné řešení. $\square$

Příklad 8.6. Pomocí Algoritmu 8.4 vyřešte úlohu

$$\max x_1 + 5x_2 + 3x_3 + x_4 + 2x_5 \quad \text{za podm.} \quad 3x_1 + 4x_2 + 3x_3 + 2x_4 + x_5 \leq 7, \quad x \in \{0, 1\}^5.$$

Řešení: V tabulce dole je stav pole S na začátku a v průběhu jednotlivých iterací:

indexy pole S	0	1	2	3	4	5	6	7
poč. stav	0	-1	-1	-1	-1	-1	-1	-1
přidání x_1	0	-1	-1	1	-1	-1	-1	-1
přidání x_2	0	-1	-1	1	5	-1	-1	6
přidání x_3	0	-1	-1	3	5	-1	4	8
přidání x_4	0	-1	1	3	5	4	6	8
přidání x_5	0	2	1	3	5	7	6	8

Vidíme, že optimální hodnota je 8. Optimální řešení dopočítáme zpětně, nebo je možnost v průběhu algoritmu pamatovat jednotlivá dílčí řešení. Optimální řešení jsou dvě, $(0, 1, 1, 0, 0)^T$ a $(0, 1, 0, 1, 1)^T$.

Poznamenejme, že z výsledné tabulky vidíme také optimální hodnoty pro problémy batohu s pravou stranou postupně $0, 1, \dots, b$. Nerovnost v zadání problému se nemusí nabýt jako rovnost pro optimální řešení. To je názorně vidět na případu kdy by pravá strana byla $b' = 2$. Pak optimální hodnota není 1, což je maximální hodnota pro řešení splňující nerovnost jako rovnost, ale 2 pro řešení $x = (0, 0, 0, 0, 1)^T$, které splňuje nerovnost ostře. $\square$

Složitost pseudopolynomiálního algoritmu 8.4 je $\mathcal{O}(nb)$ aritmetických operací, což vzhledem k Preprocessingu I. je také $\mathcal{O}(n \sum_{i=1}^n a_i)$. Podobně můžeme sestavit pseudopolynomiální algoritmus, který bude polynomiální vzhledem k n a koeficientům ve vektoru c .

Algoritmus 8.7 (Pseudopolynomiální algoritmus II.). Uvažujme úlohu s parametrem φ

$$\min a^T x \text{ za podm. } c^T x \geq \varphi, \quad x \in \{0, 1\}^n. \quad (8.4)$$

Pokud je optimální hodnota menší nebo rovna b , optimální řešení je přípustným řešením původní úlohy (8.1), mající hodnotu účelové funkce aspoň φ . Jinak takové přípustné řešení neexistuje. Binárním prohledáváním parametru $\varphi \in [0, K]$, kde $K = \sum_{i=1}^n c_i$ tak v řádově $\log(K)$ iteracích najdeme optimum (8.1).

Úloha (8.4) je vlastně problém batohu (stačí uvažovat substituci $x' := e - x$), lze řešit pseudopolynomiálním algoritmem 8.4 v čase $\mathcal{O}(nK)$. Celkem dostáváme složitost $\mathcal{O}(nK \log(K))$ aritmetických operací. Složitost můžeme dále snížit na $\mathcal{O}(nK)$, pokud budeme úlohu (8.4) řešit variantou algoritmu 8.4, která na konci projde pole S od největší hodnoty a vrátí první položku s účelovou hodnotou nanejvýš b .

Pro problém batohu dokonce existuje úplné polynomiální aproximační schema.

Věta 8.8 (Ibarra & Kim, 1975). *V polynomiálním čase vzhledem k $1/\varepsilon$ a velikosti vstupu lze najít přípustné $x^* \in \{0, 1\}^n$ tak, že*

$$c^T x^* \geq (1 - \varepsilon) \max c^T x \text{ za podm. } a^T x \leq b, \quad x \in \{0, 1\}^n.$$

Důkaz. Pro $\delta := \varepsilon n^{-1} \max_{i=1, \dots, n} c_i$ přeskálujeme koeficienty účelové funkce $c'_i := \lfloor c_i / \delta \rfloor$. Díky volbě δ se optimální hodnota změní pouze málo. Protože $c'_i \leq n/\varepsilon$, tak pseudopolynomiální algoritmus 8.7 vyřeší novou úlohu v čase polynomiálním vzhledem k n a $1/\varepsilon$.

Zaokrouhlení v definici c'_i způsobí, že transformace není ekvivalentní a optimum se může změnit. Na druhou stranu, díky volbě δ se optimální hodnota změní pouze málo. Je-li x^* optimum úlohy s účelovou funkcí c' , a x^{opt} optimum původní úlohy, tak

$$\begin{aligned} c^T x^* &\geq \delta c'^T x^* \geq \delta c'^T x^{\text{opt}} = \delta \lfloor c^T / \delta \rfloor x^{\text{opt}} \geq \delta(c/\delta - e)^T x^{\text{opt}} \\ &\geq c^T x^{\text{opt}} - \delta n = c^T x^{\text{opt}} - \varepsilon \max_i c_i \geq (1 - \varepsilon) c^T x^{\text{opt}}, \end{aligned}$$

kde jsme v poslední nerovnici použili fakt, že $c^T x^{\text{opt}} \geq \max_i c_i$ díky Předpokladu 2. □

Poznámka 8.9 (Celočíselné vs. racionální koeficienty). Problém batohu s celočíselnými koeficienty (8.1) je NP-úplný problém, ale nikoliv silně NP-úplný problém, protože pro něj existuje pseudopolynomiální algoritmus. Pokud ovšem budeme uvažovat racionální hodnoty $a, c \in \mathbb{Q}^n$ a $b \in \mathbb{Q}$, úloha se stane silně NP-úplnou [Wojtczak, 2018]; převod na celočíselné koeficienty přenásobením nejmenším společným násobkem jmenovatelů zde nefunguje, protože by to vedlo k číslům s exponenciálně velkou velikostí v unárním zápisu. Zajímavé také je, že pro tento problém batohu s racionálními koeficienty stále existuje úplné polynomiální aproximační schema.

Přestože je problém batohu speciální úlohou celočíselného programování, lze každý rovnicový 0-1 lineární program zredukovat na problém batohu, varianta (8.3). Těto redukci se říká agregace omezení a je popsána dole. Jednoduše řečeno, rovnice dáme dohromady z pohledu q -ární číselné soustavy, kde q je dost velké.

Věta 8.10 (Glover & Woolsey, 1972). *Bud' $A \in \mathbb{Z}^{m \times n}$, $b \in \mathbb{Z}^m$ a definujme $q := 1 + 2 \max_i \{\sum_j |a_{ij}|, |b_i|\}$. Pak soustava*

$$Ax = b, \quad x \in \{0, 1\}^n \quad (8.5)$$

je ekvivalentní s

$$\sum_{j=1}^n \left(\sum_{i=1}^m q^{i-1} a_{ij} \right) x_j = \sum_{i=1}^m q^{i-1} b_i, \quad x \in \{0, 1\}^n. \quad (8.6)$$

Důkaz. „ $\Rightarrow$ “ Jasně, neboť rovnice (8.6) je jen lineární kombinací rovnic z (8.5).

„ $\Leftarrow$ “ Bud' $k \in \{1, \dots, m\}$. Vezměme obě strany rovnice (8.6) modulo q^k (sktriktně vzato, výsledek nebude mezi 0 a $q^k - 1$, ale posunutý mezi $-q^k/2$ a $q^k/2$). Bude to

$$\sum_{j=1}^n \left(\sum_{i=1}^k q^{i-1} a_{ij} \right) x_j = \sum_{i=1}^k q^{i-1} b_i, \quad (8.7)$$

neboť pro pravou stranu platí

$$\left| \sum_{i=1}^k q^{i-1} b_i \right| \leq \sum_{i=1}^k q^{i-1} |b_i| \leq \frac{1}{2} \sum_{i=1}^k q^{i-1} (q-1) = \frac{1}{2} (q^k - 1) < \frac{1}{2} q^k,$$

a podobně

$$\left| \sum_{j=1}^n \left(\sum_{i=1}^k q^{i-1} a_{ij} \right) x_j \right| = \left| \sum_{i=1}^k q^{i-1} \left(\sum_{j=1}^n a_{ij} x_j \right) \right| \leq \frac{1}{2} \sum_{i=1}^k q^{i-1} (q-1) < \frac{1}{2} q^k.$$

Protože obě strany rovnice (8.6) mohou být záporné, byl nutný odhad obou stran rovnice (8.7), aby v absolutní hodnotě byly menší než $\frac{1}{2} q^k$.

Podobně pokud vezmeme obě strany rovnice (8.6) modulo q^{k-1} , dostaneme

$$\sum_{j=1}^n \left(\sum_{i=1}^{k-1} q^{i-1} a_{ij} \right) x_j = \sum_{i=1}^{k-1} q^{i-1} b_i.$$

Odečtením od rovnice (8.7) pak získáme q^{k-1} -násobek k -té rovnice ze soustavy (8.5). Postupnou volbou $k \in \{1, \dots, m\}$ tak vygenerujeme všechny rovnice z (8.5). $\square$

Důsledek 8.11 (Papadimitriou, 1981). *Bud' $A \in \mathbb{Z}^{m \times n}$, $b \in \mathbb{Z}^m$. Při pevném m je úloha*

$$\max c^T x \text{ za podm. } Ax = b, \quad x \in \{0, 1\}^n$$

řešitelná v pseudopolynomiálním čase.

8.2 Relaxace

Relaxace pro (8.1)

Relaxace (8.1) vede na lineární program

$$\max c^T x \text{ za podm. } a^T x \leq b, \quad 0 \leq x \leq 1,$$

jehož řešení x^R můžeme vyjádřit explicitně. Předpokládejme, že proměnné jsou setříděné tak, že $\frac{c_1}{a_1} \geq \dots \geq \frac{c_n}{a_n}$. Pak x^R je ve tvaru

$$x_1^R = \dots = x_{r-1}^R = 1, \quad x_r^R = \frac{b - \sum_{i=1}^{r-1} a_i}{a_r}, \quad x_{r+1}^R = \dots = x_n^R = 0$$

pro určité $r \in \{1, \dots, n\}$. Optimální hodnota je pak

$$c^T x^R = \sum_{i=1}^{r-1} c_i + \frac{b - \sum_{i=1}^{r-1} a_i}{a_r} c_r.$$

Optimální řešení nám rovněž nabízí heuristické přípustné řešení x^H ve tvaru

$$x_1^H = \dots = x_{r-1}^H = 1, \quad x_r^H = \dots = x_n^H = 0$$

a účelovou hodnotou $c^T x^H = \sum_{i=1}^{r-1} c_i$. Optimální hodnotu tedy můžeme těsně zapouzdřit do intervalu

$$[c^T x^H, c^T x^R] = \left[\sum_{i=1}^{r-1} c_i, \sum_{i=1}^{r-1} c_i + \frac{b - \sum_{i=1}^{r-1} a_i}{a_r} c_r \right].$$

Poznámka 8.12. Setřídění posloupnosti $\frac{c_1}{a_1}, \dots, \frac{c_n}{a_n}$ vyžaduje čas $\mathcal{O}(n \log n)$. Relaxované optimum můžeme najít dokonce v lineárním čase bez třídění. Využijeme toho, že medián posloupnosti lze najít v $\mathcal{O}(n)$. Pokryjeme-li kapacitu b pomocí první půlky, zaměříme se rekursivně na ni, jinak proměnné první půlky nastavíme na 1 a rekursivně se zaměříme na druhou půlku. Tak potřebujeme pouze čas $\mathcal{O}(n) + \mathcal{O}(n/2) + \mathcal{O}(n/4) + \dots = \mathcal{O}(n)$.

Relaxace pro (8.2)

Podobně můžeme postupovat pro variantu (8.2) problému batohu. Relaxovaná úloha má optimální řešení $x^R = (\frac{b}{a_1}, 0, \dots, 0)^T$ s optimální hodnotou $\frac{b}{a_1}c_1$. Heuristické přípustné řešení jest $x^H = (\lfloor \frac{b}{a_1} \rfloor, 0, \dots, 0)^T$ s účelovou hodnotou $\lfloor \frac{b}{a_1} \rfloor c_1$. Pro tuto variantu umíme dokázat i přesnost aproximace.

Tvrzení 8.13. Heuristické řešení není víc jak dvakrát horší než optimum, tj. $c^T x^H \geq \frac{1}{2} c^T x^*$.

Důkaz. Ukážeme víc, dokonce $c^T x^H \geq \frac{1}{2} c^T x^R$. Z Preprocessingu I. víme, že $a_1 \leq b$, tedy $\lfloor \frac{b}{a_1} \rfloor \geq 1 \geq \{ \frac{b}{a_1} \}$. Nyní

$$\frac{c^T x^H}{c^T x^R} = \frac{\lfloor \frac{b}{a_1} \rfloor c_1}{\frac{b}{a_1} c_1} = \frac{\lfloor \frac{b}{a_1} \rfloor}{\lfloor \frac{b}{a_1} \rfloor + \{ \frac{b}{a_1} \}} \geq \frac{\lfloor \frac{b}{a_1} \rfloor}{2 \lfloor \frac{b}{a_1} \rfloor} = \frac{1}{2}. \quad \square$$

Příklad 8.14. Pro variantu (8.1) takováto relativní chyba neplatí. Uvažujme například úlohu s $a = (1, 100)^T$, $b = 100$, $c = (2, 100)^T$. Pak heuristické řešení je $x^H = (1, 0)^T$ s hodnotou účelové funkce $c^T x^H = 2$, ale optimální řešení je $x^* = (0, 1)^T$ s optimální hodnotou $c^T x^* = 100$. $\square$

Odhad absolutní chyby:

Tvrzení 8.15. Pro obě varianty (8.1) a (8.2) je $c^T x^* - c^T x^H \leq c^T x^R - c^T x^H \leq \max_{i=1, \dots, n} c_i$.

Důkaz. Pro (8.1) je $c^T x^R - c^T x^H \leq c_r \leq \max_i c_i$. Pro (8.2) je $c^T x^R - c^T x^H \leq c_1 \leq \max_i c_i$. $\square$

8.3 Sečné nadroviny

Budeme uvažovat sečné nadroviny tzv. typu „cover inequalities“. Buď $C \subseteq \{1, \dots, n\}$ tak, že $\sum_{i \in C} a_i > b$. Pak nemohou být všechny proměnné z C nastaveny na jedničku, a proto lze přidat podmínku

$$\sum_{i \in C} x_i \leq |C| - 1. \quad (8.8)$$

Množina C se nazývá *cover*, a speciálně je to minimální cover, pokud žádná vlastní podmnožina už cover netvoří.

Podmínku (8.8) lze jednoduše zesílit na tvar

$$\sum_{i \in J} x_i \leq |C| - 1,$$

kde $J := C \cup \{j; a_j \geq a_i \ \forall i \in C\}$.

Jiný způsob jak zesílit podmínku je tzv. *lifting*. Cílem je dostat podmínku tvaru

$$\sum_{i \notin C} \alpha_i x_i + \sum_{i \in C} x_i \leq |C| - 1,$$

kde $\alpha_i \in \mathbb{N}$. Pro jednoduchost předpokládejme, že $C = \{s+1, \dots, n\}$. Hodnoty $\alpha_1, \dots, \alpha_s$ budeme počítat postupně, V iteraci t již známe $\alpha_1, \dots, \alpha_{t-1}$ chceme určit co největší $\alpha_t \in \mathbb{N}$ aby platilo

$$\alpha_t x_t + \sum_{i=1}^{t-1} \alpha_i x_i + \sum_{i \in C} x_i \leq |C| - 1$$

pro všechna přípustná x , což vede na úlohu

$$\alpha_t = |C| - 1 - \max \left\{ \sum_{i=1}^{t-1} \alpha_i x_i + \sum_{i \in C} x_i; \sum_{i=1}^{t-1} a_i x_i + \sum_{i \in C} a_i x_i \leq b - a_t, x \in \{0, 1\}^{n-s+t-1} \right\}.$$

Toto je sice opět problém batohu, ale menší dimenze, což bývá ještě umocněno tím, že často bývá $\alpha_i = 0$. Pokud tuto úlohu vyřešíme přesně, je výsledná sečná nadrovina stěnová [Wolsey, 1998], nicméně i dolní odhad na α_t může dát silný řez.

Zbývá otázka jak volit cover C . To vyplyne často samo. Je-li x^* neceločíselné optimální řešení relaxace, pak můžeme volit $C := \{i; x_i^* > 0\}$.

Příklad 8.16. Uvažme příklad 8.6 a buď x^* optimum relaxace. Najděte příslušný minimální cover a pomocí liftingu zesilte řez. Přidejte řez do podmínek a najděte příslušné relaxované optimální řešení.

Řešení: Setříděním proměnných sestupně podle c_i/a_i dostaneme úlohu

$$\max 2x_1 + 5x_2 + 3x_3 + x_4 + x_5 \text{ za podm. } x_1 + 4x_2 + 3x_3 + 2x_4 + 3x_5 \leq 7, x \in \{0, 1\}^5.$$

Lineární relaxace má optimální řešení $x^R = (1, 1, \frac{2}{3}, 0, 0)^T$ a příslušný cover je $C = \{1, 2, 3\}$. Snadno se nahlédne, že je minimální.

Lifting α_4 : Spočítáme úlohu

$$\max x_1 + x_2 + x_3 \text{ za podm. } x_1 + 4x_2 + 3x_3 \leq 7 - a_4 (= 5), x \in \{0, 1\}^5.$$

Optimální hodnota je 2, tudíž $\alpha_4 = |C| - 1 - 2 = 0$.

Lifting α_5 : Spočítáme úlohu

$$\max x_1 + x_2 + x_3 \text{ za podm. } x_1 + 4x_2 + 3x_3 \leq 7 - a_5 (= 4), x \in \{0, 1\}^5.$$

Optimální hodnota je 2, tudíž $\alpha_5 = |C| - 1 - 2 = 0$.

Tudíž nejsilnější cover nerovnost je $x_1 + x_2 + x_3 \leq 2$. Po přidání k relaxované úloze dostaneme lineární program

$$\begin{aligned} \max \quad & 2x_1 + 5x_2 + 3x_3 + x_4 + x_5 \\ \text{za podm.} \quad & x_1 + 4x_2 + 3x_3 + 2x_4 + 3x_5 \leq 7, x_1 + x_2 + x_3 \leq 2, x \in [0, 1]^5, \end{aligned}$$

jehož optimální řešení $(0, 1, 1, 0, 0)^T$ je již celočíselné. □

8.4 Branch & cut

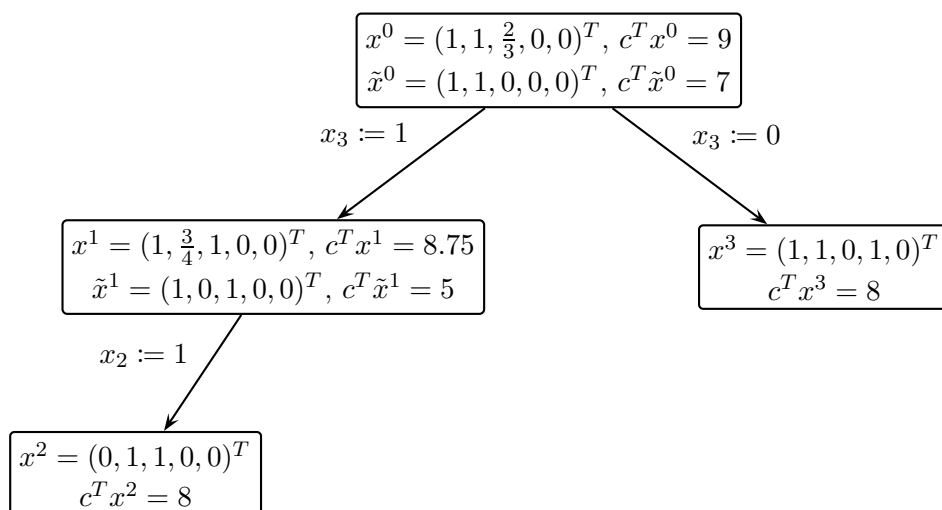
Metoda branch & bound má pro problém batohu jednodušší podobu. S každým rozvětvením klesne dimenze podproblému o 1 (tak jako pro každý 0-1 celočíselný program) a navíc z předchozího víme, že relaxovanou úlohu v každém uzlu umíme hned vyřešit. Zde se významně uplatní Preprocessing I.

Nicméně, jak jsme viděli v příkladu 4.5, i pro snadnou úlohu musí někdy metoda branch & bound provést exponenciální počet kroků. Přidáním sečných nadrovin sice ztratíme explicitní vyjádření řešení relaxovaných podúloh, zesílíme tím ale tyto relaxace a výpočetní čas většinou zlepšíme.

Podle Schrijver [1998] experimenty prováděné Balasem a Zemelem s náhodným vstupem a 10^4 proměnnými trvaly už roce 1980 pouze ca jednu sekundu. Použitá metoda kombinovala branch & bound, sečné nadroviny a dynamické programování.

Příklad 8.17. Vyřešte příklad 8.6 pomocí základní metody branch & bound.

Řešení: Postup metody je znázorněný v následujícím stromu:



V kořeni spočítáme řešení relaxované úlohy $x^0 = (1, 1, \frac{2}{3}, 0, 0)^T$ a příslušnou horní mez na optimální hodnotu $c^T x^0 = 9$. Odpovídající heuristické přípustné řešení je $\tilde{x}^0 = (1, 1, 0, 0, 0)^T$ a účelová hodnota $c^T \tilde{x}^0 = 7$. V levé větvi zafixujeme $x_3 := 1$. Optimum relaxované úlohy je $x^1 = (1, \frac{3}{4}, 1, 0, 0)^T$ s hodnotou $c^T x^1 = 8.75$. Příslušné heuristické řešení je $\tilde{x}^1 = (1, 0, 1, 0, 0)^T$, ale účelová hodnota $c^T \tilde{x}^1 = 5$ je malá, takže jej nemusíme ukládat. Další levá větev má optimum relaxace rovnou celočíselné $x^2 = (0, 1, 1, 0, 0)^T$. Účelová hodnota je $c^T x^2 = 8$, což vylepšuje dosavadní nejlepší přípustné řešení. Zároveň to dává lepší dolní odhad na optimální hodnotu. Tudíž backtrackujeme zpět: otcovský uzel můžeme vynechat, protože horní mez $c^T x^1 = 8.75$ nic lepšího neslibuje. Dojedeme až ke kořeni a vydáme se pravou větví s omezením $x_3 = 0$. Optimum relaxované úlohy je $x^3 = (1, 1, 0, 1, 0)^T$ s hodnotou $c^T x^3 = 8$. Tudíž 8 musí být optimální hodnota a x^2 a x^3 dvě optimální řešení. $\square$

8.5 Preprocessing II.

Bud' r hodnota ze sekce věnované relaxaci a předpokládejme i setříděnost proměnných. Tím pádem se bude čekat, že optimální řešení pro první proměnné budou spíš jedničky, a pro ty na konci spíš nuly. Proto postupně pro každé $i = 1, \dots, r$ zafixujeme $x_i = 0$, a pokud horní mez optimální hodnoty úlohy s touto fixací (spočítaná např. relaxací) bude menší nebo rovna dolní mezi optimální hodnoty původní úlohy (spočítané např. heuristikou), pak můžeme natrvalo přiřadit $x_i = 1$. Podobně, pro každé $i = r + 1, \dots, n$ zafixujeme $x_i = 1$, a pokud horní mez optimální hodnoty úlohy s touto fixací bude menší nebo rovna dolní mezi optimální hodnoty původní úlohy, pak natrvalo přiřadíme $x_i = 0$.

Abychom nemuseli počítat v každém kroku znovu relaxované optimum (přestože to nestojí moc práce), můžeme optimální hodnotu relaxace po zafixování $x_i = 0$, $i < r$, shora odhadnout pomocí $c^T x^R - c_i + \frac{c_r}{a_r} a_i$, což zabere konstantní čas. Podobně po zafixování $x_i = 1$, $i > r$, použijeme horní odhad $c^T x^R + c_i - \frac{c_r}{a_r} a_i$.

8.6 Aplikace: sečné nadroviny pro 0-1 programování

Uvažujme 0-1 program

$$\max c^T x \text{ za podm. } Ax \leq d, \quad x \in \{0, 1\}^n,$$

kde $A \in \mathbb{Z}^{m \times n}$, $c \in \mathbb{Z}^n$ a $d \in \mathbb{Z}^m$. Je-li matice A velká a řídká, pak jednotlivé nerovnice se chovají téměř jako samostatné problémy batohu. Je-li x^* neceločíselné optimum relaxace, tak můžeme řezné nadroviny vytvářet pomocí cover nerovnic pro jednotlivé nerovnice soustavy. Tento postup se prý osvědčil [Schrijver, 1998].

Uvažujme jen jednu z nerovnic ve tvaru $a^T x \leq b$ a chceme najít cover $C \subseteq \{1, \dots, n\}$ tak, aby

$$\sum_{i \in C} a_i > b, \quad \sum_{i \in C} x_i^* > |C| - 1,$$

nebo spíše

$$\sum_{i \in C} a_i \geq b + 1, \quad \sum_{i \in C} x_i^* \geq |C|. \quad (8.9)$$

První podmínka vychází z definice coveru a druhá slouží k odříznutí bodu x^* sečnou nadrovinou (8.8). Nyní nefunguje postup ze sekce 8.3 na vytvoření C , protože cover definovaný jako $C := \{i; x_i^* > 0\}$ by nemusel splňovat podmínku $\sum_{i \in C} a_i > b$, a musíme na to tudíž jinak. Zavedeme-li $z \in \{0, 1\}^n$ indikátorové proměnné množiny C , tak hledáme řešení

$$\sum_{i=1}^n a_i z_i \geq b + 1, \quad \sum_{i=1}^n x_i^* z_i \geq \sum_{i=1}^n z_i,$$

což vede na problém batohu

$$\min \sum_{i=1}^n (1 - x_i^*) z_i \quad \text{za podm.} \quad \sum_{i=1}^n a_i z_i \geq b + 1, \quad z \in \{0, 1\}^n. \quad (8.10)$$

Tvrzení 8.18. *Buď z^* optimální řešení a α^* optimální hodnota (8.10).*

- (1) *Je-li $\alpha^* > 0$, tak x^* splňuje každou cover nerovnici typu (8.9) pro $a^T x \leq b$.*
- (2) *Je-li $\alpha^* \leq 0$, tak $C := \{i; z_i^* = 1\}$ je cover pro nejvíce porušenou cover nerovnost $\sum_{i \in C} x_i \leq |C| - 1$ (je porušena o $1 - \alpha^*$).*

Přestože takovéto generování sečných nadrovin vyžaduje řešit problémy batohu jako podúlohy, je takto strávený čas výrazně menší v porovnání s původní úlohou. Navíc místo přesného řešení lze aplikovat heuristiky (ale na úkor síly řezu) nebo ε -aproximaci dle věty 8.8.

Cvičení

8.1. Uvažujme varianty problému batohu

$$\max c^T x \quad \text{za podm.} \quad a^T x \leq b, \quad x \in \{0, 1\}^n, \quad (P_1)$$

$$\max c^T x \quad \text{za podm.} \quad a^T x \leq b, \quad x \geq 0, \quad x \in \mathbb{Z}^n, \quad (P_2)$$

$$\max c^T x \quad \text{za podm.} \quad Ax \leq b, \quad x \in \{0, 1\}^n, \quad (P_3)$$

$$\max c^T x \quad \text{za podm.} \quad a^T x = b, \quad x \in \{0, 1\}^n, \quad (P_4)$$

kde $c, A, a, b \geq 0$.

- (a) Ukažte, že předpoklad $a, b, c > 0$ není v (P_2) na újmu obecnosti. Jak je to u (P_3) a (P_4) ?
 - (b) Rozhodněte, která z variant (P_1) – (P_4) je převoditelná na nějakou jinou, popř. které jsou na sebe navzájem převoditelné.
 - (c) Pro (P_3) navrhněte heuristiku a horní odhad její relativní chyby.
 - (d) Prostudujte metodu Branch & Bound pro problémy (P_3) a (P_4) .
 - (e) Prostudujte preprocessing pro problémy (P_2) – (P_4) .
- 8.2. Pro problém batohu (8.1) s předpoklady $a > 0$ a $\max_i \{a_i\} \leq b$ ukažte, že podmínka $x_i \geq 0$ je stěnová.
- 8.3. Aplikujte Lagrangeovu relaxaci (a porovnejte ji s klasickou celočíselnou relaxací) na problém batohu (8.1).
- 8.4. Najděte minimální covery a příslušné sečné nadroviny pro množinu

$$\{x \in \{0, 1\}^7 : 11x_1 + 6x_2 + 6x_3 + 5x_4 + 5x_5 + 4x_6 + x_7 \leq 19\}.$$

8.5. Řešte problém batohu pomocí metody branch & bound:

$$\begin{array}{ll}\max & 17x_1 + 10x_2 + 25x_3 + 17x_4 \\ \text{za podm.} & 5x_1 + 3x_2 + 8x_3 + 7x_4 \leq 12, \\ & x_1, x_2, x_3, x_4 \in \{0, 1\}.\end{array}$$

- 8.6. Pizzerie *U mlsného Karla* nabízí n druhů pizzy, přičemž pizza i se peče t_i minut. Karel má 2 pece. Poradte Karlovi jak v co nejkratším čase upéci všechny druhy pizzy. To jest, zformulujte to jako optimalizační úlohu.
- 8.7. Malý Karlík má kartičky s přirozenými čísly a chce je rozdělit an dvě hromádky tak, aby součty v obou hromádkách byly stejné. Kolik maximálně se musí odebrat kartiček, aby to zaručeně nebylo možné?

Kapitola 9

Set covering

Pokrývací úloha *set covering* má tvar

$$\min c^T x \text{ za podm. } Ax \geq e, \quad x \in \{0, 1\}^n, \quad (9.1)$$

kde $A \in \{0, 1\}^{m \times n}$ a $c \in \mathbb{N}^n$.

Příklad 9.1. Zformulujte pomocí (9.1) úlohu vrcholového pokrytí grafu a úlohu maximálního párování v grafu. Protože úloha vrcholového pokrytí grafu je NP-těžká, je NP-těžký i problém set covering.

Řešení (Vrcholové pokrytí): Buď $G = (V, E)$, $|V| = n$, graf a chceme najít nejmenší množinu vrcholů tak, aby z každé hrany obsahovala aspoň jeden konec. Problém vede na formulaci

$$\min e^T x \text{ za podm. } A^T x \geq e, \quad x \in \{0, 1\}^n,$$

kde A je matice incidence grafu G (definice 1.11). Proměnné jsou booleovské indikátory vrcholů, které vybereme, a i -tá nerovnost v omezeních pak říká, že hrana i musí být pokryta.

Řešení (Maximální párování): Buď $G = (V, E)$, $|E| = m$, graf a chceme najít největší množinu disjunktních hran. Problém vede na formulaci

$$\max e^T x \text{ za podm. } Ax \leq e, \quad x \in \{0, 1\}^m,$$

kde A je matice incidence grafu G . Binární proměnné indikující výběr hrany a i -tá nerovnost v omezeních pak říká, že vrchol i musí být pokryt maximálně jednou hranou. Poznámka: Při váženém párování nahradíme účelovou funkci $e^T x$ za $c^T x$.

Nyní zavedeme substituci $y := e - x$ a úloha bude

$$m - \min e^T y \text{ za podm. } Ay \geq Ae - e, \quad y \in \{0, 1\}^m.$$

Přestože to není přesně požadovaný tvar (9.1), je to velmi podobný problém. Jediný rozdíl je v pravé straně omezení, kdy vektor $Ae - e$, udávající seznam stupňů vrcholů snížený o 1, se obecně neskládá jen z jedniček. Nicméně za předpokladu, že G neobsahuje vrcholy stupně nula nebo jedna (což je bez újmy na obecnosti), je vektor $Ae - e$ nezáporný. $\square$

Příklad 9.2 (Emergency servis centers). Pomocí minimálního počtu nouzových středisek chceme pokrýt celý kraj. Tento problém vede přímo na formulaci (9.1), kde x_j udává jestli se j -té středisko otevře či nikoli a a_{ij} udává zda j -té středisko dokáže obsloužit i -tou oblast. $\square$

Poznámka 9.3 (Set partitioning). Podobná úloha je problému set covering je např. *set partitioning* s rovnostmi namísto nerovností:

$$\min c^T x \text{ za podm. } Ax = e, \quad x \in \{0, 1\}^n,$$

kde $A \in \{0, 1\}^{m \times n}$ a $c \in \mathbb{N}^n$. Jak ukazujeme dole, tato úloha lze převést na (9.1), takže se budeme nadále věnovat pouze set coveringu.

Důkaz. Úloha set partitioningu je ekvivalentní s úlohou

$$\min c^T x + \alpha e^T y \text{ za podm. } Ax - y = e, \quad y \geq 0, \quad x \in \{0, 1\}^n, \quad y \in \mathbb{N}^m,$$

kde $\alpha > 0$ je dostatečně velké (stačí brát $\alpha = e^T c + 1$). Eliminací y (tj., dosazením $y := Ax - e$) dostaneme úlohu

$$\min c^T x + \alpha e^T (Ax - e) \text{ za podm. } Ax \geq e, \quad x \in \{0, 1\}^n,$$

neboli

$$\alpha m + \min (c^T + \alpha e^T A)x \text{ za podm. } Ax \geq e, \quad x \in \{0, 1\}^n.$$

Pokud je optimální hodnota aspoň α , znamená to, že původní úloha set partitioningu není přípustná. V opačném případě se optimální řešení i hodnota rovnají. $\square$

9.1 Branch & bound

Metodu branch & bound pro úlohu set coveringu prvně zkoumali Lemke, Salkin & Spielberg (1971).

Metoda branch & bound v principu funguje tak, jak jsme ji popisovali v kapitole 4, ale má přeci jenom jistá specifika.

Pokud větvíme úlohu na dvě podúlohy podle proměnné x_k , tak obě podúlohy mají menší dimenzi díky přidání podmínce $x_k = 1$ resp. $x_k = 0$. V prvním případě se navíc sníží počet omezení, protože pro $a_{ik} = 1$ se stává i -tá nerovnice redundantní. Tudíž můžeme vypustit $e^T A_{*k}$ nerovnic.

Jak volit k podle kterého se větvíme? Existuje několik přístupů:

- $k := \arg \min_j c_j$, protože tato proměnná má nejmenší vliv na účelovou funkci.
- $k := \arg \max_j e^T A_{*j}$, protože čím více jedniček má matice A v k -tém sloupci, tím více nerovností pak bude redundantních v jedné větvi prohledávacího stromu a rychleji najdeme dobré řešení.
- $k := \arg \min_j \frac{c_j}{e^T A_{*j}}$, kombinace předchozích dvou přístupů.
- Typicky se volba ještě zužuje na tzv. *preferable set* P , což jsou indexy jedniček v tom řádku matice A s nejméně jedničkami. Protože aspoň jedna z nich musí být pokryta, je větší šance, že větev stromu s $x_k = 1$ vede k optimu.

9.2 Preprocessing

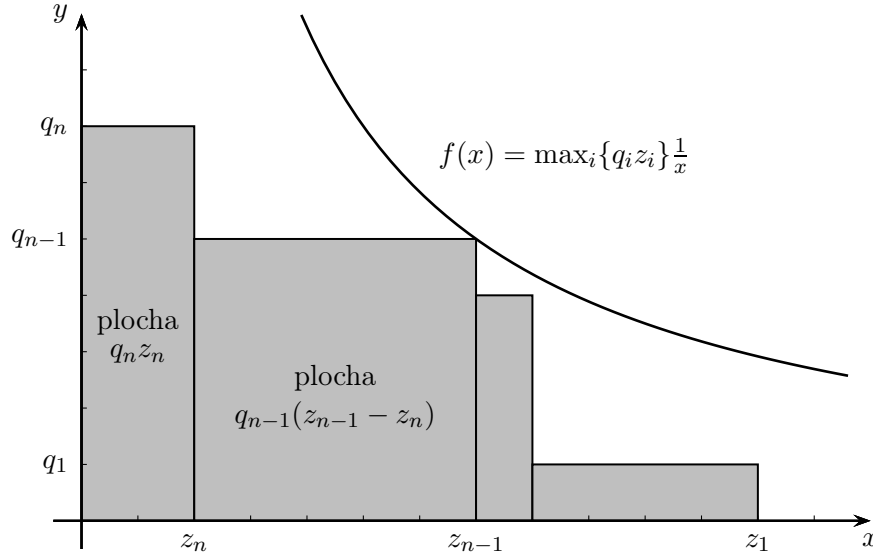
Několik jednoduchých pravidel, které se projeví zejména během procházení výpočetního stromu metodou branch & bound:

- Existuje-li řádek matice A se samými nulami, tj. $A_{i*} = 0^T$, pak je úloha nepřípustná (a naopak).
- Existuje-li řádek matice A s právě jednou jedničkou na k -té pozici, tj. $A_{i*} = e_k^T$, pak můžeme zafixovat $x_k := 1$.
- Řádková dominance: Je-li $A_{i*} \geq A_{j*}$, pak i -tá nerovnost je redundantní a lze vypustit.
- Sloupcová dominance: Pokud pro nějakou indexovou množinu $J \subseteq \{1, \dots, n\} \setminus \{k\}$ platí $\sum_{j \in J} A_{*j} \geq A_{*k}$ a $\sum_{j \in J} c_j \leq c_k$, pak lze zafixovat $x_k := 0$.

9.3 Hladový algoritmus

Algoritmus 9.4 (Hladový algoritmus pro set covering problem).

- 1: Polož $M := \{1, \dots, m\}$, $x^H = 0$ a $S_j := \{i; a_{ij} = 1\}$.
- 2: Dokud $M \neq \emptyset$ dělej:



Obrázek 9.1: Ilustrace k důkazu Lemmatu 9.5.

- $j^* := \arg \min_{j: x_j^H=0} \frac{c_j}{|M \cap S_j|}$,
- $x_{j^*}^H := 1$,
- $M := M \setminus S_{j^*}$.

Množina M udává indexy ještě nepokrytých nerovností, volba j^* odpovídá heuristice ze sekce 9.1.

Lemma 9.5. *Budťe $0 < q_1 \leq \dots \leq q_n$ reálná a $z_1 \geq \dots \geq z_n$ přirozená. Pak platí*

$$\sum_{i=1}^{n-1} q_i(z_i - z_{i+1}) + q_n z_n \leq \max_{i=1, \dots, n} \{q_i z_i\} H(z_1), \quad (9.2)$$

kde $H(k) = 1 + 1/2 + 1/3 + \dots + 1/k$.

Důkaz. Levá strana nerovnosti (9.2) reprezentuje součet obsahů obdélníků $(0, z_n) \times (0, q_n)$, $(z_n, z_{n-1}) \times (0, q_{n-1})$, $\dots$, $(z_2, z_1) \times (0, q_1)$, viz obrázek 9.1. Graf funkce $f(x) = \max_i \{q_i z_i\} x^{-1}$ leží nad těmito obdélníky, což se snadno ověří dosazením hodnot $z_n, \dots, z_1$: Pro $x = z_k$ obdélník zasahuje vysoko q_k , zatímco $f(z_k) = \max_i \{q_i z_i\} z_k^{-1} \geq q_k z_k z_k^{-1} = q_k$.

Součet obsahů obdélníků shora odhadneme plochou pod grafem $f(x)$, přesněji řešeno plochu pod grafem pouze odhadneme součtem funkčních hodnot v bodech $1, \dots, z_1$ (protože $z_n, \dots, z_1 \in \mathbb{Z}$, a tudíž dostaneme jemnější dělení). To dá přímo pravou stranu nerovnosti (9.2). $\square$

Věta 9.6. *Budť x^* optimální řešení (9.1). Pak $c^T x^H \leq H(d) c^T x^*$, kde $d := \max_{j=1, \dots, n} |S_j|$.*

Důkaz. Uvažujme relaxovanou úlohu k (9.1) ve tvaru lineárního programu

$$\min c^T x \text{ za podm. } Ax \geq e, \quad x \geq 0. \quad (9.3)$$

Duální problém je

$$\max e^T u \text{ za podm. } A^T u \leq c, \quad u \geq 0. \quad (9.4)$$

Namísto požadovaného odhadu ve skutečnosti dokážeme více, a to

$$c^T x^H = H(d) \cdot e^T u \leq H(d) \cdot c^T x^R \leq H(d) \cdot c^T x^*,$$

kde x^R je optimální řešení relaxace (9.3) a u je jisté přípustné řešení duálního problému (9.4).

Označme jako T počet iterací hladového algoritmu a pro iteraci t nechť M_t značí aktuální množinu M indexů nepokrytých nerovnic. Dále označme

$$q_t := \frac{c_{j_t^*}}{|M_t \cap S_{j_t^*}|} = \min_{j: x_j^H=0} \frac{c_j}{|M_t \cap S_j|}.$$

Nyní definujme přípustné řešení duálního problému (9.4) jako

$$u_i = \frac{q_t}{H(d)} \quad \text{pro } i \in M_t \setminus M_{t+1}.$$

Takto definované u je duálně přípustné, neboť pro každé $j \in \{1, \dots, n\}$ je

$$\begin{aligned} (A^T u)_j &= \sum_{i=1}^m u_i a_{ij} = \sum_{t=1}^T \frac{q_t}{H(d)} \sum_{i \in M_t \setminus M_{t+1}} a_{ij} = \sum_{t=1}^T \frac{q_t}{H(d)} \left(\sum_{i \in M_t} a_{ij} - \sum_{i \in M_{t+1}} a_{ij} \right) \\ &= \frac{1}{H(d)} \sum_{t=1}^T q_t (|M_t \cap S_j| - |M_{t+1} \cap S_j|). \end{aligned}$$

Podle Lemmatu 9.5 s volbou q_t a $z_t := |M_t \cap S_j|$ můžeme tuto hodnotu shora odhadnout

$$\begin{aligned} (A^T u)_j &\leq \frac{1}{H(d)} \max_{t=1, \dots, T} \{q_t |M_t \cap S_j|\} \cdot H(|M_1 \cap S_j|) \\ &\leq \frac{1}{H(d)} \max_{t=1, \dots, T} \left\{ \frac{c_j}{|M_t \cap S_j|} |M_t \cap S_j| \right\} \cdot H(d) = c_j. \end{aligned}$$

Nakonec porovnáme hodnoty duální účelové funkce a hladového řešení:

$$e^T u = \frac{1}{H(d)} \sum_{t=1}^T q_t (|M_t| - |M_{t+1}|) = \frac{1}{H(d)} \sum_{t=1}^T c_{j_t^*} = \frac{1}{H(d)} c^T x^H. \quad \square$$

Hladový algoritmus tedy dává řešení, která nejsou víc než $H(d)$ -krát horší než optimum. Navíc, poměrovou chybu můžeme odhadnout $H(d) \leq H(m) \approx \log(m)$. Celkem se věří, že neexistuje řádově lepší polynomiální aproximační algoritmus, protože by to pak znamenalo silné důsledky pro třídu NP [Feige, 1998].

Cvičení

9.1. Pomocí branch & bound metody vyřešte úlohu set partitioningu a daty

$$c = (3, 2, 1, 1, 3, 2)^T, \quad A = \begin{pmatrix} 1 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 \end{pmatrix}.$$

9.2. Pomocí branch & bound metody vyřešte úlohu set coveringu a daty

$$c = (4, 4, 4, 8, 3, 6)^T, \quad A = \begin{pmatrix} 1 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 \end{pmatrix}.$$

9.3. Najděte příklad, kdy se nabyde (co nejtěsněji) mez z věty 9.6.

Kapitola 10

Problém obchodního cestujícího

V problému obchodního cestujícího (TSP, traveling salesman problem) chceme najít nejkratší cestu procházející n městy tak, abychom každé město navštívili právě jednou. Tomuto populárnímu problému se věnuje řada prací jako Applegate et al. [2006]; Cook [2012].

Matematická formulace úlohy: Dán úplný orientovaný graf s vrcholy $V = \{1, \dots, n\}$ a matice ohodnocení $C \in \mathbb{R}^{n \times n}$, kde $c_{ii} = \infty \forall i$. Pak řešíme optimalizační úlohu

$$\min \sum_{i,j:i \neq j} c_{ij}x_{ij} \text{ za podm. } \sum_{i:i \neq k} x_{ik} = \sum_{j:j \neq i} x_{kj} = 1 \forall k \in V, \quad (10.1a)$$

$$x_{ij} \in \{0, 1\} \forall i \neq j, \quad (10.1b)$$

$$x \text{ představuje hamiltonovskou kružnici.} \quad (10.1c)$$

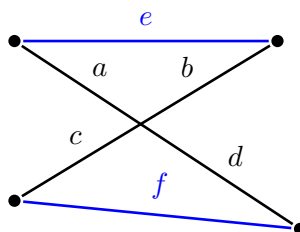
Rozeznáváme různé typy TSP. *Symetrické TSP* má matici C symetrickou a úloha se zjednodušuje na

$$\min \sum_{e \in E} c_e x_e \text{ za podm. } \sum_{e:k \in e} x_e = 2 \forall k \in V, x_e \in \{0, 1\} \forall e \in E, \quad (10.1c).$$

Pokud navíc platí trojúhelníková nerovnost $c_{ij} \leq c_{ik} + c_{kj}$ pro všechna $i, j, k, i \neq j$, tak se jedná o *metrické TSP*. A nejspeciálnějším typem je *eukleidovské TSP*, pro něj existuje vnoření do roviny tak, že vrcholy odpovídají bodům a c_{ij} eukleidovským vzdálenostem mezi nimi.

Tvrzení 10.1. *V eukleidovském TSP existuje optimální cesta a nakreslení v rovině s nekřížícími se hranami.*

Důkaz. Uvažujme dvojici křížících se hran, vyznačených černě na obrázku:



Tvrdíme, že tyto dvě hrany lze nahradit dvěma vodorovnými, vyznačenými modře (Případně dvěma svislými, podle toho, která z těchto dvou variant vede na hamiltonovskou kružnici. Ta druhá varianta vede na dvě podkružnice.) Z trojúhelníkové nerovnosti máme $e \leq a + b$, $f \leq c + d$, z čehož součtem je $e + f \leq (a + d) + (b + c)$. Tudíž součet délek nových hran je nanejvýš rovno součtu délek původních hran. Nejkratší cestu proto neprodloužíme. $\square$

Tvrzení 10.2. *Optimální cesta v eukleidovském TSP prochází body na hranici konvexního obalu bodů v tom pořadí, ve kterém jsou uspořádány podél této hranice.*

Příklad 10.3. Použití TSP v praxi:

1. Typickým příkladem je doprava & logistika (rozdávání zboží zákazníkům, ...). Moderní aplikace zahrnují také například TSP s drony, kdy dopravní vozidlo projíždí páteřní cestu a drony pak obsluhují jednotlivé zákazníky z dopravního vozidla.
2. Výroba a testování čipů: Při ověřování funkčnosti hotových čipů se spojují jednotlivé komponenty do cest, na kterých se provádí testování zkušebními daty. TSP umožňuje minimalizovat cesty, a tak i zkrátit testovací fázi [Cook, 2012].
3. Videohry: Při načítání textur ke scénám je třeba načítat z disku z různých míst. Aby se zefektivnila rychlost načítání, má smysl poskládat data cíleně, např. podle nejlepší cesty v grafu, kde vrcholy odpovídají texturám a délka hrany počtu scén, které obsahují jednu texturu, ale ne druhou [Cook, 2012].
4. Psychologie: Test se skládá z nalezení TSP s vrcholy označenými 1, A, 2, B, ..., 12, L, 13. Existuje souvislost s úspěšností v testu a poškozením mozku. Průzkum v r. 1990 ukázal, že je nejrozšířenějším testem a je stále stejný, aby se dosahovalo statisticky relevantních výsledků [Cook, 2012].
5. Archeologie: (relativní) datování hrobů. Petrie už v roce 1899 využil něco jako TSP k nalezení časové souslednosti hrobů v Horním Egyptě. Vrcholy TSP odpovídají hrobům a vzdálenosti pak Hammingově vzdálenosti určitých rysů a artefaktů nalezených v hrobech [Grötschel, 2012, str. 199].
6. řazení genů aj. □

Definice 10.4 (Aproximační faktor). Říkáme, že algoritmus řeší TSP s aproximačním faktorem φ pokud dává přípustné řešení, které není víc jak φ -krát horší než optimum.

Poznámka 10.5 (Složitost). TSP je NP-těžký problém, protože pomocí něho dokážeme vyřešit úlohu existence hamiltonovské kružnice v grafu. Toto platí i pokud se omezíme na bipartitní grafy, dokonce i jen na grafy ve tvaru podmnožiny 2D mřížky (Itai, Papadimitriou & Szwarcfiter).

NP-těžkost zůstane zachována i pro eukleidovské TSP, stejně jako pro různé modifikace TSP jako např. nalezení nejdelší cesty, bottleneck TSP (nalezení cesty, jejíž nejdelší hrana je co nejmenší) aj. Další složitostní výsledky:

- [Papadimitriou and Vempala, 2006]
Je NP-těžké aproximovat nesymetrické metrické TSP s faktorem $\frac{117}{116}$.
- [Papadimitriou and Vempala, 2006]
Je NP-těžké aproximovat symetrické metrické TSP s faktorem $\frac{220}{219}$.
- [Arora, 1996; Mitchell, 1999]
Existuje polynomiální aproximační schema pro eukleidovské TSP.

Zatím je otevřená otázka jestli existuje aproximační algoritmus na metrické TSP s konstantním faktorem menším než 1.5 (srov. Christofidův algoritmus dole). Pro obecné TSP žádný takový algoritmus neexistuje. Otevřené je také jestli eukleidovské TSP je ve třídě NP, protože díky eukleidovské metrice musíme počítat s odmocninami.¹⁾

Tvrzení 10.6. *Pro obecné TSP neexistuje polynomiální algoritmus s konstantním aproximačním faktorem.*

Důkaz. Redukcí z problému hledání hamiltonovské kružnice grafu $G = (V, E)$. Hrany ohodnot 1, a chybějící hrany doplň s váhou $1 + r$, kde $r > 0$. Pokud původní graf G obsahuje hamiltonovskou kružnici, tak optimální hodnota TSP je n , jinak aspoň $n + r$. Poměr je tedy $1 + r/n$ a může být libovolně velký. □

Jaká je nejhorší časová složitost pro TSP? Pokud bychom enumerovali všechny možné hamiltonovské kružnice, složitost by byla $\mathcal{O}(n!)$. Pomocí dynamického programování (Held & Karp, 1962) a rekursivního volání podúloh můžeme složitost snížit na $\mathcal{O}(n^2 2^n)$.

¹⁾Toto souvisí z tzv. Grahamovým problémem součtu odmocnin. Uvažme dvě řady čísel 1, 25, 31, 84, 87, 134, 158, 182, 198, a 2, 18, 42, 66, 113, 116, 169, 175, 199. Druhá má větší součet odmocnin, ale pokud ke každému číslu přičteme 10^6 , tak je součet odmocnin větší jen o $\approx 3 \cdot 10^{-37}$.

10.1 Formulace

Podmínku (10.1c) na hamiltonovskou kružnici lze zformulovat několika způsoby

- (1) *podmínka Miller–Tucker–Zemlin (1960)*: přidáme proměnné v_i , $i = 1, \dots, n$, $1 \leq v_i \leq n$, a podmínku

$$v_i - v_j + nx_{ij} \leq n - 1, \quad i = 1, \dots, n, \quad j = 2, \dots, n. \quad (10.2)$$

- (2) *podmínka Dantzig–Fulkerson–Johnson (1954)*:

$$\sum_{i \in U} \sum_{j \notin U} x_{ij} \geq 1, \quad \forall U \subseteq \{1, \dots, n\}, \quad 1 \leq |U| \leq n - 1. \quad (10.3)$$

- (3) *podmínka Dantzig–Fulkerson–Johnson (1954) jinak*:

$$\sum_{i \neq j \in U} x_{ij} \leq |U| - 1, \quad \forall U \subseteq \{1, \dots, n\}, \quad 1 \leq |U| \leq n - 1. \quad (10.4)$$

Důkaz.

- (1) Sporem, nechť máme bez újmy na obecnosti podcyklus $2, 3, \dots, k$, to jest $x_{23} = x_{34} = \dots = x_{k-1,k} = x_{k2} = 1$. Pak (10.2) mají instance

$$\begin{aligned} v_2 - v_3 &\leq -1, \\ v_3 - v_4 &\leq -1, \\ &\vdots \\ v_k - v_2 &\leq -1, \end{aligned}$$

a jejich součtem odvodíme $0 \leq -k$, což je spor.

Naopak, nechť nyní x odpovídá hamiltonovské kružnici. Setříd' vrcholy podle pořadí navštívení s tím, že začneme ve vrcholu 1. Přiřaď $v_i := k$ pokud vrchol i je v pořadí k -tý na kružnici (tedy speciálně $v_1 = 1$). Nyní, buďte $i \in \{1, \dots, n\}$, $j \in \{2, \dots, n\}$ libovolné. Pokud $x_{ij} = 0$, tak podmínka (10.2) má tvar $v_i - v_j \leq n - 1$, což je automaticky splněno. Pokud $x_{ij} = 1$, tak podmínka (10.2) má tvar $v_i - v_j \leq -1$, což je také splněno, protože vrchol j následuje na i na kružnici.

- (2) Jasně z toho, že pokud množinu vrcholů rozdělíme na dvě neprázdné části, tak hamiltonovská kružnice musí přejít od jedné do druhé.
- (3) Jasně z toho, že uvnitř každé vlastní části grafu je odpovídající část hamiltonovské kružnice pouze stromem (či lesem). $\square$

Pro symetrické TSP pak (10.3) a (10.4) mají tvar

$$\sum_{e \in E(U, V \setminus U)} x_e \geq 2, \quad \forall U \subseteq \{1, \dots, n\}, \quad 3 \leq |U| \leq \lfloor n/2 \rfloor,$$

a

$$\sum_{e \in E(U)} x_e \leq |U| - 1, \quad \forall U \subseteq \{1, \dots, n\}, \quad 3 \leq |U| \leq \lfloor n/2 \rfloor, \quad (10.5)$$

kde $E(U, U')$ značí množinu hran spojujících vrcholy z U s vrcholy z U' , a pro zjednodušení $E(U) := E(U, U)$.

Miller–Tucker–Zemlinova podmínka má výhodu, že omezení není mnoho, ale na druhou stranu nejsou příliš silné (že by byly stěnové ve smyslu pojmu ze strany 10). Naopak, (10.3) resp. (10.4) obsahují exponenciální počet nerovnic, ale jsou zaručeně stěnové pro $n \geq 3$ (Grötschel, Padberg, 1979). Přes vysoký počet nerovnic nemáme zaručeno, že řešení relaxované úlohy je optimální. Přesto se tato formulace (v této nebo jiné formě) používá spíše než Miller–Tucker–Zemlinova.

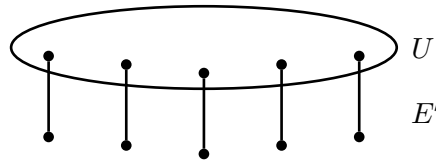
Poznámka 10.7 (Transformace nesymetrického TSP na symetrické TSP). Nesymetrické TSP můžeme snadno převést na symetrické TSP (Jonker & Volgenant, 1983). Základem transformace je zdvojení vrcholů, takže nová úloha má množinu vrcholů $V^* = V \cup V' = \{1, \dots, n\} \cup \{1', \dots, n'\}$. Nová matice vzdáleností C^* bude tvaru

$$C^* = \begin{pmatrix} U & \hat{C} \\ \hat{C}^T & U \end{pmatrix},$$

kde prvky matice U jsou dostatečně velké hodnoty (v principu může i být ∞). Matice $\hat{C}$ je rovna matici C , jen s tím rozdílem, že $\hat{c}_{i,i'} = \varepsilon$, kde $\varepsilon > 0$ je dost malá hodnota (můžeme ale i položit tuto hodnotu jako nulovou nebo i zápornou). Hrany typu (i', j) , z nového do starého vrcholu, tak přispívají podle vzdálenosti c_{ij} , zatímco hrany typu (i, j') , přispívají podle obrácené vzdálenosti c_{ji} . Hrany typu (i, j) a (i', j') se nepoužijí, protože jsou moc drahé, a naproti tomu hrany typu (i, i') se použijí, protože jsou dostatečně levné.

10.2 Sečné nadroviny

Zaměříme se na podmínku (10.4), která jde následujícím způsobem ještě zesílit [Nemhauser and Wolsey, 1999], a pomůže odříznout další nepřipustná řešení při celočíselné relaxaci. *Hřebenem* nazveme množinu $U \subseteq V$ z (10.4) spolu s množinou hran E' , kde hran je lichý počet a každá má právě jeden konec v U . Pak můžeme pro symetrické TSP odvodit následující podmínku. Pokud $|E'| \geq 3$, tak je podmínka stěnová.



Věta 10.8 (Hřebenová nerovnost). *Za výše uvedených předpokladů každé přípustné řešení splňuje*

$$\sum_{e \in E(U)} x_e + \sum_{e \in E'} x_e \leq |U| + \frac{1}{2}(|E'| - 1). \quad (10.6)$$

Důkaz. Uvažujme nerovnosti

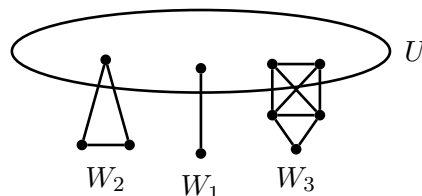
$$\begin{aligned} \sum_{e: e=\{i,j\} \in E} x_e &= 2 \quad \forall i \in U, \\ x_e &\leq 1 \quad \forall e \in E', \\ -x_e &\leq 0 \quad \forall e \in E(U, V \setminus U) \setminus E'. \end{aligned}$$

Součtem dostaneme

$$\sum_{e \in E(U)} 2x_e + \sum_{e \in E'} 2x_e \leq 2|U| + |E'|.$$

Vydělením 2 a zaokrouhlením pravé strany dolů pak máme (10.6). $\square$

Podmínku můžeme zobecnit tím, že místo hřebenu budeme uvažovat *zobecněný hřeben* (Chvátal, 1973). V něm zuby hřebenu nahradíme jakýmkoliv podgrafy $W_1, \dots, W_k$ lichého počtu tak, aby byly disjunktní a měli aspoň jeden vrchol uvnitř i vně U . Pak můžeme odvodit následující podmínku.



Věta 10.9 (Zobecněná hřebenová nerovnost). *Za výše uvedených předpokladů každé přípustné řešení splňuje*

$$\sum_{e \in E(U)} x_e + \sum_{i=1}^k \sum_{e \in E(W_i)} x_e \leq |U| + \sum_{i=1}^k |W_i| - \frac{1}{2}(3k+1). \quad (10.7)$$

Důkaz. Jednoduše odvodíme

$$\sum_{e \in E(U)} 2x_e + \sum_{i=1}^k \sum_{e \in E(U, V \setminus U) \cap E(W_i)} x_e \leq 2|U|,$$

jakožto další nerovnosti typu (10.5):

$$\begin{aligned} \sum_{e \in E(W_i)} x_e &\leq |W_i| - 1, \quad \forall i = 1, \dots, k \\ \sum_{e \in E(U \cap W_i)} x_e &\leq |U \cap W_i| - 1, \quad \forall i = 1, \dots, k \\ \sum_{e \in E(W_i \setminus U)} x_e &\leq |W_i \setminus U| - 1, \quad \forall i = 1, \dots, k. \end{aligned}$$

Součtem dostaneme

$$\sum_{e \in E(U)} 2x_e + \sum_{i=1}^k \sum_{e \in E(W_i)} 2x_e \leq 2|U| + \sum_{i=1}^k (|W_i| - 1 + |U \cap W_i| - 1 + |W_i \setminus U| - 1).$$

Protože $|W_i \setminus U| = |W_i| - |U \cap W_i|$, pravá strana nerovnosti se zjednoduší na

$$2|U| + \sum_{i=1}^k (2|W_i| - 3).$$

Vydělením 2 a zaokrouhlením pravé strany dolů pak máme (10.7). □

Protože pravá strana (10.7) se dá vyjádřit jako $|U| + \frac{1}{2}(\sum_{i=1}^k (2|W_i| - 3) - 1)$, je jasné, že se jedná o zobecnění (10.6).

Aby toho nebylo málo, můžeme zobecněné hřebeny ještě dále zobecňovat na tzv. *klikové stromy* (Grötschel–Pulleyblank, 1986). Např. tak, že zuby hřebenu budou vlastní hřebeny, nebo že se různé hřebeny navzájem propojí skrze společné zuby. Platí, že takováto zobecnění generují vesměs stěnové nerovnosti, přesto se tímto postupem k celočíselnému polyedru M_I nemusíme vůbec dopracovat.

Není znám žádný polynomiální algoritmus na generování nesplněných hřebenových nerovností. Na druhou stranu, není zatím ani známo, že by to bylo NP-těžké, takže problém je otevřen.

10.3 Relaxace a dolní odhady na optimální hodnotu

Eukleidovské TSP – kontrolní zóny

Pro eukleidovské TSP můžeme dolní odhad na optimum získat jako optimální hodnotu lineárního programu

$$\max \sum_{i=1}^n 2r_i \quad \text{za podm. } r_i + r_j \leq c_{ij} \quad \forall i, j, \quad r_i \geq 0 \quad \forall i. \quad (10.8)$$

Libovolné přípustné řešení nám vytvoří kolem každého vrcholu i kružnici s poloměrem r_i . Protože se tyto kružnice neprotínají, k navštívení vrcholu i musím projít kružnicí a urazit vzdálenost aspoň $2r_i$.

Metrické TSP

Minimální kostra grafu určitě dá dolní odhad na optimum. Můžeme ke kostře ještě přidat nejkratší (v kostře neobsaženou) hranu vycházející z libovolného listu kostry. A samozřejmě vybereme list, z něhož vycházející nejkratší hrana (mimo kostru) je co nejdelší

Jiná možnost je uvažovat tzv. minimální 1-strom. Najdeme minimální kostru na grafu bez vrcholu $v = 1$ (nebo jakéhokoli jiného) a přidáme k ní dvě nejkratší hrany vycházející z v . Hodnota takového minimálního 1-stromu typicky bývá $\approx 10\%$ pod optimum.

Přiřazovací problém

Pokud ve formulaci TSP relaxujeme podmínku (10.1c), dostaneme dopravní problém, resp. jeho speciální případ nazývaný přiřazovací problém. Dopravní problém je řešitelný pomocí lineárního programování nebo speciálních metod (např. tzv. maďarská metoda (König & Egerváry, 1931, Kuhn, 1955), která hledá největší perfektní párování v odpovídajícím bipartitním grafu iterativním zvětšováním menšího párování a s časem $\mathcal{O}(n^4)$ dává silně polynomiální algoritmus).

To, že se jedná o dopravní problém, nahlédneme snadno. Množinu vrcholů zdvojíme $V \cup V'$ a hrany povedou jen z V do V' . Pokud nastavíme kapacity výrobců i spotřebitelů na 1, a ceny jako c_{ij} , dostáváme přímo dopravní problém, viz příklad 1.13.

Přiřazovací problém pro symetrické TSP

Pro symetrické TSP, relaxovaná úloha (i bez podmínek $x_e \leq 1$) má tvar jednoduchého lineárního programu

$$\min \sum_{e \in E} c_e x_e \quad \text{za podm.} \quad \sum_{e \ni k} x_e = 2 \quad \forall k \in V, \quad x_e \geq 0.$$

Pro něj vždy existuje optimální řešení se složkami v $\{0, 1, 2\}$. Zajímavé jest, že duální úlohou jsou kontrolní zóny (10.8); podmínky $r_i \geq 0 \quad \forall i$ tak sice nevzniknou explicitně, ale platí implicitně díky trojúhelníkové nerovnosti.

Pokud přidáme omezení $x_e \leq 1$, $e \in E$, dolní odhad se nám zlepší a optimální řešení bude mít složky v $\{0, \frac{1}{2}, 1\}$.

Celočíselná relaxace podle podcest

Pokud relaxujeme pouze podmínku celočíselnosti, dostaneme také lineární program. Při podmínce (10.3) či (10.4) ale má úloha exponenciální počet nerovnic. Principiálně můžeme i takovouto úlohu řešit polynomiálně, protože dovedeme polynomiálně rozhodnout separační problém. *Separací problém* zní:

Dán konvexní polyedr M a bod $y \in \mathbb{Q}^n$. Rozhodni, zda $y \in M$, a pokud ne, tak najdi oddělující nadrovinu, tj. $a \in \mathbb{Q}^n$ takové, že $a^T x < a^T y \quad \forall x \in M$.

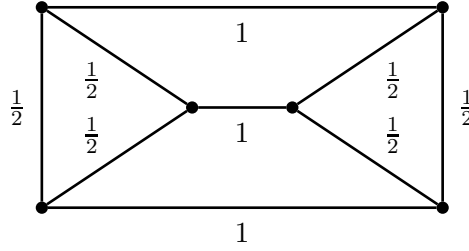
Pokud je separační problém řešitelný efektivně, pak i jakýkoli lineární program nad M , a to s využitím elipsoidového algoritmu [Schrijver, 1998].

Uvažme např. podmínku (10.3); porušenou podmínku pro separační problém najdeme pomocí efektivních algoritmů pro toky v sítích. Uvažme náš graf jako síť s kapacitami x_{ij} a najdeme maximální tok mezi vrcholy 1 a $j \in V$. Maximální tok nám dává zároveň i minimální řez. Je-li jeho hodnota alespoň 1, tak všechny podmínky z (10.3) jsou splněny, jinak minimální řez dává rovnou množinu U . Nestačí však řešit pouze jednu úlohu, ale musíme (pokud ještě nemáme porušenou podmínku) projít všechny toky z vrcholu 1 do jakéhokoli jiného, a pro nesymetrické TSP i v opačném směru.

Aproximační faktor je 1.5, to jest, optimální cesta není delší než 1.5-násobek dolní meze.

Následující příklad ukazuje, že i přes exponenciální počet podmínek celočíselná relaxace nemusí dávat celočíselné řešení.

Příklad 10.10. Uvažujme následující graf. Čísla na hranách udávají hodnotu přípustného řešení x . Hrany, které v grafu nejsou znázorněny, mají příslušnou hodnotu $x_e = 0$.



Příslušné řešení x je přípustné řešení celočíselné relaxace, tudíž splňuje všechna omezení typu (10.3) či (10.4). Nicméně, toto řešení stále ještě není celočíselné. $\square$

Lagrangeova relaxace

Potenciálně lepší horní odhad než je celočíselná relaxace nám může dát Lagrangeova relaxace (sekce 6.2). Uvažujme symetrické TSP a analogii formulace (10.4), navíc s redundantní podmínkou $\sum_e x_e = n$.

$$\begin{aligned} \min \sum_{e \in E} c_e x_e \quad \text{za podm.} \quad & \sum_{e \ni i} x_e = 2 \quad \forall i \in \{1, \dots, n\}, \\ & \sum_{e \in E(U)} x_e \leq |U| - 1, \quad \forall 1 \notin U \subset \{1, \dots, n\}, \quad 2 \leq |U| \leq n-1, \\ & \sum_{e \in E} x_e = n, \\ & x \in \{0, 1\}^{|E|}. \end{aligned}$$

Lagrangeovsky relaxujeme rovnice pro všechna $i \neq 1$ a dostaneme úlohu (P_u)

$$\begin{aligned} \min \sum_{e \in E} c_e x_e + \sum_{i=2}^n 2u_i - \sum_{e=\{i,j\} \in E, i \neq 1} (u_i + u_j) x_e \\ \text{za podm.} \quad & \sum_{e \ni 1} x_e = 2, \\ & \sum_{e \in E(U)} x_e \leq |U| - 1, \quad \forall 1 \notin U \subset \{1, \dots, n\}, \quad 2 \leq |U| \leq n-1, \\ & \sum_{e \in E} x_e = n, \\ & x \in \{0, 1\}^{|E|}. \end{aligned}$$

Přípustná řešení této úlohy jsou právě 1-stromy. Minimální 1-strom najdeme jednoduše jako minimální kostru na podgrafu na $\{2, \dots, n\}$ a přidáme dvě nejkratší hrany z vrcholu 1. Tedy úlohu (P_u) dovedeme efektivně řešit, což je v Lagrangeově relaxaci to podstatné.

Podle Důsledku 6.7 dostaneme stejný odhad jako celočíselnou relaxací. Nemusíme se ale vypořádávat s exponenciálním počtem podmínek. Proto je tato relaxace je asi nejúspěšnější pro dolní odhady TSP. Původně pochází již od autorů Held–Karp (1970, 1971).

10.4 Heuristiky

Heuristiky dávají více či méně dobré přípustné řešení a tím pádem i horní odhad na optimální hodnotu TSP. Předpokládáme zde metrické TSP. Známé heuristiky jsou např.

Nejbližší soused

Vyber počáteční vrchol. Přesuň se do nejbližšího sousedního vrcholu, z něj pak do jemu nejbližšímu sousedu (kterého jsme ještě nenavštívili) atd. dokud nepokryjeme všechny vrcholy a cestu neuza-
vřeme.

Časová složitost je $\mathcal{O}(n^2)$, aproximační faktor $\frac{1}{2}(\lceil \log(n) \rceil + 1)$ a nelze zlepšit.

Hladový algoritmus

Setříd' hrany vzestupně dle délek a postupně buduj kružnici přidáváním nejkratších hran tak, abychom neporušili přípustnost, to jest, aby nevznikla podkružnice či vrchol stupně 3.

Časová složitost je $\mathcal{O}(n^2 \log(n))$. Pro větší n se chová lépe než Nejbližší soused, aproximační faktor $\frac{1}{2}(\lceil \log(n) \rceil + 1)$.

Vkládací algoritmus

Induktivně přidávám vrcholy do již zbudované kružnice. Vkládám tak, aby podcesta vzrostla co nejméně. Přidávaný vrchol může být: ten s minimálním nárůstem cesty či ten nejbližší nějakému vrcholu na kružnici (aproximační faktor 2), anebo ten s největší vzdáleností od vrcholů na kružnici (aproximační faktor $\sim \log(n)$, ale chová se dobře a robustně) [Cook, 2012].

Popíšeme podrobněji variantu přidání nejbližšího vrcholu k nějakému vrcholu na kružnici T (Nearest insertion, Nemhauser and Wolsey [1999]). Definuj $(i^*, j^*) := \arg \min_{i \in V \setminus T, j \in T} c_{ij}$. Pak ke kružnici T přidáme hranu $\{i^*, j^*\}$ a vrchol i^* napojíme na toho výhodnějšího ze dvou sousedů vrcholu j^* .

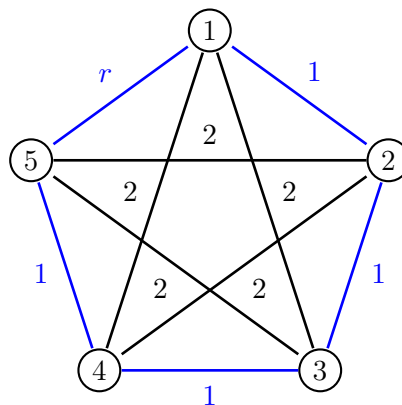
Christofides (1976)

Christofidův algoritmus najde řešení s aproximačním faktorem $3/2$. Nejprve si ukažme metodu s faktorem 2: Najdi minimální kostru a zdvojnásob její hrany, takže na nich existuje eulerovský tah (jehož délka je nanejvýš optimum TSP). Tím, že přeskočíme/vyškrtneme vícenásobná navštívení téhož vrcholu celkovou délku nezvýšíme (díky trojúhelníkové nerovnosti) a dostaneme hamiltonovskou kružnici.

Vylepšení na faktor $3/2$ provedeme tak, že k nalezené minimální kostře přidáme minimální párování (to lze najít v polynomiálním čase); minimální párování stačí hledat na množině vrcholů, které mají na kostře lichý stupeň. Faktor $3/2$ pak plyne z toho, že minimální párování má cenu maximálně poloviční jak TSP (každá hamiltonovská kružnice se rozpadá na dvě disjunktní párování).

Heuristik existuje ještě celá řada. Nejmenovali jsme třeba ještě metody: Savings (Clarke–Wright, 1964), Sweep pro eukleidovské TSP či Borůvkův algoritmus.

Příklad 10.11 (bez trojúhelníkové nerovnosti). Uvažme graf s $n = 5$, kde hrany na cestě 1-2-3-4-5 jsou ohodnoceny 1, hrana $(5, 1)$ má délku r a ostatní 2.



Metoda nejbližšího souseda iniciovaná ve vrcholu 1 najde kružnici 1-2-3-4-5-1 délky $4 + r$ (na obrázku vyznačeno modře), ale optimální je 7. Tedy pro dost velké r dostaneme libovolně špatný aproximační faktor. To je důvod proč pro heuristiky většinou předpokládáme metrické TSP. $\square$

Kromě heuristik na nalezení dobrého přípustného řešení existují i lokálně vylepšující algoritmy, které začínou s počátečním přípustným řešením a iterativně se ho pokoušejí zlepšit lokálními změnami.

k -výměna

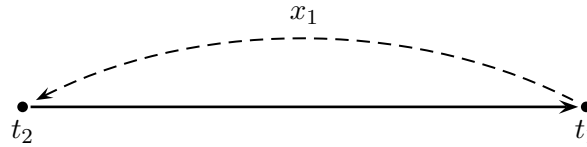
Kromě heuristik na nalezení výchozího přípustného řešení existují také lokálně vylepšující techniky. Např. k -výměna spočívá v odstranění k hran z kružnice a přidání nových k hran tak, aby vznikla zase hamiltonovská kružnice. Pokud je nová kružnice lepší, zapamatujeme si ji. Vyzkoušíme-li všechny takovéto k -výměny, dostaneme jako řešení „lokální minimum“. Počet k -výměn může být ale velký, takže se časově vyplatí jen pro malá k . Tabulka ukazuje počet možných napojení po odstranění k hran

k	2	3	4	5	6	7	8
počet přeuspořádání	1	4	20	148	1358	15104	198144

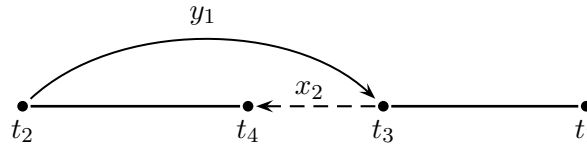
Lin–Kernighan–Helsgaun (LKH)

Namísto zkoušení všech možných k -výměn přišli Lin–Kernighan (1973) s variabilní k -výměnou, založenou na sekvenční výměně. Efektivní implementaci pak navrhl Helsgaun [2000].

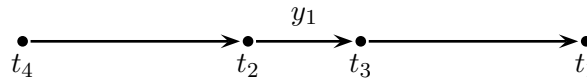
Sekvenční výměnu si lze představit takto: Z hamiltonovské kružnice odstraníme hranu $x_1 = (t_1, t_2)$ a dostaneme hamiltonovskou cestu z vrcholu t_2 do vrcholu t_1 .



Když nyní přidáme hranu $y_1 = (t_2, t_3)$, je jen jediná volba jak odstranit hranu $x_2 = (t_3, t_4)$, abychom zase dostali cestu.



Nová cesta vede z vrcholu t_4 do vrcholu t_1 .



Tento postup „přidání hrany a odstranění hrany“ opakujeme tak dlouho než uzavření cesty na kružnici nezlepší dosavadní nejlepší řešení nebo bychom se opakovali. V druhém případě backtrackujeme zpět a zkoušíme jiné volby hran.

Zefektivnit tento postup lze mnoha způsoby. Následující lemma říká, že stačí uvažovat pouze takové výběry hran y_k , aby částečné součty vah hran $\sum_{i=1}^k g_i > 0$, kde $g_i := c(x_i) - c(y_i)$.

Lemma 10.12. *Budte $g_0, \dots, g_{p-1} \in \mathbb{R}$ tak, že $\sum_{i=0}^{p-1} g_i > 0$. Pak existuje i^* pro nějž částečné součty $\sum_{i=0}^k g_{i^*+i} > 0 \ \forall k = 0, \dots, p-1$, kde indexy sčítáme modulo p .*

Důkaz. Definujme

$$\begin{aligned} r_1 &:= \min \{r \geq 0; \sum_{i=0}^r g_i \leq 0\}, \\ r_2 &:= \min \{r \geq r_1 + 1; \sum_{i=r_1+1}^r g_i \leq 0\}, \\ &\vdots \\ r_\ell &:= \min \{r \geq r_{\ell-1} + 1; \sum_{i=r_{\ell-1}+1}^r g_i \leq 0\}. \end{aligned}$$

Nyní stačí volit $i^* := r_\ell + 1$, protože podle definice posledního r_ℓ je $\sum_{i=i^*}^k g_i > 0$ pro všechna $k = i^*, \dots, p-1$. 1. Podle předpokladu je $\sum_{i=0}^{p-1} g_i > 0$, tudíž $\sum_{i=i^*}^{p-1} g_i + \sum_{i=0}^{i^*-1} g_i > 0$ pro všechna $k = 0, \dots, i^* - 1$. $\square$

Dále, výběr y_i je omezen na pět hran nejbližších minimálnímu 1-stromu. Kromě sekvenčních výměn také zkouší nesekvenční 4- a 5-výměny (tvar květu). Takováto 4-výměna se dá použít i jako popostrčení do nové vylepšující sekvence místo zkoušení nových začátků s náhodnými cestami (tzv. zřetězený algoritmus).

Uvádí se, že pro $n < 50$ najde LKH optimum skoro ve všech případech a pro $n \approx 100$ v ca 30% případů. A i pokud nenajde optimum, tak nebývá víc než 1% až 2% od optimální hodnoty. V současnosti (2011) drží rekord o nejlepší řešení World TSP, úlohy s 85 900 městy za Zemi.

Další

Simulované žihání bylo původně vymyšleno právě pro TSP a NP-těžké problémy obecně [Kirkpatrick et al., 1983].

Genetické algoritmy. Zde je otázka jak křížit jedince (=cesty). Metoda EAX (Edge-Assembly Crossover) vezme sjednocení cest, a u kružnic se střídavými hranami vezme hrany 2. cesty a nahradí jimi ty doplňkové hrany u celé 1. cesty. Varianta Nagata [2006] našla zatím nejlepší řešení u 100 000 vrcholového grafu Mony Lisy.

10.5 Historie a výpočty

Na rozdíl od problému batohu je TSP skutečně stále těžké vyřešit i pro středně velké instance. Tabulka dole ilustruje vývoj při řešení TSP na největších vyřešených instancích své doby. Čas od roku 2000 je souhrnný čas všech výpočetních jednotek při paralelním běhu.

rok	1957	1987	1998	2001	2004	2006
n	49	666	13509	15112	24978	85900
popis	státy USA	města světa	města USA	města Německa	města Švédska	LaserLogic chip
čas				23 let	84 let	

Vítězem (nejen) instance z roku 2006 je program *Concorde*, viz Cook [2011]. Program je založený na metodě branch & cut, a používají se dobré heuristiky typu Lin–Kernighan–Helsgaun.

Úloha TSP o nalezení cesty 1 904 711 městy Země z r. 2001 má Helsgaunem nalezené řešení vzdálené max. 0.047% od optima. Jedna z největších instancí TSP z r. 2003 se skládá z 526 280 881 nebeských objektů (Cook & Applegate). Zatím nejlepší nalezená cesta mezi hvězdami je nanejvýš 0.41% od optima.

Z historie

Na přelomu 19. a 20. století projíždělo Spojenými státy na 350 000 obchodních cestujících. Jeden z nich, H.M. Cleveland roku 1925 procestoval od 9.7. do 24.8. celkem 350 míst v Maine [Cook, 2012].

Plán okružní cesty zajímal i právníky navštěvující soudní okrsky ve svých obvodech. Jedním z nich byl i budoucí prezident A. Lincoln, který kolem roku 1850 spravoval 14 místních soudních dvorů v Osmém obvodu ve státě Illinois.

Podobné okružní cesty vykonávali (a svým způsobem někteří ještě vykonávají) i kazatelé.

Označení TSP pro tento problém se poprvé objevilo v tisku roku 1949 [Robinson, 1949].

Cvičení

- 10.1. Dokažte, že dimenze celočíselného polyedru (konvexní obal celočíselných přípustných bodů) z úlohy (10.1) je rovna $\frac{1}{2}n(n-3)$, a to například tak, že najdete $\frac{1}{2}n(n-3)+1$ afinně nezávislých přípustných řešení.
- 10.2. Navrhněte jak polynomiálně řešit separační problém v celočíselné relaxaci i pro exponenciální formulaci (10.4).
- 10.3. Pro symetrické TSP s $n \geq 3$ dokažte, že nerovnice (10.5) je stěnová, a to například tak, že najdete $\frac{1}{2}n(n-3)-1$ afinně nezávislých přípustných řešení.
- 10.4. Pro symetrické TSP s $n = 4$ najděte všechny stěnové nerovnosti.

10.5. Aplikujte Lagrangeovu relaxaci na úlohu TSP s tím, že relaxujete jiné podmínky.

Zejména, co se stane, když relaxujeme *všechny* rovnicové podmínky?

10.6. Aplikujte na symetrickou úlohu TSP na vrcholech $v_1, \dots, v_5$ s maticí vzdáleností

$$C = \begin{pmatrix} - & 8 & 4 & 9 & 9 \\ 8 & - & 6 & 7 & 10 \\ 4 & 6 & - & 5 & 6 \\ 9 & 7 & 5 & - & 4 \\ 9 & 10 & 6 & 4 & - \end{pmatrix}$$

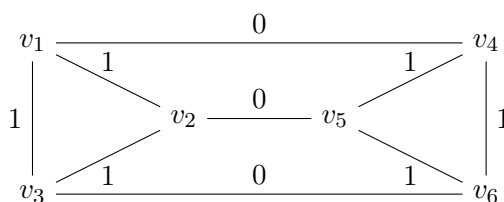
následující heuristiky:

- | | |
|--|---|
| (a) nejbližší soused (nearest neighbor), | (d) vkládání nejbližšího (nearest insertion), |
| (b) hladový algoritmus, | (e) metoda minimální kostry, |
| (c) metoda úspor (savings), | (f) metoda 2-výměn. |

10.7. Řešte Lagrangeovu relaxaci založenou na 1-stromech pro symetrické TSP s maticí vzdáleností

$$C = \begin{pmatrix} - & 10 & 2 & 4 & 6 & 2 \\ 10 & - & 9 & 3 & 1 & 3 \\ 2 & 9 & - & 5 & 6 & 1 \\ 4 & 3 & 5 & - & 2 & 5 \\ 6 & 1 & 6 & 2 & - & 3 \\ 2 & 3 & 1 & 5 & 3 & - \end{pmatrix}.$$

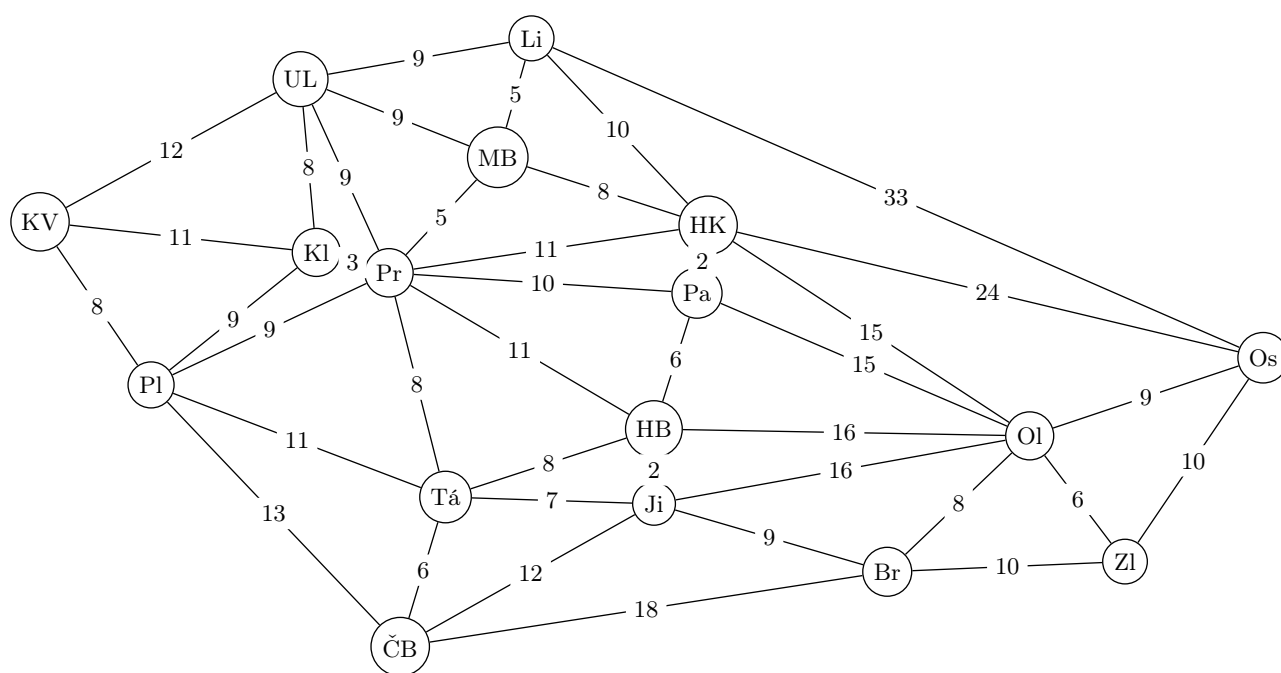
10.8. Uvažujme instanci problému obchodního cestujícího na grafu



s danými cenami na hranách, přičemž cena hrany (v_i, v_j) , která v grafu není znázorněná, je rovná ceně nejlevnější cesty mezi v_i a v_j . Najděte pro tuto úlohu hřebenovou nerovnost, která odřeže neceločíselné řešení

$$\begin{aligned} x_{14} &= x_{25} = x_{36} = 1, \\ x_{12} &= x_{23} = x_{13} = x_{46} = x_{56} = x_{45} = \frac{1}{2}, \\ (\text{pro ostatní hrany } e \text{ je } x_e &= 0). \end{aligned}$$

10.9. Najděte co nejkratší okružní cestu po městech České republiky (vzdálenosti mají za jednotku 10 km). Aplikujte různé heuristiky i přesný výpočet.



Kapitola 11

Facility location problem

Facility location problem (FLP), nebo též plant location problem a do češtiny občas překládaný jako problém umístění zařízení, se zabývá otázkou kolik a jaké ze zařízení $i = 1, \dots, m$ vybudovat, abychom uspokojili spotřebitele $j = 1, \dots, n$ a celkem nás to stálo co nejméně peněz. Konkrétně se může jednat o problém umístění nově vybudovaných nemocnic v rozvojové zemi, umístění elektráren nebo umístění skládek pro ukládání odpadu.

Označme: M_i kapacita i -tého zařízení a f_i cena jeho vybudování, c_{ij} cena za převoz jednotky zboží od i ku j , a d_j požadavek j -tého spotřebitele. Proměnná x_{ij} bude udávat množství zboží přepravované od i ku j , a binární proměnná y_i udává zda se i -té zařízení postaví či ne. Nyní úlohu můžeme formulovat jako celočíselný lineární program

$$\min \sum_{i=1}^m \sum_{j=1}^n c_{ij} x_{ij} + \sum_{i=1}^m f_i y_i \quad \text{za podm.} \quad \sum_{i=1}^m x_{ij} = d_j \quad \forall j, \quad (11.1a)$$

$$\sum_{j=1}^n x_{ij} \leq M_i \quad \forall i, \quad (11.1b)$$

$$x_{ij} \leq M_i y_i \quad \forall i, j, \quad (11.1c)$$

$$x_{ij} \geq 0 \quad \forall i, j, \quad (11.1d)$$

$$y \in \{0, 1\}^m. \quad (11.1e)$$

Podmínku (11.1b)–(11.1c) můžeme ekvivalentně nahradit jedním typem nerovností

$$\sum_{j=1}^n x_{ij} \leq M_i y_i \quad \forall i,$$

nicméně není tak výhodná. Přestože je počet omezení ve formulaci (11.1) větší, jsou silnější, a proto je formulace (11.1b)–(11.1c) výhodnější.

Kromě tohoto modelu budeme ještě uvažovat případ bez kapacit (UFLP, *uncapacitated facility location problem*). Zde stačí podmínky (11.1b)–(11.1c) nahradit podmínkou $x_{ij} \leq (\sum_{j=1}^n d_j) y_i$. Elegantnější je ale zavést si nové proměnné $z_{ij} := x_{ij}/d_j$ udávající jaký podíl požadavku j -tého spotřebitele pochází od i . Substitucí $c'_{ij} := d_j c_{ij}$ pak dostáváme model

$$\min \sum_{i=1}^m \sum_{j=1}^n c'_{ij} z_{ij} + \sum_{i=1}^m f_i y_i \quad \text{za podm.} \quad \sum_{i=1}^m z_{ij} = 1 \quad \forall j, \quad (11.2a)$$

$$z_{ij} \leq y_i \quad \forall i, j, \quad (11.2b)$$

$$z_{ij} \geq 0 \quad \forall i, j, \quad (11.2c)$$

$$y \in \{0, 1\}^m. \quad (11.2d)$$

Zde opět bychom mn podmínek v (11.2b) mohli nahradit m podmínkami

$$\sum_{j=1}^n z_{ij} \leq n y_i \quad \forall i, \quad (11.3)$$

nicméně ty původní jsou stěnové, a tudíž i mnohem vhodnější pro řadu metod. Jeden knižní zdroj uváděl numerické experimenty, kdy model s podmínkou (11.3) trval 15 hodin výpočtu, kdežto s podmínkou (11.2b) pouze 2 sekundy. Tudíž na modelování velmi záleží!

Poznámka 11.1. Buď $y \in \{0, 1\}^m$ optimální řešení v úloze UFLP. Pak optimální rozvozy z_{ij} spočítáme jednoduše hladovým algoritmem. Buď $j \in \{1, \dots, n\}$ libovolné a vyber $i^* \in \arg \min_i: y_i=1 c'_{ij}$. Pak optimální řešení je $z_{i^*j} = 1$ a $z_{ij} = 0$ pro $i \neq i^*$. Tudíž existuje optimální řešení UFLP takové, že všechny složky y_i, z_{ij} jsou binární.

Příklad 11.2. Chceme ukázat, že podmínky (11.2b) jsou stěnové pro speciální případ s $m = 1$. Přesněji řečeno, aby úloha nebyla tak degenerovaná, uvažujeme trochu jiné omezení typu

$$e^T z \leq ny, \quad z \leq e, \quad z \in \mathbb{R}^n, \quad y \in \{0, 1\},$$

a dokážeme, že podmínka $z_i \leq y$ je stěnová pro každé $i = 1, \dots, n$.

Řešení: Stačí vzít $n + 1$ bodů $(0, 0)$, $(e_i, 1)$ a $(e_i + e_j, 1)$ pro všechna $j \neq i$. Body zřejmě splňují zadaná omezení a leží v nadrovině $z_i = y$.

Nyní musíme ověřit, že jsou afinně nezávislé. Připomeňme, že body $x_0, x_1, \dots, x_n$ jsou afinně nezávislé právě tehdy, když body $x_1 - x_0, \dots, x_n - x_0$ jsou lineárně nezávislé. Volbou $x + 0 := (0, 0)$ se problém redukuje na lineární nezávislost bodů $(e_i, 1)$ a $(e_i + e_j, 1)$, $j \neq i$, což se nahlédne snadno. $\square$

Věta 11.3 (Megiddo and Tamir, 1982; Cornuéjols et al., 1990). *UFLP je NP-těžký problém.*

Důkaz. Ukážeme redukcí z vrcholového pokrytí, jehož rozhodovací verze zní: Mějme graf $G = (V, E)$, $|V| = m$, $|E| = n$ a $k \in \mathbb{N}$, a ptáme se, zda existuje množina vrcholů $U \subseteq V$ velikosti k a taková, že každá hrana má aspoň jeden konec v U .

Vytvoříme úlohu UFLP následujícím způsobem: Zařízení budou vrcholy V , spotřebitelé budou hrany E , ceny za vybudování jsou $f = e$ a ceny za dopravu jsou $c'_{ie} = 1$ pokud $i \in e$ a $c'_{ie} = 2$ jinak. Pak existuje vrcholové pokrytí velikosti nanejvýš k právě tehdy, když optimální hodnota UFLP je nanejvýš $k + n$. $\square$

11.1 Relaxace

Celočíselnou relaxací dostaneme jako vždy lineární program. Nicméně pro úlohu UFLP s podmínkami (11.3) můžeme optimum celočíselné relaxace snadno vyčíst. V tomto případě má úloha tvar

$$\begin{aligned} \min \quad & \sum_{i=1}^m \sum_{j=1}^n c'_{ij} z_{ij} + \sum_{i=1}^m f_i y_i \quad \text{za podm.} \quad \sum_{i=1}^m z_{ij} = 1 \quad \forall j, \\ & \sum_{j=1}^n z_{ij} \leq n y_i \quad \forall i, \\ & z_{ij} \geq 0 \quad \forall i, j, \\ & y_i \geq 0 \quad \forall i. \end{aligned}$$

Protože koeficienty f_i v účelové funkci jsou z podstaty problému kladné, chceme proměnné y_i co nejmenší. Tudíž se nerovnost $\sum_{j=1}^n z_{ij} \leq n y_i$ nabyde jako rovnost. Substitucí $y_i := \frac{1}{n} \sum_{j=1}^n z_{ij}$ dostaneme úlohu

$$\min \quad \sum_{i=1}^m \sum_{j=1}^n \left(c'_{ij} + \frac{1}{n} f_i \right) z_{ij} \quad \text{za podm.} \quad \sum_{i=1}^m z_{ij} = 1 \quad \forall j, \quad z_{ij} \geq 0 \quad \forall i, j.$$

Tato úloha se ekvivalentně vyjádří

$$\sum_{j=1}^n \min_{i=1}^m \left(c'_{ij} + \frac{1}{n} f_i \right) z_{ij} \quad \text{za podm.} \quad \sum_{i=1}^m z_{ij} = 1, \quad z_{ij} \geq 0 \quad \forall i, j.$$

Úloha je tedy separabilní, rozdělí se na n podúloh. Pro konkrétní $j \in \{1, \dots, n\}$ má j -tá podúloha tvar

$$\min \sum_{i=1}^m \left(c'_{ij} + \frac{1}{n} f_i \right) z_{ij} \quad \text{za podm.} \quad \sum_{i=1}^m z_{ij} = 1, \quad z_{ij} \geq 0 \quad \forall i.$$

Bud' $i^* \in \arg \min_{i=1, \dots, m} (c'_{ij} + \frac{1}{n} f_i)$. Pak optimální řešení podúlohy, a tudíž i celočíselné relaxace, je $z_{i^*j} = 1$ a $z_{ij} = 0$ pro $i \neq i^*$.

11.2 Branch & bound

Jisté specifikum metody Branch & bound v úloze FLP (11.1) může být volba proměnné $y_k \notin \mathbb{Z}$ pro větvení. Rozumné volby jsou:

- $k := \arg \max_i M_i$, protože rychle najdeme přípustné řešení.
- $k := \arg \min_i \frac{f_i}{M_i}$, protože navíc zahrneme i cenu výstavby i -tého zařízení.
- $k := \arg \min_i \frac{f_i + \sum_{j=1}^n c_{ij} d_j}{M_i}$, protože navíc zahrneme i ceny za převoz zboží.

11.3 Heuristiky pro UFLP

Aneb jak najít rychle co nejlepší přípustné řešení.

Lineární relaxace

Relaxací celočíselných podmínek $y \in \{0, 1\}^m$ na spojitě $y' \in [0, 1]^m$ se úloha redukuje na lineární program, jehož optimální hodnota dá dolní odhad na původní optimální hodnotu, a z optimálního řešení y' můžeme dostat přípustné řešení původní úlohy prostým zaokrouhlením nahoru $y := \lceil y' \rceil$. Protože ale typicky $m \ll n$, tak bude přípustné řešení $y = e$, což většinou je silně nadhodnocené.

Hladový algoritmus

Vytvářet přípustné $y \in \{0, 1\}^m$ můžeme také hladově. Na začátku položíme $y = 0$, a pak postupně přidáváme taková zařízení, která co nejvíce zmenší celkovou cenu.

Časová složitost je $\mathcal{O}(m^2 n)$, pokud si pro dané y pamatujeme i příslušné řešení z .

Lokální vylepšování

Začneme s libovolným přípustným $y \in \{0, 1\}^m$ a testujeme zda nedostaneme lepší řešení změnou jednoho bitu y , tj. přidáním či odebráním jednoho zařízení.

Příklad 11.4. Uvažujme úlohu UFLP s daty

$$C' = \begin{pmatrix} 3 & 9 & 2 & 6 \\ 5 & 9 & 7 & 6 \\ 0 & 7 & 6 & 6 \\ 6 & 7 & 4 & 0 \end{pmatrix}, \quad f = \begin{pmatrix} 3 \\ 2 \\ 3 \\ 3 \end{pmatrix}.$$

Najděte optimální řešení, optimální řešení obou relaxací, a aplikujte různé heuristiky.

Řešení: Optimální řešení je

$$y^* = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 1 \end{pmatrix}, \quad Z^* = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \end{pmatrix}.$$

a optimální cena je 17.

Relaxace s podmínkou tvaru (11.2b) vede přímo na optimum, zatímco relaxace s podmínkou (11.3) vede na

$$y^R = \begin{pmatrix} \frac{1}{4} \\ 0 \\ \frac{1}{2} \\ \frac{1}{4} \end{pmatrix}, \quad Z^R = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

s cenou 12.

Zaokrouhlením relaxovaného optima dostaneme přípustné řešení $y^1 = \lceil y^R \rceil = (1, 0, 1, 1)^T$, kterému odpovídá hodnota účelové funkce 18.

Hladový algoritmus pracuje takto: V první iteraci spočítáme ceny přípustných řešení:

$$y = (1, 0, 0, 0)^T : \text{cena je } 23,$$

$$y = (0, 1, 0, 0)^T : \text{cena je } 29,$$

$$y = (0, 0, 1, 0)^T : \text{cena je } 22,$$

$$y = (0, 0, 0, 1)^T : \text{cena je } 20.$$

Vybereme tedy poslední, nejlevnější možnost. V druhé iteraci můžeme zařízení přidat na pozice $i \in \{1, 2, 3\}$:

$$y = (1, 0, 0, 1)^T : \text{cena je } 19,$$

$$y = (0, 1, 0, 1)^T : \text{cena je } 21,$$

$$y = (0, 0, 1, 1)^T : \text{cena je } 17.$$

Vybereme opět poslední řešení a nyní můžeme zařízení přidat pouze na pozice $i \in \{1, 2\}$:

$$y = (1, 0, 1, 1)^T : \text{cena je } 19,$$

$$y = (0, 1, 1, 1)^T : \text{cena je } 18.$$

Protože se ceny zhoršily, zůstaneme u řešení $y = (0, 0, 1, 1)^T$ s cenou 17, které je shodou okolností optimální.

Pro lokální vylepšování začneme s výše uvedeným řešením $y^1 = (1, 0, 1, 1)^T$. Změnou jedné složky $0 \leftrightarrow 1$ dostaneme možná řešení

$$y = (0, 0, 1, 1)^T : \text{cena je } 17,$$

$$y = (1, 1, 1, 1)^T : \text{cena je } 20,$$

$$y = (1, 0, 0, 1)^T : \text{cena je } 18,$$

$$y = (1, 0, 1, 0)^T : \text{cena je } 21.$$

Nejlepší je tedy řešení $y = (0, 0, 1, 1)^T$ s cenou 17. V druhé iteraci prozkoumáme

$$y = (0, 1, 1, 1)^T : \text{cena je } 19,$$

$$y = (0, 0, 0, 1)^T : \text{cena je } 20,$$

$$y = (0, 0, 1, 0)^T : \text{cena je } 22.$$

Žádnou změnou složky už se tedy cena nezlepší. □

11.4 Bendersova dekompozice pro UFLP

Aplikujme postup popsany v Sekci 6.1 na model (11.2). Úlohu (11.2) přepíšeme jako

$$\min_{y \in \{0,1\}^m} \left\{ \sum_{i=1}^m f_i y_i + \min \left\{ \sum_{i=1}^m \sum_{j=1}^n c'_{ij} z_{ij}; \sum_{i=1}^m z_{ij} = 1 \forall j, z_{ij} \leq y_i \forall i, j, z_{ij} \geq 0 \forall i, j \right\} \right\}.$$

Protože je vnitřní lineární program separabilní, rozpadne se na součet n jednotlivých lineárních programů

$$\min_{y \in \{0,1\}^m} \left\{ \sum_{i=1}^m f_i y_i + \sum_{j=1}^n P_j(y) \right\}$$

kde $P_j(y)$ má tvar

$$\min \sum_{i=1}^m c'_{ij} z_{ij} \quad \text{za podm.} \quad \sum_{i=1}^m z_{ij} = 1, \quad z_{ij} \leq y_i \quad \forall i, \quad z_{ij} \geq 0 \quad \forall i. \quad P_j(y)$$

Duální úloha k $P_j(y)$ má tvar

$$\max u - \sum_{i=1}^m y_i w_i \quad \text{za podm.} \quad u - w_i \leq c'_{ij} \quad \forall i, \quad w_i \geq 0 \quad \forall i. \quad D_j(y)$$

Protože $w_i \geq 0$ a také $w_i \geq u - c'_{ij}$, a zároveň má být w_i co nejmenší, pro optimální w_i musí platit $w_i = (u - c'_{ij})^+$. Aby (u, w) byl vrchol duálního polyedru, musí rovněž $u = c'_{kj}$ pro nějaké $k \in \{1, \dots, m\}$ (jinak mohu u posunout nahoru či dolů, a w se změní také lineárně); v zásadě po dosažení $w_i = (u - c'_{ij})^+$ pak v úloze $D_j(y)$ maximalizujeme po částech lineární konkávní funkci. Tedy optimálními vrcholy mohou být jen vektory

$$(u, w_1, \dots, w_m) = (c'_{kj}, (c'_{kj} - c'_{1j})^+, \dots, (c'_{kj} - c'_{mj})^+), \quad k = 1, \dots, m.$$

Pokud $y \neq 0$, což můžeme směle předpokládat, je primární polyedr neprázdný, a tedy duální úloha $D_j(y)$ omezená. Proto nebudeme neomezené směry uvažovat a Bendersova dekompozice vede na formulaci

$$\min \sum_{i=1}^m f_i y_i + \sum_{j=1}^n \alpha_j \quad \text{za podm.} \quad \alpha_j \geq c'_{kj} - \sum_{i=1}^m (c'_{kj} - c'_{ij})^+ y_i, \quad y \in \{0,1\}^m, \quad e^T y \geq 1, \quad \alpha \in \mathbb{R}^n.$$

Přestože počet binárních proměnných zůstal stejný, počet spojitých proměnných z_{ij} se zmenšil z mn na pouhých n .

11.5 Lagrangeova relaxace pro UFLP

Lagrangeova relaxace podmínek $\sum_{i=1}^m z_{ij} = 1, j = 1, \dots, n$, modelu (11.2) vede na úlohu

$$\min \sum_{i=1}^m \sum_{j=1}^n (c'_{ij} - u_j) z_{ij} + \sum_{i=1}^m f_i y_i + \sum_{j=1}^n u_j \quad \text{za podm.} \quad z_{ij} \leq y_i \quad \forall i, j, \quad z_{ij} \geq 0 \quad \forall i, j, \quad y \in \{0,1\}^m.$$

Úloha se dá rozdělit na m nezávislých podúloh a ekvivalentně vyjádřit jako

$$\sum_{i=1}^m (P_u^i) + \sum_{j=1}^n u_j,$$

kde

$$\min \sum_{i=1}^m (c'_{ij} - u_j) z_{ij} + f_i y_i \quad \text{za podm.} \quad 0 \leq z_{ij} \leq y_i \quad \forall j, \quad y_i \in \{0,1\}. \quad (P_u^i)$$

Optimální řešení (P_u^i) se dá vyjádřit explicitně. Je-li $y_i = 0$, tak $z_{ij} = 0 \quad \forall j$. Je-li $y_i = 1$, tak pokud $c'_{ij} < u_j$, volíme $z_{ij} = 1$, jinak $z_{ij} = 0$. Optimální hodnota je pak

$$\left(\sum_{i=1}^m (c'_{ij} - u_j)^- + f_i \right)^- = \min \left(\sum_{i=1}^m \min(c'_{ij} - u_j, 0) + f_i, 0 \right).$$

Tudíž úloha (P_u) z Lagrangeovy relaxace má jednoduchý tvar.

Cvičení

Pro úlohu UFLP:

- 11.1. Dokažte, že konvexní obal množiny přípustných bodů má dimenzi $mn + m - n$.
- 11.2. Aplikujte Bendersovu dekompozici pro formulaci úlohy s podmínkou (11.3) namísto (11.2b) a porovnejte oba přístupy.
- 11.3. Aplikujte Lagrangeovu relaxaci (a porovnejte ji s klasickou celočíselnou relaxací) s tím, že se relaxují podmínky $x_{ij} \leq y_j \quad \forall i, j$.

Pro úlohu FLP:

- 11.4. Aplikujte Bendersovu dekompozici pro určitou formulaci.
- 11.5. Rozmyslete si heuristiky pro nalezení dobrého přípustného řešení, zejména hladový algoritmus a lokální vylepšování a jejich časovou složitost.
- 11.6. Diskutujte metodu branch & bound (volby proměnné na větvení apod.).
- 11.7. Aplikujte Lagrangeovu relaxaci (a porovnejte ji s klasickou celočíselnou relaxací). Které podmínky jsou nejvýhodnější k relaxaci?

Kapitola 12

Softwarové úlohy

Pro následující problémy formulujte úlohu celočíselného lineárního programování a najděte její optimální řešení. K modelování a řešení úlohy můžete využít modelovací/programovací jazyk a řešič podle vlastního výběru.

Cvičení

12.1. Vyřešte cvičení 1.27.

12.2. Určete barevnost (tj. nejmenší počet barev potřebný k obarvení vrcholů tak, aby sousední vrcholy neměly stejnou barvu) a příslušné obarvení vrcholů grafu $G = (V, E)$ s množinou vrcholů $V = \{1, \dots, 8\}$ a množinou hran

$$E = \{(1, 3), (1, 4), (1, 5), (1, 8), (2, 3), (2, 6), (2, 8), (3, 5), (3, 8), (4, 6), (5, 7), (5, 8), (6, 8), (7, 8)\}.$$

12.3. Určete maximální nezávislou množinu v grafu $G = (V, E)$, kde

$$E = \{(1, 3), (1, 5), (1, 6), (1, 8), (2, 3), (2, 4), (2, 6), (3, 6), (3, 7), (4, 5), (4, 7), (5, 8), (6, 7), (6, 8)\}.$$

Maximální nezávislá množina je maximální (co do počtu vrcholů) množina vrcholů takových, že žádné dva nejsou spojeny hranou.

12.4. Určete maximální párování ve váženém úplném bipartitním grafu (U, V) (tj. hrany vedou právě mezi vrcholy z U a V), kde $U = \{1, \dots, 5\}$, $V = \{1, \dots, 7\}$ a hrany jsou váženy takto (matice vah mezi U a V):

$$\begin{pmatrix} 12 & 3 & 8 & 26 & 4 & 16 & 2 \\ 30 & 25 & 9 & 3 & 0 & 17 & 5 \\ 0 & 14 & 28 & 20 & 7 & 22 & 34 \\ 40 & 35 & 0 & 33 & 13 & 25 & 35 \\ 11 & 53 & 19 & 0 & 31 & 20 & 13 \end{pmatrix}.$$

12.5. České mince mají následující váhy (typ a váha mincí z roku 2008):

mince	50 hal.	1 Kč	2 Kč	5 Kč	10 Kč	20 Kč	50 Kč
hmotnost	0.9 g	3.6 g	3.7 g	4.8 g	7.62 g	8.43 g	9.7 g

Určete, kolik nejméně má Karel mincí v kasičce, pokud víme, že

- (a) v kasičce je celkem 25 Kč a váží nejméně 36 g,
- (b) v kasičce je celkem 55 Kč a váží nejméně 70 g.

12.6. Firma Karlovy trubky s.r.o. naskladnila nové kovové trubky standardní délky 4 m. Pro aktuální stavební projekt obdržela firma následující objednávku:

délka (<i>cm</i>)	70	130	260
množství (ks)	640	500	200

Určete minimální množství 4 *m* trubek potřebných ke splnění objednávky a rozhodněte, jaký je optimální způsob nařezání trubek.

12.7. Vyřešte následující úlohu Sudoku pomocí celočíselného lineárního programování:

4		5						
			7		2	6		
3	6				1		9	
								5
	1					7		
						5		
			9	8		1	2	7
		9	5	4			3	

Značení

Množiny

$\mathbb{N}, \mathbb{Z}, \mathbb{Q}, \mathbb{R}$	množina přirozených, celých, racionálních a reálných čísel
$\mathbb{R}_+$	množina reálných nezáporných čísel
$\mathbb{Z}_+$	množina celých nezáporných čísel
$U + V$	součet množin, $U + V = \{u + v; u \in U, v \in V\}$
$\text{int } M$	vnitřek množiny M , tj. největší otevřená podmnožina M
$\text{conv}(M)$	konvexní obal množiny M , $\text{conv}(M) = \{\sum_{i=1}^k \alpha_i x_i; x_i \in M, \alpha_i \geq 0, \sum_{i=1}^k \alpha_i = 1, k \in \mathbb{N}\}$
$\text{conv cone}(M)$	konvexní kuželový obal množiny M , $\text{conv cone}(M) = \{\sum_{i=1}^k \alpha_i x_i; x_i \in M, \alpha_i \geq 0, k \in \mathbb{N}\}$

Matice a vektory

A^T	transpozice matice A
A_{i*}	i -tý řádek matice A
A_{*j}	j -tý sloupec matice A
A_B	podmatice matice A , tvořená sloupci dle množiny bázeckých indexů B
A_N	podmatice matice A , tvořená sloupci dle množiny nebázeckých indexů N
$\text{rank}(A)$	hodnost matice A
$\det(A)$	determinant matice A
$\text{adj}(A)$	adjungovaná matice, prvky jsou $\text{adj}(A)_{ij} = (-1)^{i+j} \det(A^{ji})$, kde A^{ji} je matice A bez j -tého řádku a i -tého sloupce
$\text{diag}(v)$	diagonální matice s diagonálními prvky $v_1, \dots, v_n$
$0_n, 0$	nulová matice (všechny složky jsou rovny 0)
1_n	jedničková matice (všechny složky jsou rovny 1)
I_n, I	jednotková matice (diagonální s jedničkami na diagonále)
e_i	jednotkový vektor, $e_i = I_{*i} = (0, \dots, 0, 1, \dots, 0)^T$
e	vektor ze samých jedniček, $e = (1, \dots, 1)^T$
$x \succ y$	vektor x je lexikograficky větší než vektor y , viz definice 3.1
$x \succeq y$	vektor x je lexikograficky větší nebo roven vektoru y , viz definice 3.1

Čísla

$\lfloor r \rfloor$	(dolní) celá část reálného čísla, $\lfloor r \rfloor = \max\{z \in \mathbb{Z}; z \leq r\}$
$\lceil r \rceil$	horní celá část reálného čísla, $\lceil r \rceil = \min\{z \in \mathbb{Z}; z \geq r\}$
$\{r\}$	zlomková část reálného čísla, $\{r\} = r - \lfloor r \rfloor$
r^+	kladná část reálného čísla, $r^+ = \max(r, 0)$
r^-	záporná část reálného čísla, $r^- = \max(-r, 0)$
$\text{sgn}(p)$	znaménko permutace p (je-li p permutace)
$\text{sgn}(r)$	znaménko čísla r (je-li r reálné číslo), tj. $\text{sgn}(r) = 1$ pro $r \geq 0$ a $\text{sgn}(r) = -1$ jinak

Významový slovníček

slovo	význam
bud'te	necht' ... jsou
jakybysmet	stejně tak, rovněž
jež	která
necht'	předpokládejme, že
vesměs	veskrze
veskrze	do jednoho
však	ale
těž	také
tudíž	proto

Literatura

- T. Achterberg. Scip: Solving constraint integer programs. *Math. Program. Comput.*, 1(1):1–41, 2009. <http://mpc.zib.de/index.php/MPC/article/view/4>. 58
- T. Achterberg and T. Berthold. Improving the feasibility pump. *Discrete Optim.*, 4(1):77 – 86, 2007. <http://dx.doi.org/10.1016/j.disopt.2006.10.004>. 63
- W. P. Adams. *The mixed-integer bilinear programming problem with extensions to zero-one quadratic programs*. PhD thesis, Department of Industrial Engineering and Operations Research, Virginia Polytechnic Institute and State University, Blacksburg, VA, USA, 1985. <http://hdl.handle.net/10919/74711>. 33
- W. P. Adams and H. D. Serali. A tight linearization and an algorithm for zero-one quadratic programming problems. *Manage. Sci.*, 32(10):1274–1290, 1986. 33
- W. P. Adams and H. D. Serali. Linearization strategies for a class of zero-one mixed integer programming problems. *Oper. Res.*, 38(2):217–226, 1990. 33
- D. L. Applegate, R. E. Bixby, V. Chvátal, and W. J. Cook. *The traveling salesman problem. A computational study*. Princeton University Press, 2006. 89
- S. Arora. Polynomial time approximation schemes for euclidean tsp and other geometric problems. pages 2–11. IEEE Computer Society, 1996. 90
- A. Atamtürk and M. W. P. Savelsbergh. Integer-Programming Software Systems. *Ann. Oper. Res.*, 140(1):67–124, 2005. <http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.104.8697&rep=rep1&type=pdf>. 60
- J. Benders. Partitioning procedures for solving mixed-variables programming problems. *Numer. Math.*, 4:238–252, 1962. 65
- J. Benders. Partitioning procedures for solving mixed-variables programming problems. reprint. *Comput. Manag. Sci.*, 2(1):3–19, 2005. 65
- T. Berthold. Primal heuristics for mixed integer programs. Master’s thesis, Technischen Universität Berlin, Germany, September 2006. <http://opus4.kobv.de/opus4-zib/frontdoor/index/index/docId/1029>. 63
- T. Berthold. Heuristics of the branch-cut-and-price-framework SCIP. Technical Report 07-30, ZIB, Berlin, 2007. <http://opus4.kobv.de/opus4-zib/frontdoor/index/index/docId/1028/>. 63
- N. Boland and T. Surendonk. A column generation approach to delivery planning over time with inhomogeneous service providers and service interval constraints. *Ann. Oper. Res.*, 108(1-4):143–156, 2001. 74
- M. R. Bussieck and S. Vigerske. *MINLP Solver Software*. Wiley, 2011. <http://www.math.hu-berlin.de/~stefan/minlpsoft.pdf>. 60
- J. J. Cochran et al. *Wiley Encyclopedia of Operations Research and Management Science*. Wiley, 2011. 73

- W. J. Cook. Concorde TSP Solver. <http://www.tsp.gatech.edu/concorde/>, 2011. 98
- W. J. Cook. *In Pursuit of the Traveling Salesman. Mathematics at the Limits of Computation*. Princeton University Press, 2012. 89, 90, 96, 98
- G. Cornuéjols. Valid inequalities for mixed integer linear programs. *Math. Program.*, 112:3–44, 2008. 49
- G. Cornuéjols, G. L. Nemhauser, and L. A. Wolsey. The uncapacitated facility location problem. In P. Mirchandani and R. Francis, editors, *Discrete Location Theory*, pages 119–171. Wiley, New York, 1990.
http://www.math.uwaterloo.ca/~jcheriya/PS_files/ln-uf1.ps.gz. 102
- R. J. Dakin. A tree-search algorithm for mixed integer programming problems. *Comput. J.*, 8:250–255, 1965. 53
- J. Edmonds. Paths, trees, and flowers. *Canad. J. Math.*, 17:449–467, 1965.
<http://www.cs.berkeley.edu/~christos/classics/edmonds.ps>. 51
- U. Feige. A threshold of $\ln n$ for approximating set cover. *J. ACM*, 45(4):634–652, 1998. 88
- R. E. Gomory. Outline of an algorithm for integer solutions to linear programs. *Bull. Amer. Math. Soc.*, 64:275–278, 1958. 40
- R. E. Gomory. An algorithm for the mixed integer problem. Technical Report RM-2597, The Rand Corporation, 1960. 46
- M. Grötschel, editor. *Optimization Stories*. German Mathematical Society, 2012. Extra Volume of Documenta Mathematica. 3, 90
- K. Helsgaun. An effective implementation of the Lin–Kernighan traveling salesman heuristic. *Eur. J. Oper. Res.*, 126(1):106–130, 2000. 97
- S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. *Sci.*, 220(4598):671–680, 1983. 98
- A. A. Korbut and J. J. Finkelstein. *Diskrete Optimierung*. Akademie-Verlag, Berlin, 1971. 3
- A. H. Land and A. G. Doig. An automatic method of solving discrete programming problems. *Econometrica*, 28:497–520, 1960. 53, 55
- M. Laurent. A comparison of the Sherali-Adams, Lovász-Schrijver, and Lasserre relaxations for 0-1 programming. *Math. Oper. Res.*, 28(3):470–496, 2003. 33
- J. D. C. Little, K. G. Murty, D. W. Sweeney, and C. Karel. An algorithm for the traveling salesman problem. *Oper. Res.*, 11(6):972–989, 1963. 53
- F. Margot. Symmetry in integer linear programming. In M. Jünger et al., editor, *50 Years of Integer Programming 1958-2008*, pages 647–686. Springer, Berlin, 2010.
<https://student-3k.tepper.cmu.edu/gsiadoc/wp/2008-E37.pdf>. 60
- N. Megiddo and A. Tamir. On the complexity of locating linear facilities in the plane. *Oper. Res. Lett.*, 1(5):194–197, 1982.
<http://theory.stanford.edu/~megiddo/pdf/complexity%20of%20locating%20linear%20facilities.pdf>. 102
- J. S. B. Mitchell. Guillotine subdivisions approximate polygonal subdivisions: A simple polynomial-time approximation scheme for geometric tsp, k-mst, and related problems. *SIAM J. Comput.*, 28(4):1298–1309, 1999. 90
- Y. Nagata. New EAX crossover for large TSP instances. In T. Runarsson, H.-G. Beyer, E. Burke, J. Merelo-Guervós, L. Whitley, and X. Yao, editors, *Parallel Problem Solving from Nature - PPSN IX*, volume 4193 of *LNCS*, pages 372–381. 2006. 98

- G. L. Nemhauser and L. A. Wolsey. *Integer and Combinatorial Optimization*. Wiley, New York, 1999. 3, 92, 96
- M. Padberg and G. Rinaldi. Optimization of a 532-city symmetric traveling salesman problem by branch and cut. *Oper. Res. Lett.*, 6(1):1–7, 1987. 58
- C. H. Papadimitriou and S. Vempala. On the approximability of the traveling salesman problem. *Combinatorica*, 26(1):101–120, 2006. 90
- J. B. Robinson. On the Hamiltonian game (a traveling-salesman problem). Technical Report RM303, RAND Research Memorandum, The RAND Corporation, 1949. 98
- T. Rothvoß. The Lasserre hierarchy in Approximation algorithms, 2013. Lecture Notes for the MAPSP 2013 Tutorial, <https://sites.math.washington.edu/~rothvoss/lecturenotes/lasserresurvey.pdf>. 33
- A. Schrijver. *Theory of linear and integer programming. Repr.* Wiley, Chichester, 1998. 3, 16, 81, 82, 94
- H. D. Sherali and W. P. Adams. *A reformulation-linearization technique for solving discrete and continuous non-convex problems*, volume 31 of *Nonconvex Optim. Appl.* Kluwer, Boston, 1999. 27
- H. A. Taha. *Integer programming. Theory, applications, and computations*. Academic Press, New York, 1975. 3
- D. Wojtczak. On strong NP-completeness of rational problems. In F. V. Fomin and V. V. Podolskii, editors, *Computer Science - Theory and Applications. CSR 2018*, volume 10846 of *LNCS*, pages 308–320, Cham, 2018. Springer. https://doi.org/10.1007/978-3-319-90530-3_26. 78
- L. A. Wolsey. *Integer Programming*. Wiley, Chichester, 1998. 3, 70, 73, 81